

Birzeit University
Faculty of Engineering & Technology
Department of Electrical & Computer Engineering

Heart Guard

An AI-Based System for ECG Image Digitisation and Cardiac Diagnosis

Prepared by:

- Robert Bannoura	1210574
- Mahmoud Awad	1212677
- Johny Bajjali	1210566

Supervised by:

Dr. Hanna Bullata

A Graduation Project submitted to the Department of Electrical and Computer Engineering in partial fulfilment of the requirements for the degree of Bachelor in Computer Engineering

Birzeit

February 2026

Abstract

Cardiovascular diseases remain a leading cause of mortality worldwide, and electrocardiograms (ECGs) are among the first-line tools used in emergency and acute-care workflows. In practice, ECG interpretation can be delayed when cardiologists are not immediately available, and many hospitals still handle ECGs primarily as paper printouts that are later scanned or photographed. This project presents **Heart Guard**, an end-to-end AI-based system that accepts ECGs as either **images** (paper/scan/photo) or **digital signals**, standardises them into a consistent representation, and produces **interpretable diagnostic decision support**.

Heart Guard combines two core components. First, an ECG digitisation module formulates digitisation as a **segmentation-driven extraction** problem. We generate large-scale synthetic ECG pages from the PTB-XL dataset and produce pixel-accurate ground-truth masks from rendering metadata to train nnU-Net models. To address the synthetic-to-real domain gap observed on hospital scans, we additionally build an internal **Flask web-based ECG labeling tool** and manually trace 100 real ECG pages to create aligned binary masks for mixed-data training. The final binary 3×4 segmentation model achieved strong performance with a five-fold average mean validation Dice of **0.8390**, while multi-class segmentation (0–12) remained unstable (mean Dice $\approx$ **0.4116**), motivating the final use of binary masks for robust waveform extraction.

Second, Heart Guard integrates **ProtoECGNet**, a prototype-based deep learning model for **interpretable multi-label ECG classification**. Using PTB-XL (100 Hz) with the official stratified split (folds 1–8 train, 9 validation, 10 test), the fused model achieved a macro-AUROC of **0.9232** and macro-F1 of **0.55** on validation, while providing case-based explanations grounded in learned prototypes. The complete system is deployed as a **Flutter** client (mobile/Windows) with a **Django** backend that orchestrates preprocessing, segmentation, digitisation, inference, and storage, enabling case creation and historical result retrieval.

Index Terms— ECG digitisation, medical image segmentation, nnU-Net, prototype-based learning, multi-label ECG classification, clinical decision support, Flutter, Django.

المستخلص

يُعد تخطيط القلب الكهربائي **(ECG)** من أكثر الفحوصات استخدامًا لتقييم اضطرابات نظم القلب وأمراضه، إلا أن جزءًا كبيرًا من السجلات الطبية في المستشفيات ما زال محفوظًا على شكل صفحات ورقية أو صور غير رقمية، مما يعيق التحليل الآلي، والمقارنة عبر الزمن، وإعادة الاستخدام البحثي. يقدّم هذا المشروع نظامًا متكاملًا باسم **Heart Guard** يهدف إلى قراءة إشارات تخطيط القلب الملتقطة من صور التقارير الطبية وتحويلها إلى بيانات رقمية قابلة للمعالجة، إضافةً إلى تقديم تشخيص مبدئي مدعوم بالذكاء الاصطناعي.

يعتمد النظام على منهجية تبدأ بتحضير صورة التخطيط وتحسينها، ثم تحديد مسارات الموجات القلبية وعزلها باستخدام نماذج الذكاء الصناعي للتجزئة، مع دعم تنسيقات التخطيط الشائعة مثل (3×4) مع شريط الإيقاع. بعد ذلك تُجرى مرحلة قراءة الإشارة عبر تحويل المسار المرئي إلى سلسلة زمنية ممثلة للجهد الكهربائي مع مراعاة معايرة الورق الطبي (سرعة 25 مم/ث، وكسب 10 مم/ملّي فولت) لتقريب القيم الفيزيائية الحقيقية. كما يتضمن النظام آليات معالجة لاحقة لتقليل الضجيج وإصلاح الانقطاعات، ثم حفظ النتائج بصيغ قياسية تسهّل استخدامها في التدريب والتحليل.

ولأغراض التشخيص، تُمرَّر الإشارات الرقمية الناتجة إلى نموذج تصنيف قائم على التعلم العميق لإنتاج تنبؤات بالحالات القلبية المحتملة، مع واجهة تشغيل تساعد على إدارة البيانات، وتتبع النتائج، وتجهيز مخرجات قابلة للاستخدام السريري والبحثي. تم تقييم النظام على مجموعة من سجلات **ECG** حقيقية، وأظهرت النتائج أن النهج المقترح قادر على استخراج إشارات متماسكة بصريًا وزمنيًا من صور المستشفى، مع تحسن ملحوظ عند استخدام أقنعة ثنائية أو متعددة الفئات حسب جودة البيانات. يساهم هذا العمل في سد الفجوة بين السجلات الورقية والتحليل الذكي، ويُعد خطوة عملية نحو أتمتة أرشفة تخطيط القلب ودعم القرار الطبي في البيئات ذات الموارد المحدودة.

Contents

1 Introduction **1**

1.1 Problem Statement 1

1.2 Research Goals 2

1.3 Organization of the Report 2

2 Literature Review **3**

2.1 Overview and Motivation 3

2.2 Deep Learning for ECG Interpretation 4

2.3 Limitations of Post-Hoc Explainability in Clinical AI 5

2.4 Prototype-Based Learning for Interpretability 6

2.5 Multi-Label Classification in ECG Analysis 6

2.6 Toward Clinically Useful and Trustworthy ECG AI 7

2.7 ECG Digitisation from Images Using Segmentation-Based Pipelines . . . 8

2.7.1 nnU-Net and Robust Medical Image Segmentation 8

2.7.2 Benchmarking and Recent Trends 9

2.7.3 Relevance to the Heart Guard Project 9

3 System Overview and Methodology **10**

3.1 System Overview 10

3.1.1 Main System Capabilities 11

3.1.2 End-to-End Pipeline Overview 11

3.2 Methodology and Pipeline Stages 12

3.2.1 Stage 1: ECG Acquisition and Input Handling 12

3.2.2 Stage 2: Case Creation, Persistence, and History Retrieval 12

3.2.3 Stage 3: Image Preprocessing and Standardisation 12

3.2.4 Stage 4: Segmentation and Digitisation (Image Inputs) 13

3.2.5 Stage 5: Signal Reader and Standardisation (Signal Inputs) 13

3.2.6 Stage 6: Preprocessing for Inference 13

3.2.7 Stage 7: Diagnostic Inference and Output Generation 14

3.2.8 Stage 8: Result Delivery and Visualisation 14

3.3 System Architecture 14

3.4 Chapter Summary 15

4 Data and Dataset Preparation **16**
4.1 Clinical Context and Motivation 17
4.2 Why We Needed an Online ECG Database 17
4.3 What We Looked For in a Database 17
4.4 Survey of Public ECG Datasets 18
4.4.1 Motivation for Choosing PTB-XL 19
4.5 Overview of the PTB-XL Dataset 19
4.5.1 Labeling and Clinical Categories 19
4.6 Local Clinical Data Collection 20
4.6.1 Contacting Hospitals and Approvals 20
4.6.2 Survey of ECG Machines and Practical Output Constraints . . . 21
4.6.3 Site Outcomes and Collected Data 21
4.6.4 Overall Local Dataset Summary 25
4.6.5 Collaboration With Clinicians 26
4.7 Dataset Preparation and Standardisation 26
4.7.1 Digitisation Dataset Preparation (Synthetic ECG Images + Masks) 26
4.7.2 Diagnosis Dataset (ProtoECGNet) Preparation 33
4.8 Final Datasets Statistics and Splits 35
4.8.1 Segmentation Datasets (nnU-Net) 36
4.8.2 ProtoECGNet Dataset Preparation and Splits 38

5 Segmentation Models and Training Strategy **40**
5.1 Problem Setting and Key Challenge: Domain Gap 41
5.2 Baseline Model: U-Net 42
5.2.1 Why U-Net? 42
5.2.2 U-Net architecture overview 42
5.2.3 Baseline implementation used in this work 42
5.3 Experiment 1: U-Net Trained on Purely Synthetic Data 43
5.3.1 Training behaviour 43
5.3.2 Qualitative output on synthetic data 43
5.3.3 Failure on real ECG scans (domain gap) 43
5.4 Why We Moved to nnU-Net 44
5.4.1 What nnU-Net is (and why it helps) 44
5.5 Experiment 2: nnU-Net with Augmentation (Limited Real Labels) 44
5.5.1 Motivation 44
5.5.2 Qualitative improvement (still imperfect) 45
5.6 Real Data Labelling: Flask-Based Annotation Tool 45

5.7 Final Binary Segmentation Model (3×4 Layout): Training on Mixed Data 45
5.7.1 Dataset composition and validation protocol . . . 45
5.7.2 Cross-validation evidence from training logs . . . 45
5.7.3 Cross-validation results (Mean Dice) . . . 47
5.7.4 Qualitative final prediction on real data . . . 47
5.8 Attempted Multi-Class Segmentation (0–12): . . . 47
5.9 Binary vs. Multi-Class Segmentation: Trade-offs for ECG Waveform Extraction . . . 48
5.10 Summary of Model Progression . . . 49

6 ECG Digitisation and Quantitative Evaluation 51
6.1 Digitisation Overview: Binary Mask to 12 Leads . . . 52
6.2 Pre-Digitisation Mask Cleaning . . . 52
6.3 ROI Splitting into 12 Lead Cells (3×4) . . . 53
6.4 Trace Extraction Inside Each ROI . . . 54
6.4.1 Baseline estimation . . . 54
6.4.2 Two extraction modes used in this work . . . 54
6.4.3 Handling missing columns without forcing straight lines . . . 54
6.4.4 Calibration block detection and removal . . . 55
6.4.5 Debug overlays . . . 55
6.5 Pixel-to-mV Conversion and Resampling . . . 55
6.5.1 Safe smoothing (optional, mild) . . . 56
6.6 Quantitative Evaluation Against Reference aECG (HL7) . . . 56
6.6.1 Time-window matching (critical) . . . 56
6.6.2 Alignment: best shift based on morphology . . . 56
6.7 Evaluation Metrics and Their Meaning . . . 57
6.7.1 Shape similarity: correlation $r(z)$ . . . 57
6.7.2 Amplitude accuracy: MAE and RMSE (mV) . . . 57
6.7.3 Pixel consistency (measured correctly on ROI grid) . . . 57
6.7.4 Calibration scale used (`px`) . . . 58
6.8 What Improved Digitisation Quality in Practice . . . 58
6.8.1 Why top/farthest selection failed . . . 58
6.9 Qualitative Output: Digitised 12-Lead Sheet . . . 59
6.10 Reporting of Digitisation Logs . . . 59

7 HeartGuard Classification Model: ProtoECGNet-Based Architecture 60
7.1 Background and Motivation . . . 62
7.2 ProtoECGNet Overview . . . 63
7.3 Model Architecture (Theory) . . . 64

7.3.1 1D Rhythm Branch . . . 64
7.3.2 2D Morphology Branch . . . 64
7.3.3 2D Global Branch . . . 65
7.3.4 Prototype Layer and Similarity Computation . . . 65
7.4 Loss Function and Training Objective . . . 66
7.4.1 Binary Cross-Entropy Loss . . . 67
7.4.2 Clustering Loss . . . 67
7.4.3 Separation Loss . . . 67
7.4.4 Diversity (Orthogonality) Loss . . . 67
7.4.5 Contrastive Loss Based on Label Co-Occurrence . . . 68
7.5 Training Procedure . . . 68
7.5.1 Stage 1: Joint Training . . . 68
7.5.2 Stage 2: Prototype Projection . . . 69
7.5.3 Stage 3: Fusion Classifier Training . . . 69
7.6 Implementation Details: Mapping the Paper to the Codebase . . . 69
7.6.1 Backbone Networks . . . 70
7.6.2 Data Loading and Preprocessing . . . 70
7.6.3 Label Grouping and Clinical Decomposition . . . 70
7.6.4 Prototype-Based Model Definitions . . . 70
7.6.5 Prototype Projection . . . 70
7.6.6 Fusion Mechanism . . . 71
7.6.7 Training Logic and Optimization . . . 71
7.6.8 Execution and Experiment Management . . . 71
7.6.9 Experimental Structure and Model Export . . . 71
7.7 Results and Discussion . . . 72
7.7.1 Evaluation Metrics . . . 72
7.7.2 Quantitative Results . . . 72
7.7.3 Discussion . . . 72
7.8 Interpretability and Case-Based Explanations . . . 73
7.8.1 Prototype-Based Explanation Mechanism . . . 73
7.8.2 Clinical Case-Based Examples . . . 74
7.8.3 Clinical Relevance and Trustworthiness . . . 74

8 Application and Backend Implementation (Flutter + Django) 75
8.1 Overview and System Architecture . . . 76
8.2 Frontend Technology: Flutter . . . 76
8.2.1 What is Flutter . . . 76
8.2.2 Why Flutter was Selected . . . 76
8.2.3 Flutter Project Structure and API Communication . . . 76

8.3 Flutter Application Implementation . . . 77
8.3.1 UI Structure, Navigation, and Theme Support . . . 77
8.3.2 Patient Management Module . . . 80
8.3.3 Patient Dashboard: Notes, ECG Intake, and Case History . . . 82
8.3.4 ECG Upload Workflow (Signal vs. Image) . . . 82
8.3.5 Result Visualization and Interpretability . . . 84
8.3.6 12-Lead ECG Reference Gallery . . . 87
8.3.7 Cross-Platform Deployment and Manual ECG Cropping . . . 89
8.4 Backend Technology: Django (Django REST Framework) . . . 91
8.4.1 What is Django . . . 91
8.4.2 Why Django and DRF were Selected . . . 91
8.4.3 Database Design (PostgreSQL) . . . 91
8.4.4 File Storage and Media Organization . . . 91
8.4.5 REST API Endpoints and Feature Integration . . . 92
8.4.6 End-to-End Processing Pipeline . . . 92
8.4.7 Robustness, Validation, and Error Handling . . . 93
8.5 Summary . . . 94

9 Results, Discussion, Limitations 95
9.1 Experimental Setup and Evaluation Protocol . . . 96
9.1.1 Datasets Used for Evaluation . . . 96
9.1.2 Evaluation of ECG Segmentation and Digitisation . . . 96
9.1.3 Evaluation of Diagnostic Classification . . . 97
9.1.4 Physician Comparison and Interpretation Protocol . . . 97
9.1.5 Scope of Evaluation . . . 98
9.2 Segmentation and Digitisation Results . . . 98
9.2.1 Quantitative Segmentation Results . . . 98
9.2.2 Qualitative Digitisation Results . . . 99
9.3 Diagnostic Classification Results (ProtoECGNet) . . . 100
9.3.1 Quantitative Classification Performance . . . 100
9.3.2 Label-Level and Case-Level Predictions . . . 101
9.4 Comparison with Physician Annotations . . . 102
9.4.1 Case-by-Case Agreement Analysis . . . 103
9.4.2 Direct vs. Indirect Label Mapping . . . 108
9.5 Discussion . . . 108
9.6 Limitations . . . 110
9.6.1 Limited Availability of Labelled Real ECG Images . . . 110
9.6.2 Absence of Paired Signal–Image Ground Truth . . . 111
9.6.3 Dependence on PTB-XL for Diagnostic Training . . . 111

9.6.4 Device and Acquisition Variability . . . 111
9.6.5 Lack of Prospective Clinical Validation . . . 111
9.6.6 Summary . . . 112

10 Conclusion and Future Work **113**
10.1 Conclusion . . . 114
10.2 Project Contributions . . . 114
10.2.1 System-Level Contributions . . . 115
10.2.2 Digitisation Contributions . . . 115
10.2.3 Diagnosis and Interpretability Contributions . . . 115
10.3 Limitations . . . 115
10.3.1 Domain Gap and Limited Real Labels . . . 116
10.3.2 Layout Coverage . . . 116
10.3.3 Digitisation Evaluation Constraints . . . 116
10.3.4 Clinical Validation and Regulatory Readiness . . . 116
10.3.5 Diagnosis Generalisation to Local Population . . . 116
10.4 Future Work . . . 116
10.4.1 Scaling Real Mask Collection and Active Learning . . . 117
10.4.2 Automatic Layout Detection and Generalised Digitisation . . . 117
10.4.3 Improved Signal Reconstruction . . . 117
10.4.4 Diagnosis: Local Fine-Tuning and External Validation . . . 117
10.4.5 Explainability Improvements for Clinical Use . . . 118
10.4.6 Deployment, Security, and MLOps . . . 118
10.5 Ethical Considerations and Intended Use . . . 119
10.6 Closing Remarks . . . 119

Acronyms and Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the Curve
CAD	Computer-Aided Diagnosis
CNN	Convolutional Neural Network
CSV	Comma-Separated Values
DICOM	Digital Imaging and Communications in Medicine
DL	Deep Learning
ECG	Electrocardiogram
EHR	Electronic Health Record
FN	False Negative
FP	False Positive
FPS	Frames Per Second
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HL7	Health Level Seven
HTTP	Hypertext Transfer Protocol
I/O	Input/Output
IoU	Intersection over Union
JSON	JavaScript Object Notation
LSTM	Long Short-Term Memory
ML	Machine Learning
MSE	Mean Squared Error
NN	Neural Network
PNG	Portable Network Graphics
PPG	Photoplethysmography
PR	Precision–Recall
RAM	Random Access Memory
ReLU	Rectified Linear Unit
ROC	Receiver Operating Characteristic
RNN	Recurrent Neural Network
R–R	R-to-R Interval
RTP	Real-Time Processing
SNR	Signal-to-Noise Ratio
SOTA	State of the Art

List of Figures

2.1 Deep learning-based ECG rhythm and beat features for heart abnormality classification (example architecture illustration). 5
2.2 Workflow of prototype learning: extracting features, comparing to learned prototypes, and producing class scores. 6
2.3 Illustration of multi-label ECG classification strategies (example: ensemble learning and modular branches). 7
2.4 Example of a segmentation-based ECG digitisation output: (a) input ECG image with noise/shadow, (b) U-Net segmentation mask output, and (c) reconstructed signal compared to the original digital signal. Adapted from [20]. 8

3.1 Heart Guard end-to-end pipeline for ECG image and signal inputs. . . . 11
3.2 Heart Guard system architecture (Flutter client, Django backend, ML services, and storage). 15

4.1 PTB-XL diagnostic superclasses and subclasses (SCP-ECG grouping). . . 20
4.2 Example record from Al Shifa' Specialized Complex: exported digital ECG (aECG format) converted into an image for representation. 22
4.3 Example paper ECG in a 12×1 rhythm-style layout collected locally. . . 23
4.4 Example paper ECG in a 3×4 layout collected locally. 23
4.5 Example of a locally collected paper ECG in a 3×4 layout, illustrating real-world variability in grid intensity, printing quality, and capture angle. 24
4.6 Example from Makassed Hospital (Jerusalem): matched ECG image converted to a 3×4+1 visualisation format used in the project pipeline. . . . 25
4.7 Synthetic ECG page formats generated from PTB-XL signals: (a) 3×4 layout, (b) 3×4 + rhythm strip layout. The generator settings were tuned to match local printed alignment observed in Palestinian hospitals. 27
4.8 Ground-truth masks generated from JSON metadata: (a) binary mask (waveform vs. background), (b) multi-class mask visualised using different colours. 28

4.9 Binary ground-truth mask used for final nnU-Net training (waveform vs. background). 28
4.10 Overlay verification examples demonstrating alignment between generated ECG pages and their ground-truth masks: (a) 3×4 layout, (b) 3×4 + rhythm strip layout. 29
4.11 Cropped side-by-side verification examples for (top) 3×4 and (bottom) 3×4 + rhythm-strip layouts, showing ECG images and their corresponding masks. 30
4.12 Examples of augmented synthetic ECG pages under two layouts: 3×4 (top row) and 3×4+1 with a rhythm strip (bottom row). Geometric transforms are applied jointly to the ECG and mask; appearance degradations are applied to the ECG only, preserving ECG–mask alignment. 31
4.13 Flask web-based ECG labeling tool. 33
4.14 Leakage-free dataset splitting strategy for ProtoECGNet. Prototypes are learned and projected exclusively from the training set, while validation and test sets are used only for evaluation and explanation retrieval. . . . 39

5.1 Example of segmentation degradation on a real hospital ECG page when training is synthetic-only: the predicted mask is incomplete and partially confuses grid lines with the waveform, illustrating the remaining domain gap. 41
5.2 Conceptual U-Net encoder–decoder with skip connections (diagram from Wikimedia Commons). [18, 25] 42
5.3 U-Net trained on synthetic-only data: qualitative comparison of ground-truth (GT) mask and predicted mask on a synthetic ECG page. 43
5.4 High-level nnU-Net workflow (overview diagram from the official nnUNet repository) illustrating dataset fingerprinting, rule-based planning, training, and postprocessing. [19, 24] 44
5.5 First nnU-Net-based model with augmentation: example segmentation on a real ECG page showing improved continuity relative to the synthetic-only U-Net baseline, but still with remaining artifacts and missed segments. . 45
5.6 nnU-Net training progress example (Fold 1): training/validation loss, pseudo-dice trend, epoch duration stability, and learning rate schedule as reported automatically by nnU-Net. 46
5.7 Final binary segmentation model (3×4 layout): qualitative example on a real ECG page showing good waveform continuity and alignment with the underlying trace. 47

5.8 Failure case of the multi-class segmentation model: predictions show inconsistent class labels and missing waveform segments, indicating unstable training and poor generalisation. 48

6.1 Binary segmentation mask used as digitiser input (foreground = ECG trace). 52
6.2 ROI splitting result: detected 12 lead cells (3×4) overlayed on the mask. Each lead is digitised independently. 53
6.3 Per-lead extraction overlays (example): extracted polyline points (red) plotted over each ROI mask. 55
6.4 Final digitised 12-lead signal plot reconstructed from the extracted mV signals (3×4 layout). 59

7.1 Internal architecture of ProtoECGNet. Each ECG is processed in parallel by three branches: a 1D CNN with global temporal prototypes for rhythm interpretation, a 2D CNN with localized prototypes for morphology-based reasoning, and a 2D CNN with global prototypes for diffuse signal abnormalities. Similarity scores from all branches are fused using a linear classifier to produce final predictions. 65
7.2 Prototype-based multi-branch explanation in ProtoECGNet. Highlighted regions indicate ECG segments that are most similar to learned prototypes in each branch. These regions directly contribute to the final prediction, enabling transparent and case-based explanations of model decisions. . . 66

8.1 Home dashboard (Desktop/Web, Light Mode). The main entry point to the application with shortcuts to Patients, ECG Upload/Analysis, and the 12-Lead ECG Gallery. 77
8.2 Home dashboard (Desktop/Web, Dark Mode). The same layout with a dark theme to enhance readability in low-light environments. 78
8.3 Home screen of the ECG Analyzer mobile application (iOS). The main entry point provides direct navigation to core workflows, including Patients, ECG Upload/Analysis, and the 12-lead ECG reference gallery. 79
8.4 Patients screen (Desktop/Web). Users can create patient profiles, browse stored records, view the number of associated ECG cases, and delete patients when required. 80
8.5 Patients screen (iOS). Users can create new patient profiles and browse stored records. Each entry includes demographic information and the number of associated ECG cases for longitudinal monitoring. 81
8.6 Patient dashboard (Desktop/Web). Provides doctor notes, ECG intake controls, and historical ECG cases with predicted results for longitudinal follow-up. 82

8.7 Upload options dialog (Desktop/Web). Users select between uploading a digital ECG signal or an ECG image to ensure correct backend routing. . 83
8.8 Signal type selection (Desktop/Web). Users choose Philips XML ECG or aECG, and the backend applies format-specific parsing before inference. . 83
8.9 Image layout selection (Desktop/Web). The chosen layout guides cropping and preprocessing to correctly extract lead regions. 84
8.10 Result view (Desktop/Web, Preview). Displays the processed ECG region used as input to the inference pipeline. 85
8.11 Result view (Desktop/Web, Scores). Predicted diagnostic outcomes presented with confidence scores in a structured format. 85
8.12 ECG result view (iOS). The interface displays the original uploaded ECG image and the processed (cropped) input used for inference, followed by the predicted diagnostic outcome and confidence score. 86
8.13 12-Lead ECG reference gallery (Desktop/Web). Provides categorized and searchable ECG examples for educational use and visual comparison. . . 87
8.14 12-lead ECG reference gallery (iOS). The gallery provides categorized and searchable ECG examples for educational use and visual comparison, and it is not used for automated inference. 88
8.15 Manual ECG cropping interface (Desktop/Laptop). Providing manual cropping on desktop and mobile improves robustness and ensures correct preprocessing of scanned or uploaded ECG images before backend inference. 89
8.16 Manual ECG cropping interface (iOS). Users can manually adjust the cropping region to precisely select the relevant ECG area before processing and inference. 90

List of Tables

4.1 Comparison of public ECG datasets considered for Heart Guard. 18
4.2 Final dataset statistics for nnU-Net segmentation on the 3×4 layout (binary masks). 36
4.3 Initial synthetic dataset used for nnU-Net multi-class segmentation on the 3×4 + rhythm-strip layout (labels 0–12). 37
4.4 Final synthetic dataset statistics for nnU-Net binary segmentation on the 3×4 + rhythm-strip layout. 38
4.5 Final dataset statistics and splits for ProtoECGNet training and evaluation using PTB-XL. 38

5.1 Final binary 3×4 segmentation model: mean validation Dice across five folds. 47
5.2 Attempted multi-class segmentation (0–12): mean validation Dice across five folds. 48
5.3 Practical comparison of binary and multi-class segmentation for ECG waveform extraction. In this work, binary masks were sufficient for digitisation and substantially more robust than multi-class training under real-scan variability. 49

7.1 Performance comparison between the HeartGuard implementation and the original ProtoECGNet results. 72

8.1 Mapping between Flutter features and Django REST APIs 92

9.1 Comparison between physician-annotated ECG findings and model predictions for hospital-grade XML ECG data (Al-Maqassed Hospital). 104
9.2 Comparison between physician-annotated ECG findings and model predictions for ambulatory aECG data (Al Shifa Specialized Complex). 106

Chapter 1

Introduction

Contents

1.1 Problem Statement 1

1.2 Research Goals 2

1.3 Organization of the Report 2

1.1 Problem Statement

Cardiovascular diseases are the leading cause of death worldwide [1], making rapid detection and timely clinical decisions essential for improving patient outcomes. In many healthcare environments—particularly in understaffed and high-pressure hospitals and emergency rooms—cardiologists are not always available to interpret ECGs immediately. During busy shifts, large patient volumes and limited specialist coverage can delay ECG review, even when the case may be time-sensitive. As a result, the initial assessment often depends on general physicians who may lack the specialised training required to confidently diagnose certain ECG abnormalities. These delays and uncertainties can slow decision-making at critical moments, potentially worsening outcomes in conditions where minutes matter.

The Heart Guard project addresses this gap by providing an AI-assisted system that analyses an ECG (from an image or a signal) and generates diagnostic decision support. The system is designed to support clinicians—not replace them—by offering fast, consistent preliminary insights that help medical staff make quicker, better-informed decisions, prioritise urgent cases, and reduce the time to intervention. By enabling faster triage and earlier detection, the project aims to strengthen emergency care workflows and contribute to saving lives.

1.2 Research Goals

The main goals of this project are:

- Build a unified pipeline that can ingest an ECG as either an image (paper/scan/photo) or a digital signal.
- Digitise and standardise ECG inputs into a consistent signal representation suitable for automated processing.
- Integrate an AI diagnostic model (ProtoECGNet) to provide fast, AI-assisted cardiac diagnosis outputs for clinical decision support.
- Develop a cross-platform user application using Flutter (mobile and Windows) for ECG capture/upload, case management, and result visualisation.
- Implement a Django backend that performs preprocessing, secure data handling, and model inference, and exposes APIs consumed by the Flutter application.

1.3 Organization of the Report

This report is structured into seven chapters. Chapter 1 outlines the problem context, motivation, and objectives of the Heart Guard project. Chapter 2 surveys related literature on ECG digitisation, medical image segmentation, and interpretable deep learning for ECG diagnosis. Chapter 3 describes the overall system architecture, presenting the end-to-end pipeline and the Flutter–Django design. Chapter 4 introduces the datasets and explains the data preparation steps for both segmentation-driven digitisation and ProtoECGNet-based diagnosis. Chapter 5 presents the segmentation models and training methodology, including the transition from synthetic-only training to mixed real–synthetic training, as well as a comparison between binary and multi-class settings. Chapter 6 details the digitisation stage that converts segmentation masks into 12-lead signals and reports quantitative evaluation against reference signals. Chapter 7 describes the ProtoECGNet diagnostic model, its training procedure, and classification results, and explains how the diagnosis module is integrated into the complete Heart Guard system.

Chapter 2

Literature Review

Contents

2.1 Overview and Motivation 3

2.2 Deep Learning for ECG Interpretation 4

2.3 Limitations of Post-Hoc Explainability in Clinical AI 5

2.4 Prototype-Based Learning for Interpretability 6

2.5 Multi-Label Classification in ECG Analysis 6

2.6 Toward Clinically Useful and Trustworthy ECG AI 7

2.7 ECG Digitisation from Images Using Segmentation-Based Pipelines 8

2.7.1 nnU-Net and Robust Medical Image Segmentation 8

2.7.2 Benchmarking and Recent Trends 9

2.7.3 Relevance to the Heart Guard Project 9

2.1 Overview and Motivation

Over the past decade, there has been rapid growth in the use of artificial intelligence (AI)—particularly deep learning—for electrocardiogram (ECG) interpretation and clinical decision support. This trend is driven by the global burden of cardiovascular diseases (CVDs), which remain the leading cause of death worldwide [1]. In practice, ECG interpretation is often time-critical, especially in emergency departments where rapid triage decisions may be required. However, specialist cardiologists are not always available on-demand, and interpretation may fall to general physicians with variable expertise. This gap has motivated research into automated systems that can deliver fast, consistent preliminary assessments to support clinical workflows.

This chapter reviews key research themes relevant to the Heart Guard project: (1) deep learning for ECG interpretation, (2) limitations of post-hoc explainability for clinical AI, (3) prototype-based learning as an interpretable alternative, (4) multi-label ECG classification for real clinical complexity, and (5) ECG digitisation from images using segmentation-based pipelines, including modern frameworks such as nnU-Net. These directions motivate Heart Guard's design, which combines ECG digitisation with interpretable diagnosis support.

2.2 Deep Learning for ECG Interpretation

Deep learning has become one of the most effective approaches for automatic ECG analysis, due to its ability to learn complex discriminative features directly from raw signals. Convolutional neural networks (CNNs) have been widely used for arrhythmia recognition and abnormality detection because they can capture local morphological patterns (e.g., QRS complexes) as well as rhythm-level characteristics over time.

A prominent milestone is the work of Hannun et al., who demonstrated cardiologist-level performance on arrhythmia classification using an end-to-end deep neural network trained on large-scale single-lead ambulatory ECG recordings [2]. This result reinforced the feasibility of deploying deep learning as an effective assistant for ECG interpretation and reducing inter-reader variability in routine diagnosis.

In parallel, public challenges and open datasets have accelerated progress by providing standard benchmarks and evaluation protocols. For example, the PhysioNet/Computing in Cardiology Challenge 2017 focused on classifying short single-lead ECG recordings into normal rhythm, atrial fibrillation, other rhythm, or noise, encouraging practical and robust algorithm development [7, 8].

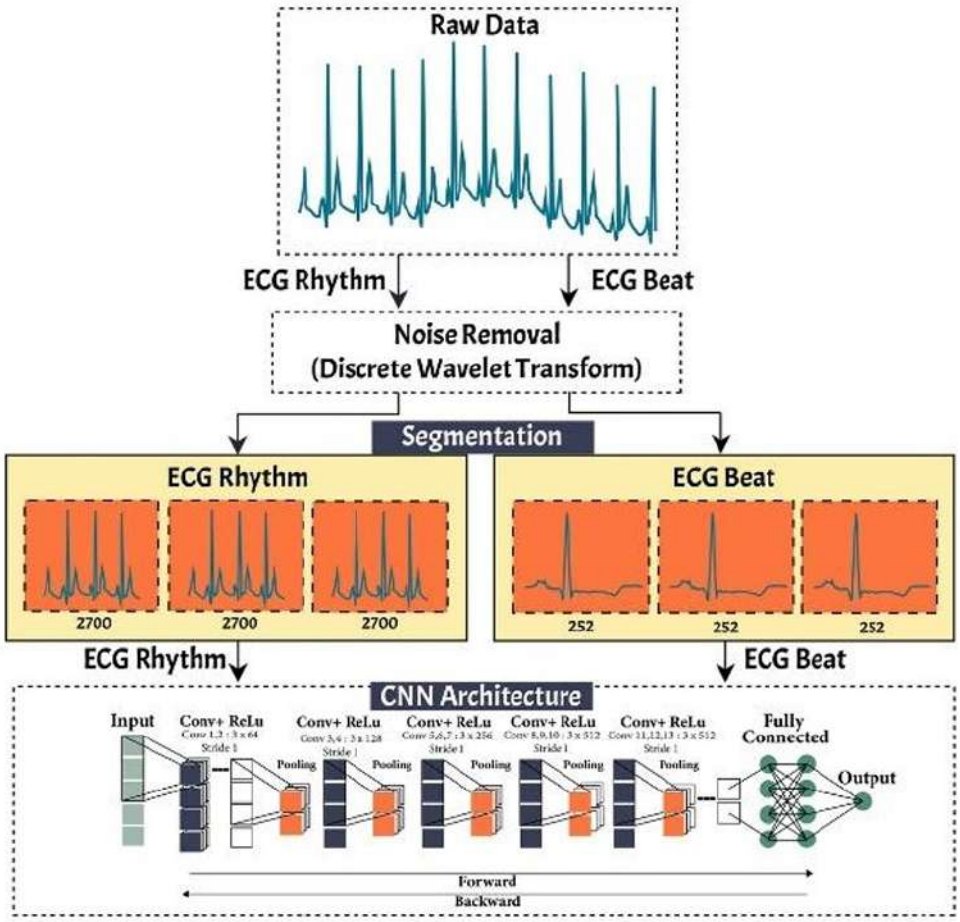


Figure 2.1: Deep learning-based ECG rhythm and beat features for heart abnormality classification (example architecture illustration).

2.3 Limitations of Post-Hoc Explainability in Clinical AI

Despite strong predictive performance, many deep learning models remain difficult to interpret, which creates barriers for clinical adoption and trust. A common approach is to apply post-hoc explainability techniques (e.g., saliency maps, attention visualisations, SHAP/LIME) after training. However, these explanations may be unstable, incomplete, or difficult to validate clinically.

Rudin argues that for high-stakes domains such as medicine, reliance on post-hoc explanations for black-box models can be misleading and advocates for inherently interpretable model designs whenever possible [3]. Similarly, Ghassemi et al. warn that current explainability methods may provide a *false sense of security* for patient-level decision support and emphasize rigorous external validation and careful deployment processes over superficial explainability requirements [5].

These critiques are directly relevant to systems intended for clinical support, because

the goal is not only accuracy, but also *trustworthy reasoning, robustness, and safe integration* into real workflows.

2.4 Prototype-Based Learning for Interpretability

Prototype-based learning offers a promising direction for making model decisions more transparent. Instead of producing predictions with opaque internal reasoning, prototype-based models learn representative patterns during training and classify new samples by measuring similarity to these prototypes.

Chen et al. introduced ProtoPNet, a framework that explains decisions through "this looks like that" reasoning—linking model outputs to learned prototypical parts or segments that resemble labelled training examples [4]. This design supports case-based reasoning and can help clinicians audit predictions by comparing a new ECG to representative learned examples, improving interpretability without necessarily sacrificing accuracy.

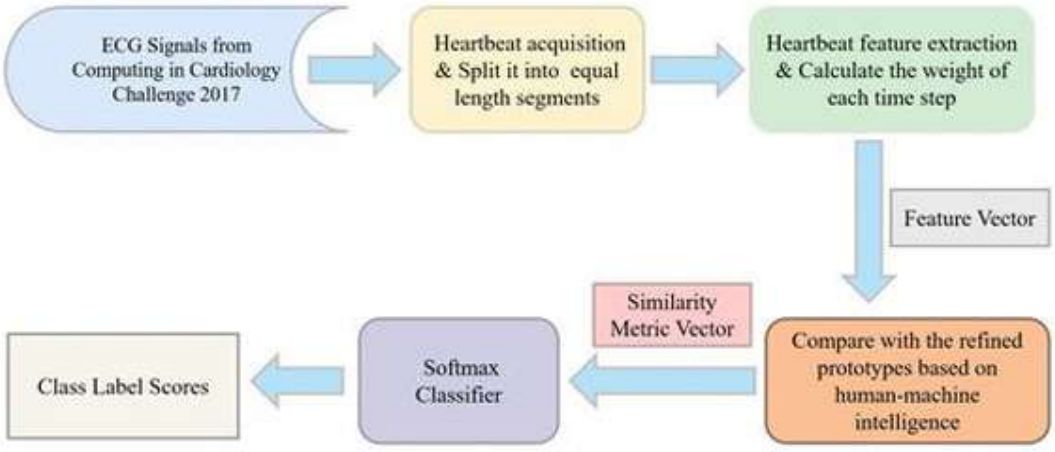


Figure 2.2: Workflow of prototype learning: extracting features, comparing to learned prototypes, and producing class scores.

2.5 Multi-Label Classification in ECG Analysis

Real ECGs often reflect multiple co-existing conditions. For example, a patient may exhibit arrhythmia along with ischemic changes or conduction abnormalities. This makes ECG interpretation naturally multi-label, where multiple diagnoses may apply to the same recording. Traditional single-label classification can be insufficient because it forces a single "best" label even when the clinical reality is overlapping pathology.

Recent work explicitly targets this challenge. Prabhakararao and Dandapat propose temporal convolutional models for multi-label ECG classification and emphasize that

practical screening often requires detecting multiple concurrent disorders [9]. Their findings support the broader shift from simplified academic settings toward clinically realistic modelling assumptions.

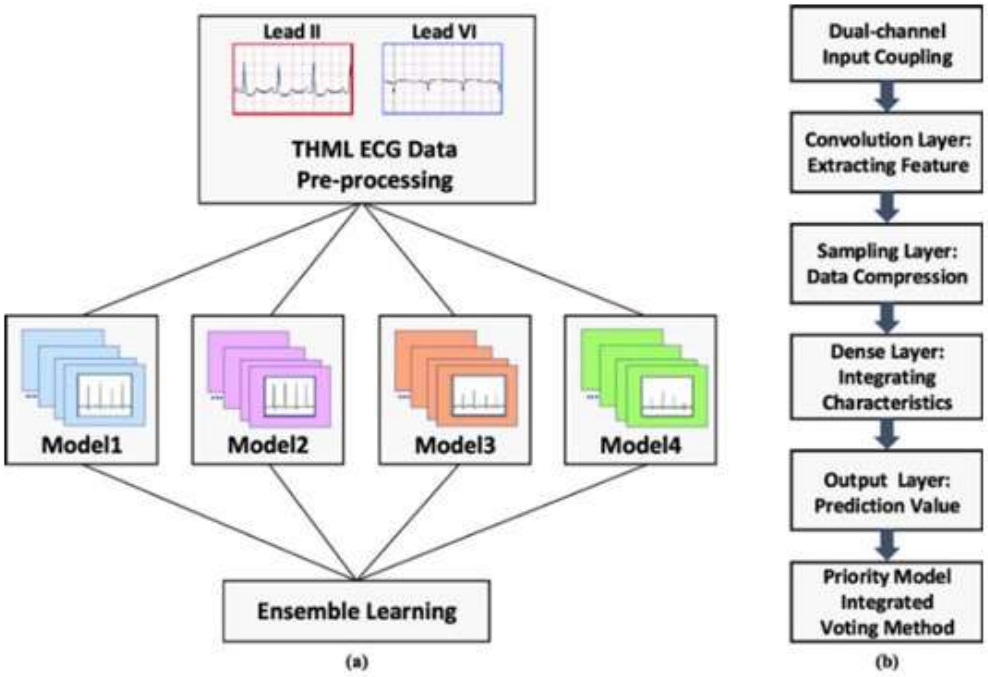


Figure 2.3: Illustration of multi-label ECG classification strategies (example: ensemble learning and modular branches).

2.6 Toward Clinically Useful and Trustworthy ECG AI

Beyond accuracy, clinical AI systems must address reliability, safety, and fairness concerns. Review work in ethical and trustworthy medical ML highlights that model performance alone is insufficient; deployment must consider distribution shift, bias amplification, dataset representativeness, and clear responsibility boundaries in decision-making [6]. These considerations are essential for systems such as Heart Guard that aim to support time-critical triage, where errors may carry high clinical cost.

Overall, the literature suggests a broad shift: from purely predictive systems toward decision-support tools that combine strong performance with interpretable reasoning, robust validation, and clinically aligned design.

2.7 ECG Digitisation from Images Using Segmentation-Based Pipelines

A key practical challenge in many hospitals is that ECGs may exist as *paper printouts, scanned reports, or mobile phone photos*, rather than clean digital waveforms. Converting these images into usable signals enables automated analysis, archiving, and AI-based diagnosis. This task is difficult due to background grid lines, varying paper quality, noise, skew/rotation, and overlapping traces.

A notable approach is to formulate ECG digitisation as an image segmentation problem. Li et al. proposed a deep learning method for digitising highly noisy paper-based ECGs by using U-Net to segment waveform pixels from background, allowing end-to-end extraction even under challenging layouts. This segmentation framing is important because it aligns naturally with modern biomedical segmentation methods.

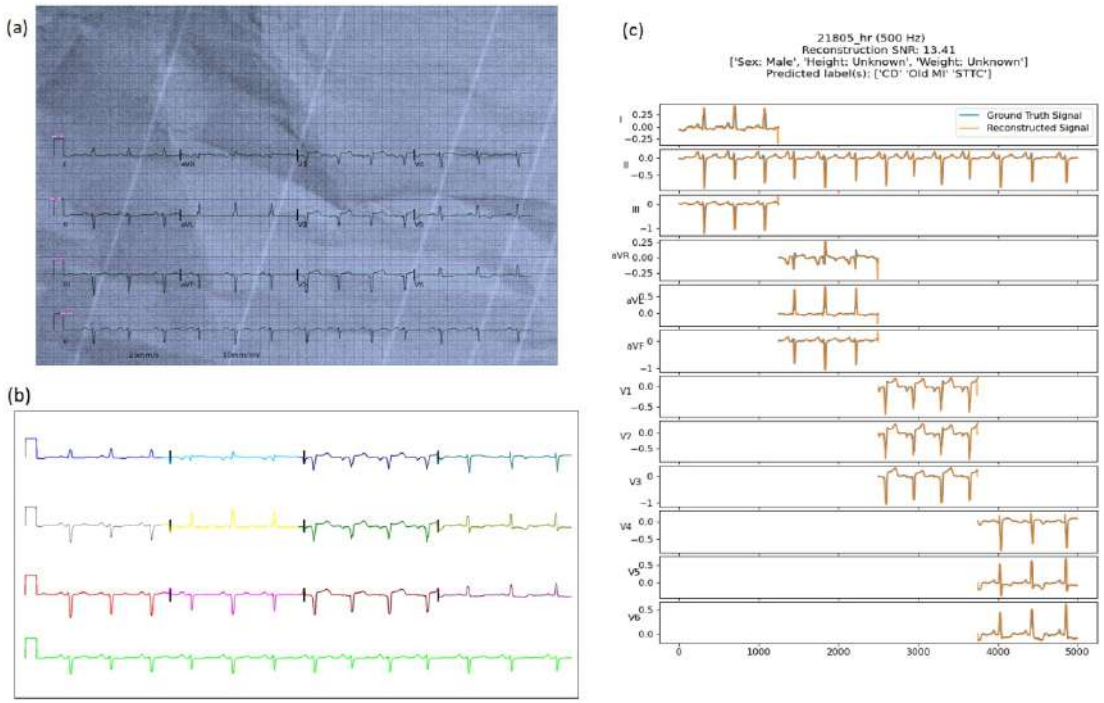


Figure 2.4: Example of a segmentation-based ECG digitisation output: (a) input ECG image with noise/shadow, (b) U-Net segmentation mask output, and (c) reconstructed signal compared to the original digital signal. Adapted from [20].

2.7.1 nnU-Net and Robust Medical Image Segmentation

In biomedical imaging, nnU-Net is a widely adopted self-configuring segmentation framework that automatically adapts preprocessing, architecture, and training choices to a target dataset, often achieving strong results without extensive manual tuning [19]. This is particularly relevant to ECG image segmentation because ECG images can vary sig-

nificantly in style (scanner vs. camera), resolution, grid intensity, and lead layout. Using nnU-Net-style training pipelines can improve robustness and simplify engineering effort when scaling to diverse hospitals and acquisition conditions.

2.7.2 Benchmarking and Recent Trends

Community benchmarks further highlight the importance of ECG digitisation. For instance, recent competition and research directions (including modern CinC/PhysioNet tasks) increasingly focus on ECGs captured from images or paper printouts, reflecting real clinical data pipelines rather than ideal digital signals [20].

2.7.3 Relevance to the Heart Guard Project

The Heart Guard system builds upon these trends by combining: (1) segmentation-based extraction for ECG digitisation (image $\rightarrow$ clean representation), (2) standardised preprocessing for consistent input formation, and (3) AI-assisted diagnosis models designed to provide fast decision support outputs. By addressing both image-based ECG acquisition and diagnostic inference, Heart Guard aligns with the research trajectory toward *deployable, end-to-end clinical systems* rather than isolated models.

Chapter 3

System Overview and Methodology

Contents

3.1 System Overview 10
3.1.1 Main System Capabilities 11
3.1.2 End-to-End Pipeline Overview 11
3.2 Methodology and Pipeline Stages 12
3.2.1 Stage 1: ECG Acquisition and Input Handling 12
3.2.2 Stage 2: Case Creation, Persistence, and History Retrieval . . . 12
3.2.3 Stage 3: Image Preprocessing and Standardisation 12
3.2.4 Stage 4: Segmentation and Digitisation (Image Inputs) 13
3.2.5 Stage 5: Signal Reader and Standardisation (Signal Inputs) . . 13
3.2.6 Stage 6: Preprocessing for Inference 13
3.2.7 Stage 7: Diagnostic Inference and Output Generation 14
3.2.8 Stage 8: Result Delivery and Visualisation 14
3.3 System Architecture 14
3.4 Chapter Summary 15

3.1 System Overview

Heart Guard is an end-to-end decision-support system designed to assist medical staff by producing rapid AI-based diagnostic insights from an electrocardiogram (ECG). In real hospital workflows, ECG recordings originate from an ECG machine, but they can reach the system through different acquisition routes. Heart Guard supports the following input types:

- **ECG Images:** paper printouts, scanned reports, or mobile phone photos.

- **Digital ECG Signals:** exported ECG waveform recordings in a supported digital format.

The system is built around two design goals: (1) accept ECGs in the forms that hospitals actually produce (image or signal), and (2) standardise them into a consistent representation that can be processed reliably by the diagnostic model. For image-based ECGs, Heart Guard performs digitisation using segmentation to isolate waveform traces from gridlines and acquisition artefacts. For signal-based ECGs, the system parses the input format and applies preprocessing to create model-ready inputs.

From the user perspective, Heart Guard is accessed through a cross-platform Flutter application (mobile and Windows) that supports ECG capture/upload, case tracking, and result visualisation. Heavy processing—digitisation, preprocessing, and inference—is performed on a Django backend that exposes secure APIs consumed by the Flutter client.

3.1.1 Main System Capabilities

The system provides the following capabilities:

- Accept ECG input as an **image** or a **digital signal**.
- Digitise image ECGs using **segmentation-based extraction** (nnU-Net) to recover waveform representations.
- Apply **standardised preprocessing** to format all inputs consistently for inference.
- Perform diagnostic inference using **ProtoECGNet**, a prototype-based deep learning model designed to provide interpretable ECG classification.
- Store and retrieve patient/case history, including **dated results, visualisations, and doctor reports**.
- Visualise results and case history through the Flutter application.

3.1.2 End-to-End Pipeline Overview

Figure 3.1 summarises the end-to-end pipeline from ECG acquisition to diagnosis output and visualisation.

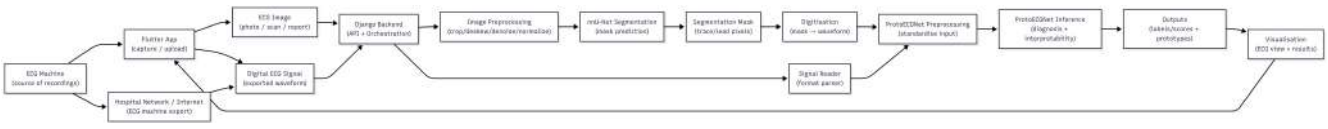


Figure 3.1: Heart Guard end-to-end pipeline for ECG image and signal inputs.

3.2 Methodology and Pipeline Stages

This section describes the major processing stages implemented in Heart Guard. The pipeline is modular, allowing improvements to individual stages (e.g., upgrading segmentation or changing the inference model) without redesigning the entire system.

3.2.1 Stage 1: ECG Acquisition and Input Handling

Heart Guard supports three realistic acquisition routes:

- **Flutter upload/capture:** the user captures an ECG photo or uploads an existing ECG image/signal through the Flutter application.
- **Direct network signal export:** an ECG machine exports a digital signal over the hospital network/internet directly to the Django backend (without passing through Flutter).

All inputs are associated with case metadata (e.g., timestamp, acquisition route, optional identifiers according to the workflow). The backend validates file type/size and supported formats before continuing.

3.2.2 Stage 2: Case Creation, Persistence, and History Retrieval

To behave like a real application, the backend supports both **write** and **read** operations over patient records:

- **Write path:** create a new patient record (when needed), create a new case entry (timestamped), store uploaded ECG files, and store the produced outputs (diagnosis, visualisations, and doctor reports).
- **Read path:** retrieve an existing patient profile, load prior cases, and display historical results and dated reports when requested.

In practice, large binary files (images, masks, rendered visualisations) are stored in object/file storage, while structured records (patients, cases, timestamps, diagnosis labels/scores, report metadata, storage references/paths) are stored in a relational database.

3.2.3 Stage 3: Image Preprocessing and Standardisation

For ECG images, the backend performs preprocessing to reduce variability across scanners and cameras. Typical operations include:

- cropping and alignment (deskew/perspective correction when needed),

- normalisation (contrast/brightness adjustments),
- noise handling (scanner artefacts/shadows),
- format unification (e.g., JPEG to PNG).

The goal is to produce a consistent input for segmentation and digitisation, even when the image quality differs across hospitals and devices.

3.2.4 Stage 4: Segmentation and Digitisation (Image Inputs)

Digitisation is treated as a segmentation-driven extraction problem:

- **Segmentation (nnU-Net):** predict a mask that isolates ECG trace/lead pixels from gridlines and background.
- **Digitisation:** convert the segmentation mask into a structured waveform representation suitable for downstream processing.

This approach is designed to handle real-world issues such as grid intensity variation, scanning noise, motion blur, and lighting effects in photographed ECG reports.

3.2.5 Stage 5: Signal Reader and Standardisation (Signal Inputs)

For digital ECG signals, the backend uses a dedicated signal reader to parse the input according to its format. Once parsed, the signal is standardised into the same internal representation used by the digitised image pipeline. This ensures that downstream preprocessing and inference behave consistently across both sources.

3.2.6 Stage 6: Preprocessing for Inference

After digitisation (images) or parsing (signals), the pipeline applies model-specific preprocessing to produce inference-ready inputs. These steps are kept consistent across all acquisition routes to minimise distribution shift between image-derived and signal-derived ECGs. Typical operations include:

- scaling/normalisation,
- lead/layout formatting,
- windowing/segmentation into the required input structure,
- building the final tensor representation expected by ProtoECGNet.

3.2.7 Stage 7: Diagnostic Inference and Output Generation

Inference is performed using ProtoECGNet to generate AI-assisted decision support outputs. The produced outputs are structured for clinical usefulness, such as:

- predicted diagnosis labels and confidence scores,
- interpretable evidence (e.g., prototype similarity outputs),
- visualisations that support quick review and triage.

All outputs are stored with timestamps so cases can be reviewed later, compared across time, and included in dated doctor reports.

3.2.8 Stage 8: Result Delivery and Visualisation

The Django backend returns inference outputs and visualisation data through API responses. The Flutter client visualises:

- the original input (image view or waveform view),
- the processed/digitised representation (when applicable),
- diagnosis results and supporting interpretability evidence,
- patient history and prior cases (read from the database on request).

3.3 System Architecture

Heart Guard is implemented using a client–server architecture with distinct processing and persistence responsibilities:

- **Flutter Client (Mobile/Windows):** UI, capture/upload, case management, patient history viewing.
- **Django Application Server:** REST API, authentication, orchestration, read-/write access to persistence.
- **ML/Processing Services:** image preprocessing, nnU-Net segmentation, digitisation, signal parsing, ProtoECGNet preprocessing and inference.
- **Persistence Layer:** relational database for structured records, and object storage for large files and generated artefacts.

This separation keeps the client lightweight and responsive while centralising compute-heavy processing and data management on the backend.

Figure 3.2 illustrates component interactions and the separation between the application server and the persistence layer.

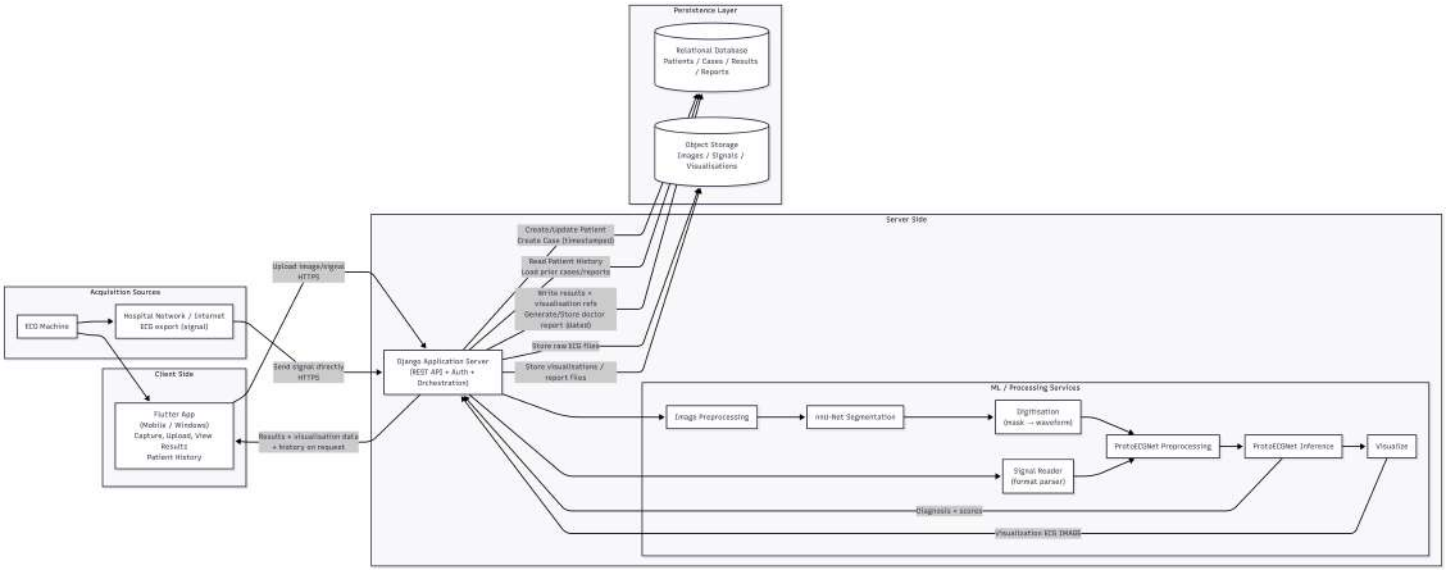


Figure 3.2: Heart Guard system architecture (Flutter client, Django backend, ML services, and storage).

3.4 Chapter Summary

This chapter presented the Heart Guard methodology and architecture: a modular pipeline that accepts ECG images or signals, standardises them via digitisation/parsing and preprocessing, performs interpretable diagnostic inference using ProtoECGNet, and delivers results through a Flutter application backed by a Django server with separate database and object storage. The next chapters will detail the dataset preparation, the digitisation module, and the diagnostic model implementation and evaluation.

Chapter 4

Data and Dataset Preparation

Contents

4.1 Clinical Context and Motivation 17

4.2 Why We Needed an Online ECG Database 17

4.3 What We Looked For in a Database 17

4.4 Survey of Public ECG Datasets 18

4.4.1 Motivation for Choosing PTB-XL 19

4.5 Overview of the PTB-XL Dataset 19

4.5.1 Labeling and Clinical Categories 19

4.6 Local Clinical Data Collection 20

4.6.1 Contacting Hospitals and Approvals 20

4.6.2 Survey of ECG Machines and Practical Output Constraints . . 21

4.6.3 Site Outcomes and Collected Data 21

4.6.4 Overall Local Dataset Summary 25

4.6.5 Collaboration With Clinicians 26

4.7 Dataset Preparation and Standardisation 26

4.7.1 Digitisation Dataset Preparation (Synthetic ECG Images + Masks) 26

4.7.2 Diagnosis Dataset (ProtoECGNet) Preparation 33

4.8 Final Datasets Statistics and Splits 35

4.8.1 Segmentation Datasets (nnU-Net) 36

4.8.2 ProtoECGNet Dataset Preparation and Splits 38

4.1 Clinical Context and Motivation

Electrocardiograms (ECGs) are among the first-line tools used in emergency rooms and acute care settings to assess cardiac conditions. In time-critical scenarios, delayed interpretation can slow down triage and intervention. Although the Heart Guard system is designed to support clinicians rather than replace them, its value depends strongly on the quality and realism of the data used to develop and validate the AI pipeline. Therefore, dataset selection and preparation were treated as a core engineering step rather than a secondary task.

4.2 Why We Needed an Online ECG Database

At the beginning of the project, our first priority was to establish a reliable baseline for training, evaluation, and system prototyping. Since deep learning models typically require large, labelled datasets to generalise well across diverse patient populations, we initially relied on a public ECG database rather than local clinical records. This allowed us to start development immediately, validate the modelling pipeline, and benchmark our approach using a well-documented reference dataset.

For these reasons, an online public ECG database was required to:

- provide **large-scale labelled data** for initial model training and benchmarking,
- enable **reproducible experiments** that can be compared against prior research,
- cover **a broad range of diagnoses** and diverse ECG patterns,
- support the **multi-label** nature of real clinical interpretation,
- serve as a strong baseline before integrating **locally collected ECG data** later in the project.

4.3 What We Looked For in a Database

When surveying public ECG databases, we evaluated each candidate using criteria aligned with Heart Guard's goals (digitisation + diagnosis + clinical usefulness):

- **Clinical realism:** preference for standard **12-lead** recordings used in hospitals.
- **Scale:** enough recordings to train modern deep learning models.
- **Label quality and consistency:** medically meaningful labels with clear definitions and minimal ambiguity.

- **Multi-label support:** ability to represent multiple co-existing conditions in the same recording.
- **Signal quality and metadata:** availability of sampling rate details and basic patient/context metadata.
- **Public access and documentation:** licensing, reproducibility, and tool compatibility.

4.4 Survey of Public ECG Datasets

Before finalising the dataset used for training and benchmarking, we reviewed several widely used ECG datasets and assessed their suitability in terms of size, diagnostic richness, and clinical alignment:

- **MIT-BIH Arrhythmia Database:** historically important and widely used for arrhythmia research; however, it contains a relatively small number of subjects, limiting generalisability for large deep learning models.
- **INCART 12-lead Arrhythmia Database:** offers 12-lead recordings and good temporal resolution, but remains limited in size and diagnostic breadth compared to modern large-scale datasets.
- **Chapman University / Shaoxing Dataset:** large-scale recordings and useful for classification research, but the labelling structure is less suitable for realistic multi-label clinical deployment in our target setting.
- **PhysioNet 2020 Challenge Dataset:** large and diverse, but standardising labels and ensuring consistency across sources can be more challenging due to heterogeneity.
- **PTB Diagnostic ECG Dataset:** valuable clinical dataset and predecessor to PTB-XL, but smaller and with less comprehensive coverage.

Table 4.1: Comparison of public ECG datasets considered for Heart Guard.

Dataset	# Records	Leads	Sampling Rate (Hz)	Label Type	Multi-label	Public
MIT-BIH	~48	2	360	Beat-level	No	Yes
INCART	~75	12	257	Beat-level	No	Yes
Chapman (Shaoxing)	~10,000	12	500	Diagnosis	Limited	Yes
PhysioNet 2020	~43,000	1–12	Varies	Diagnosis	Varies	Yes
PTB-XL	21,801	12	100 & 500	Diagnosis (multi-label)	Yes	Yes

4.4.1 Motivation for Choosing PTB-XL

After weighing the strengths and limitations of the surveyed datasets, we selected **PTB-XL** as the primary public dataset for Heart Guard. The key reasons are:

- **Scalability:** PTB-XL provides a large number of recordings, supporting robust deep learning training.
- **Clinical alignment:** the dataset contains **12-lead** ECGs consistent with hospital practice.
- **Multi-label design:** diagnostic annotations reflect realistic overlap of conditions.
- **Strong documentation and adoption:** widely used as a benchmark, supporting fair comparison and reproducibility.
- **Engineering compatibility:** easy integration with standard tooling and ML pipelines.

4.5 Overview of the PTB-XL Dataset

PTB-XL is a large publicly available clinical ECG dataset containing 21,801 twelve-lead recordings collected from 18,869 patients. Each recording is 10 seconds long and is provided at 100 Hz and 500 Hz sampling rates. The dataset includes multi-label annotations based on SCP-ECG statements and is accompanied by metadata such as age, sex, and signal quality. [12]

4.5.1 Labeling and Clinical Categories

The dataset supports multi-label annotation, which reflects clinical reality where more than one condition may appear in the same ECG. In our work, we focused primarily on the **diagnostic** label categories (and their superclasses), as they align most directly with decision-support outputs.

Example diagnostic superclasses (PTB-XL):

- **NORM:** normal ECG.
- **MI:** myocardial infarction patterns.
- **HYP:** hypertrophy-related findings.
- **STTC:** ST/T changes (ischemic patterns and repolarisation abnormalities).
- **CD:** conduction disturbances (e.g., bundle branch blocks, AV blocks).

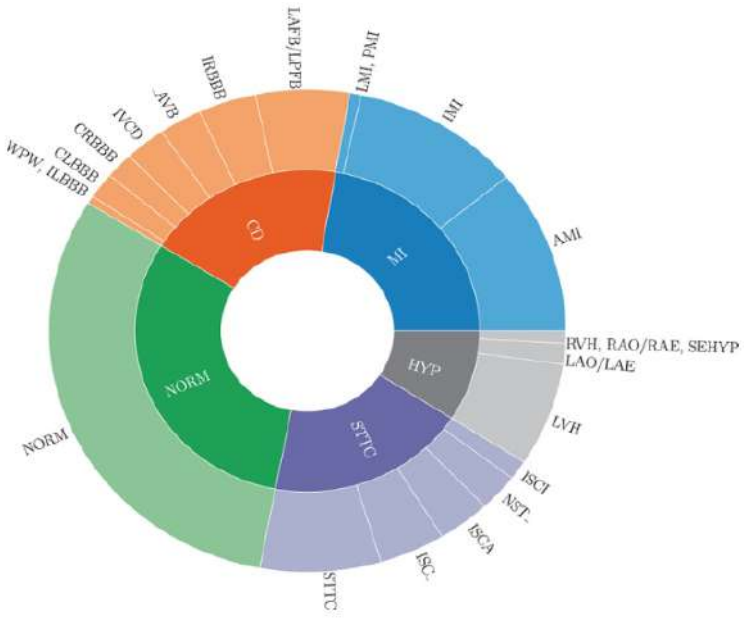


Figure 4.1: PTB-XL diagnostic superclasses and subclasses (SCP-ECG grouping).

4.6 Local Clinical Data Collection

While PTB-XL provides a strong public baseline for training and benchmarking, Heart Guard is ultimately intended for deployment in local hospital environments in Palestine. In these settings, ECG workflows differ significantly from ideal research conditions: ECGs are commonly handled as **paper printouts** that are later **scanned or photographed** and stored on local computers. In practice, most sites do **not** rely on routine internet-connected signal export, and direct access to archived ECGs on hospital computers was not permitted due to privacy and anonymisation constraints.

Additionally, not all ECG machines supported practical signal export in the first place. In several sites, devices either did not provide signal export, required proprietary vendor software, or failed during USB export attempts. These operational constraints motivated a core design direction of Heart Guard: **robust digitisation of paper ECGs using segmentation**, enabling the system to function under realistic local workflows rather than assuming clean digital signals.

4.6.1 Contacting Hospitals and Approvals

Local data collection required coordination with multiple healthcare providers and careful attention to privacy and ethical handling. We prepared official university letters stating that we are Computer Engineering students working on the Heart Guard graduation project. The letters described the project objectives, the type of ECG data requested (images and/or digital exports), and our commitment to anonymisation and responsible

data management. Using these documents, we contacted and held discussions with the following healthcare sites:

- Al Shifa' Specialized Complex
- Aldman Clinics
- Bethlehem Arab Society for Rehabilitation Hospital
- Doctor Ahmad Maslamani Hospital
- Makassed Hospital (Jerusalem)
- Top Clinic

4.6.2 Survey of ECG Machines and Practical Output Constraints

Across sites, we examined ECG machines and assessed what outputs could realistically be obtained without compromising privacy:

- In most environments, ECG **signals were not routinely exported online** and machines were not integrated with networked data pipelines.
- Accessing ECG archives stored on hospital computers was often not allowed, since extracting data from those systems could compromise patient privacy or anonymisation.
- Some machines supported signal export only through **proprietary formats** requiring vendor software, while others failed during USB export attempts.

These constraints collectively strengthened the need for image-based digitisation as the primary path for local deployment.

4.6.3 Site Outcomes and Collected Data

Over multiple visits conducted across approximately eight months, we collected or attempted to collect data from each site. The outcomes are summarised below.

Al Shifa' Specialized Complex

This site provided our largest set of exportable signals. We collected approximately **400** digital ECG recordings in `aECG`-style format. These recordings were later converted into ECG image representations for pipeline testing and visual validation. In addition, we obtained a small number of paper ECG pages (approximately **3** pages), including examples consistent with a **12×1** rhythm-style layout.

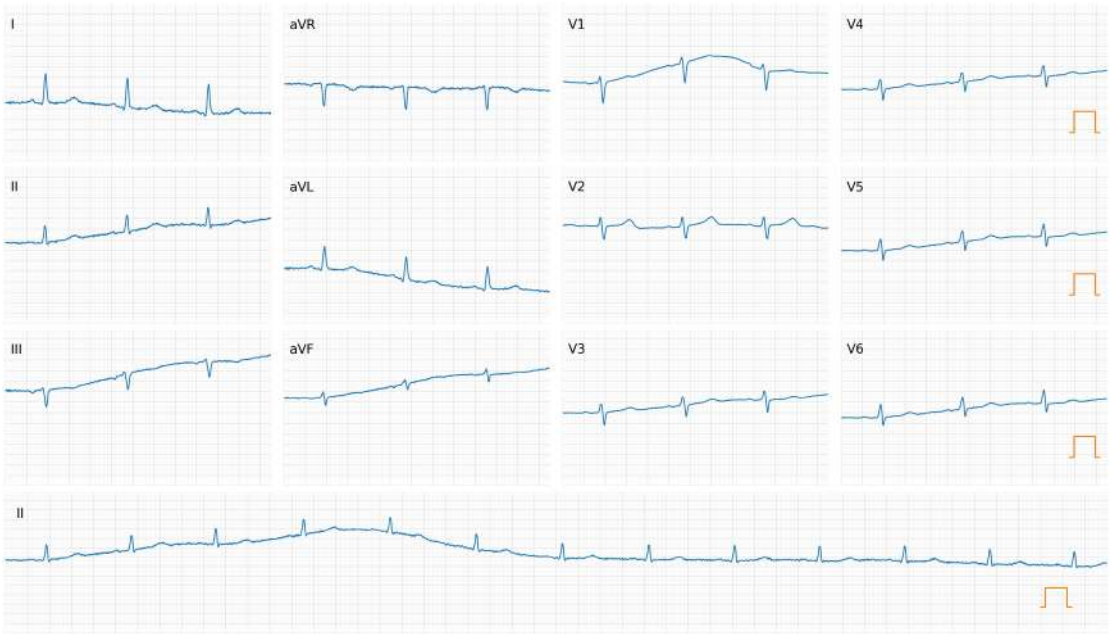


Figure 4.2: Example record from Al Shifa' Specialized Complex: exported digital ECG (aECG format) converted into an image for representation.

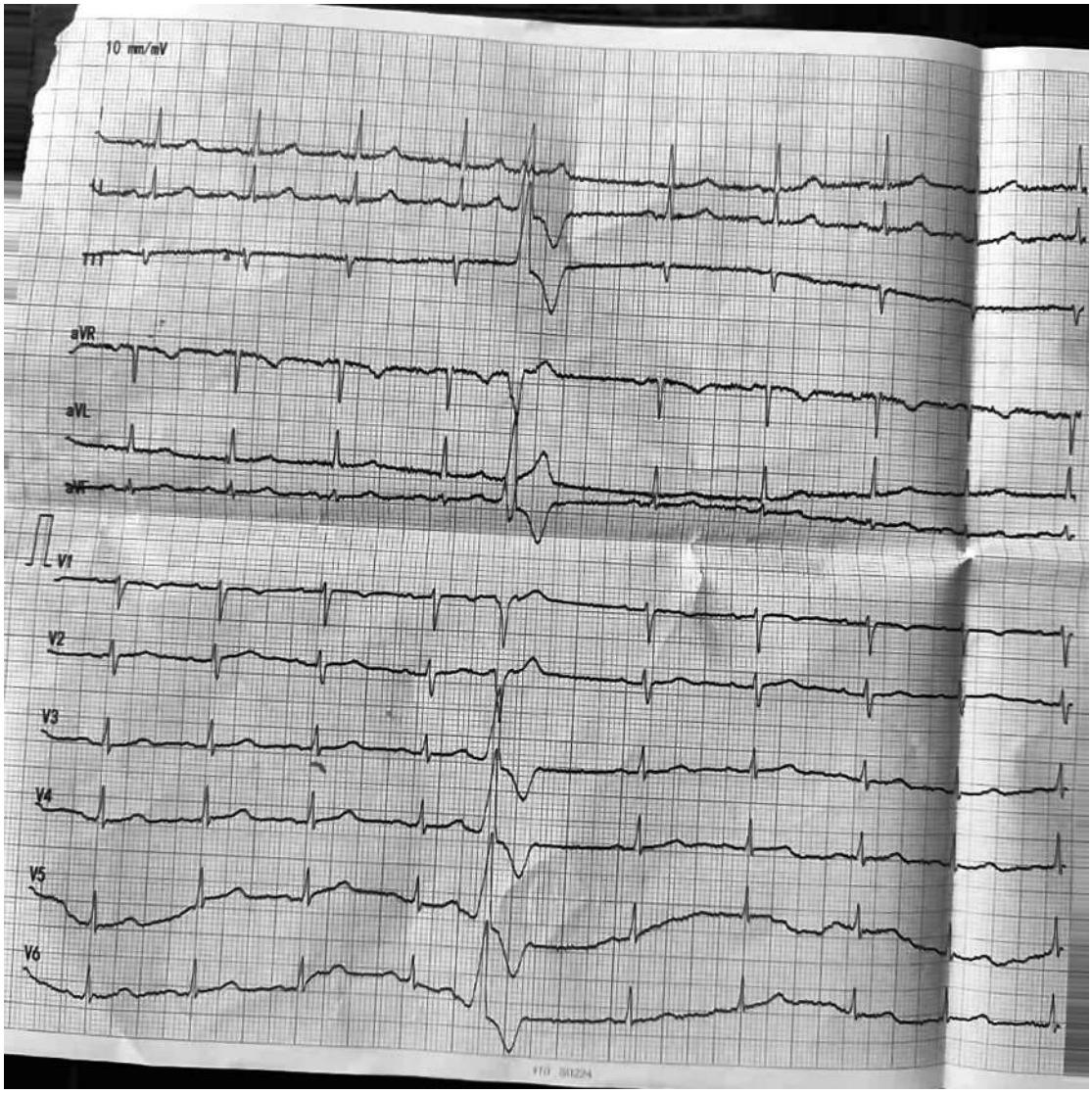


Figure 4.3: Example paper ECG in a 12×1 rhythm-style layout collected locally.

Aldman Clinics

The device contained approximately **500** records on the machine; however, repeated attempts to export data to USB failed. The export process consistently froze mid-transfer, preventing collection of usable signal files, while we got a sample on the 3x4 ECG image.

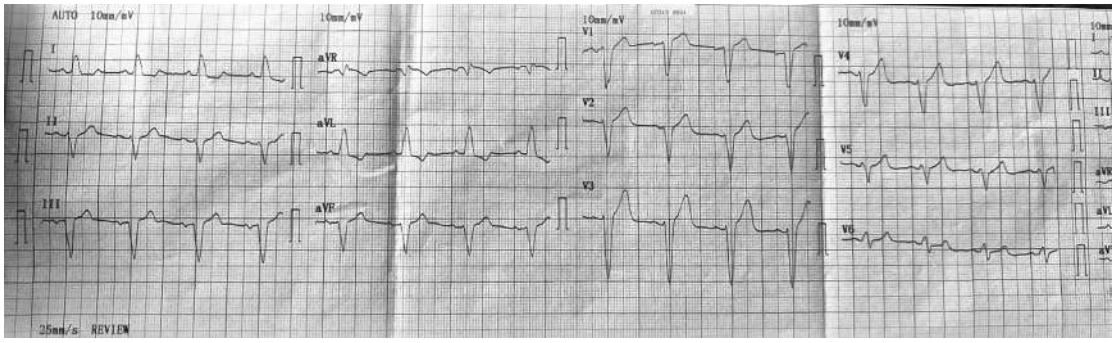


Figure 4.4: Example paper ECG in a 3×4 layout collected locally.

Bethlehem Arab Society for Rehabilitation Hospital

We exported approximately **20** recordings. Each recording consisted of multiple `.dat`-type files (three per record). Based on the device documentation, converting these files into standard waveforms required proprietary **Nihon Kohden** software, which was not available for our use. Despite this, the site contributed substantially to the image digitisation track: across our local collection effort, we gathered many paper ECG images in a **3×4** format (contributing to the overall image set used for segmentation and digitisation evaluation).

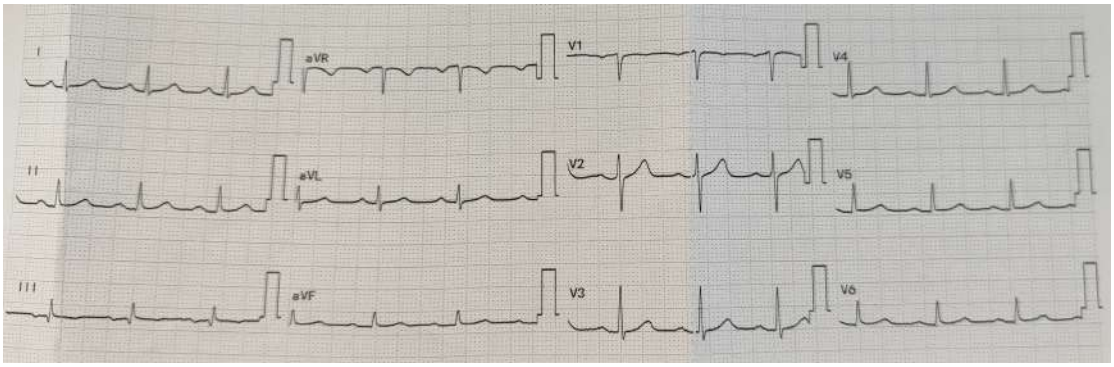


Figure 4.5: Example of a locally collected paper ECG in a 3×4 layout, illustrating real-world variability in grid intensity, printing quality, and capture angle.

Doctor Ahmad Maslamani Hospital

Signal export was attempted, but the ECG machine consistently refused or failed to export usable files to USB. As a result, no reliable signal dataset could be collected from this site.

Makassed Hospital (Jerusalem)

We initially found approximately **200** older readings stored on-device. However, due to archive access constraints, it was not possible to retrieve complete matched resources at that stage. We later requested new recordings to be captured and retained for our study. After subsequent visits, we obtained **127** XML files; however, only **32** matching ECG images were provided with corresponding XML metadata.

For analysis, these matched ECG images were converted into a **3×4 + 1** visualisation format (3×4 leads plus a long rhythm strip). However, an important limitation was observed: the device-exported signal corresponding to the XML covered only approximately **2.4 seconds per lead**, while the printed ECG image contained a longer duration (approximately **7.5 seconds** plus a rhythm strip). Due to this mismatch in temporal coverage, the XML resources could not be reliably used to generate accurate segmentation masks aligned to the full printed waveform.



Figure 4.6: Example from Makassed Hospital (Jerusalem): matched ECG image converted to a 3×4+1 visualisation format used in the project pipeline.

Top Clinic

Top Clinic uses an older ECG machine that produced paper ECG printouts (commonly in **3×4** format) without practical signal export capability. This reinforced the importance of the image digitisation workflow, since this paper-based pipeline closely matches real local practice.

4.6.4 Overall Local Dataset Summary

Across all cooperating sites, the local dataset collected to support Heart Guard development consists primarily of:

- approximately **1700** locally acquired ECG paper images (predominantly **3×4** layouts, with a smaller number of other layouts), used to evaluate segmentation robustness and image-to-signal digitisation under real acquisition conditions,
- approximately **400** exportable digital ECG recordings (`aECG`-format), used to test the diagnosis pipeline on real signals when direct digital export was available,
- a limited subset of **paired XML + ECG image** examples from Makassed Hospital (Jerusalem), used to explore feasibility of mask generation and matching under practical device constraints.

Overall, the collected data reflects local clinical practice where ECGs are commonly handled as paper printouts that are later scanned or photographed, and where consistent access to clean, exportable digital signals is not guaranteed. This directly motivates Heart Guard's emphasis on robust ECG digitisation from images for realistic deployment.

These outcomes justify the project's emphasis on ECG digitisation from images: the system must operate reliably in environments where paper ECGs are the dominant practical format and clean digital signals are not consistently accessible.

4.6.5 Collaboration With Clinicians

During this phase, we communicated with cardiologists and other doctors to validate the clinical usefulness of Heart Guard. Feedback indicated that rapid AI-assisted ECG interpretation could be valuable in understaffed and high-pressure environments, particularly for triage support and preliminary assessment. This feedback influenced our priorities: focus on deployability, speed, and interpretability rather than accuracy alone.

4.7 Dataset Preparation and Standardisation

Heart Guard processes two main input types: **ECG images** (paper printouts, scans, photos) and **digital ECG signals**. To ensure consistent behaviour across sources, we prepared separate but compatible dataset pipelines: one for **segmentation-based digitisation** (image → mask → signal) and one for **diagnosis inference** (signal → ProtoECGNet). In both cases, the objective is to minimise layout and formatting variation so that later stages operate on a stable and well-defined representation.

4.7.1 Digitisation Dataset Preparation (Synthetic ECG Images + Masks)

Training the digitisation module as a segmentation task requires paired supervision in the form *ECG image + pixel-level ground-truth mask*. Because real hospitals do not provide waveform masks, we generated a large synthetic segmentation dataset from PTB-XL signal recordings and used the generator outputs as ground truth for training.

Step 1: Synthetic ECG Page Generation and Layout Matching

Synthetic ECG pages were generated using the ECG Digitiser image generation pipeline.[1] We modified and tuned the rendering process so that the synthetic page alignment (lead placement, spacing, and rhythm-strip placement) matches the local printed layouts observed in Palestinian hospitals. This was done to reduce the domain gap between synthetic training data and real deployment images.

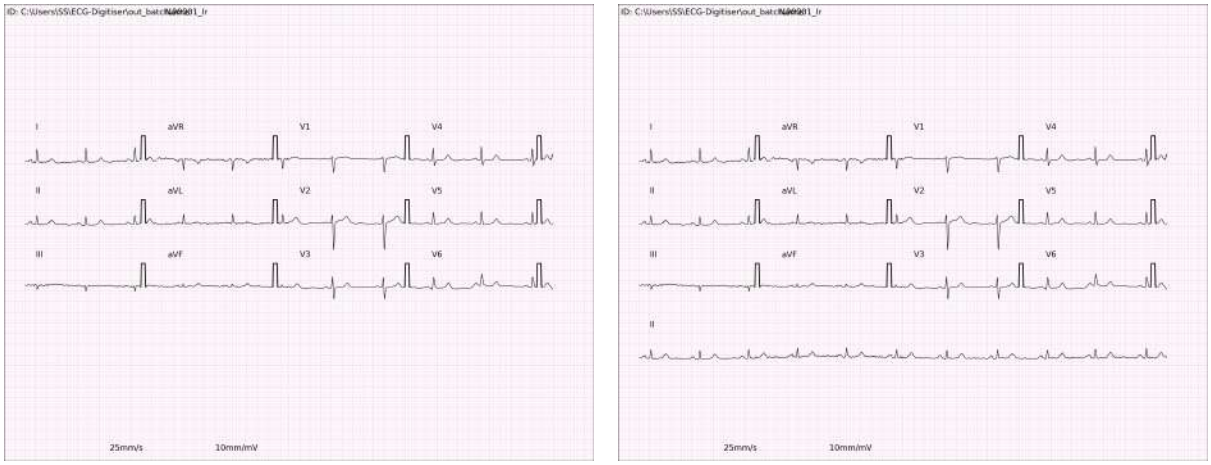
Two formats were produced:

- **3×4 format:** standard 12-lead printed layout.

[1] `https://github.com/felixkrones/ECG-Digitiser`

- **3×4 + rhythm strip format:** the same 12-lead layout with an additional long rhythm strip.

Figure 4.7 shows examples of the two generated page formats as they appear.



(a) 3×4 (b) 3×4 + rhythm

Figure 4.7: Synthetic ECG page formats generated from PTB-XL signals: (a) 3×4 layout, (b) 3×4 + rhythm strip layout. The generator settings were tuned to match local printed alignment observed in Palestinian hospitals.

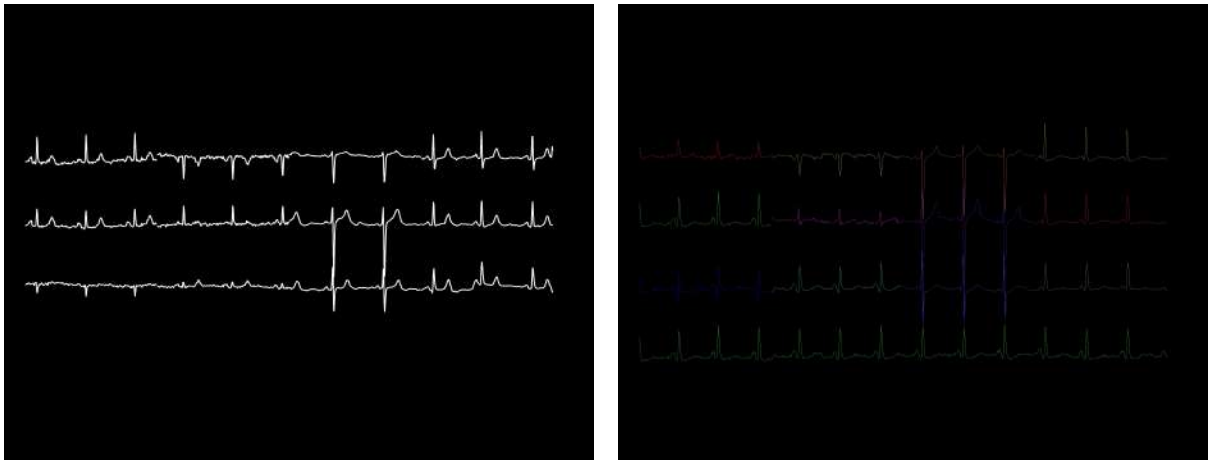
Step 2: Ground-Truth Mask Generation (Binary and Multi-class)

For each generated ECG page, the pipeline also produces a `.json` file containing waveform drawing metadata. We use this JSON information to render pixel-accurate segmentation masks aligned to the corresponding ECG image, enabling supervised segmentation training.

Two mask variants were prepared:

- **Binary masks:** waveform pixels vs. background.
- **Multi-class masks:** background plus waveform-related classes used in multi-class segmentation experiments.

Figure 4.8 shows examples of the produced binary and multi-class masks.



(a) Binary mask (b) Multi-class mask

Figure 4.8: Ground-truth masks generated from JSON metadata: (a) binary mask (waveform vs. background), (b) multi-class mask visualised using different colours.

Binary vs. Multi-class Masks (Final Training Choice)

We initially experimented with **multi-class** segmentation (labels 0–12) for the 3×4 + rhythm-strip layout to explicitly separate waveform-related regions. In practice, the multi-class setting degraded substantially, producing unstable class assignments and missing waveform segments (see Figure 5.8).

Because the digitisation objective is to recover a clean waveform trace (not to categorise page components), we adopted **binary segmentation** as the final training target for both layouts (*waveform vs. background*). The dataset preparation and paired augmentation pipeline remain identical to the multi-class setup described earlier; the only change is that the ground-truth mask is binary.



Figure 4.9: Binary ground-truth mask used for final nnU-Net training (waveform vs. background).

Qualitative examples of the remaining synthetic-to-real domain gap are reported later

in the Results chapter (see Figure 5.1).

Step 3: Alignment Verification (Overlay Inspection)

Because segmentation performance depends heavily on label correctness, we verified that each generated mask aligns precisely with the printed waveform by producing overlays (ECG image + mask) and visually inspecting correspondence. This verification step ensures that waveform labels coincide exactly with trace locations before training.



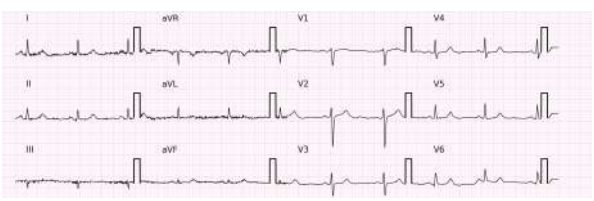
(a) 3×4 overlay

(b) 3×4 + rhythm overlay

Figure 4.10: Overlay verification examples demonstrating alignment between generated ECG pages and their ground-truth masks: (a) 3×4 layout, (b) 3×4 + rhythm strip layout.

Step 4: Cropping and Normalisation (Paired Image–Mask Processing)

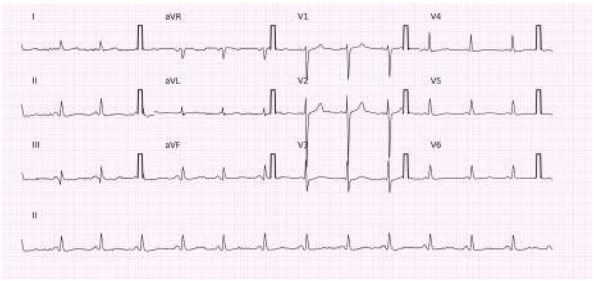
After generation, ECG pages were cropped and normalised into consistent input sizes suitable for nnU-Net training. All geometric operations were applied **identically** to both the ECG image and its corresponding mask to preserve pixel-level correspondence. For clarity, we visualised cropping using paired image–mask examples where the cropped ECG and its cropped mask are shown together.



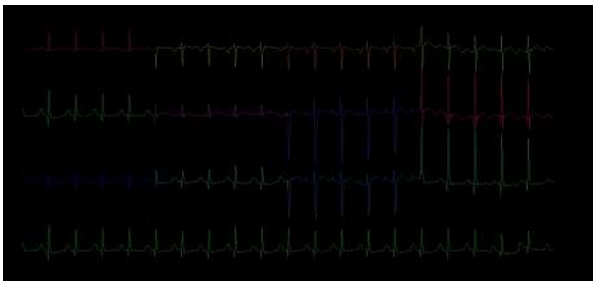
(a) 3×4 ECG (cropped)



(b) 3×4 mask (cropped)



(c) 3×4 + rhythm ECG (cropped)



(d) 3×4 + rhythm mask (cropped)

Figure 4.11: Cropped side-by-side verification examples for (top) 3×4 and (bottom) 3×4 + rhythm-strip layouts, showing ECG images and their corresponding masks.

Step 5: Paired Augmentation

To improve robustness and narrow the domain gap between perfectly clean synthetic pages and real acquisition conditions (camera/scanner re-sampling, uneven illumination, compression artifacts, blur, mild skew, and small rotations), we applied **paired augmentation** to every training sample. In paired augmentation, the ECG image and its ground-truth mask are processed **together as one unit** so that pixel-level correspondence is preserved.

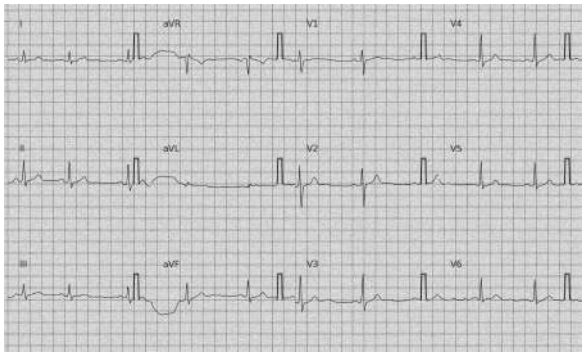
Geometric transforms (paired). All spatial transforms are applied **identically** to the ECG image and its mask using nearest-neighbor resampling for the mask to avoid creating invalid intermediate labels. We use mild settings to mimic realistic capture noise without distorting clinical morphology: small rotations, translations, and scaling; occasional mild perspective warp; and a camera/scanner resampling effect via **downscale–upscale** (downsample followed by upsample back to the original size). This ensures the mask remains a valid label map after augmentation.

Appearance transforms (image-only). Photometric and degradation effects are applied to the ECG image only (the mask is not altered), while keeping ECG–mask alignment unchanged. These include brightness/contrast and gamma shifts, mild blur (Gaussian and occasional motion blur), additive Gaussian noise, and **JPEG-like compression artifacts**. We additionally simulate real paper/scan characteristics using optional **paper texture blending**, **soft shadows**, **vignetting**, and rare vertical wrinkle-like artifacts. Augmentations are sampled stochastically per image to create diverse yet realistic variations.

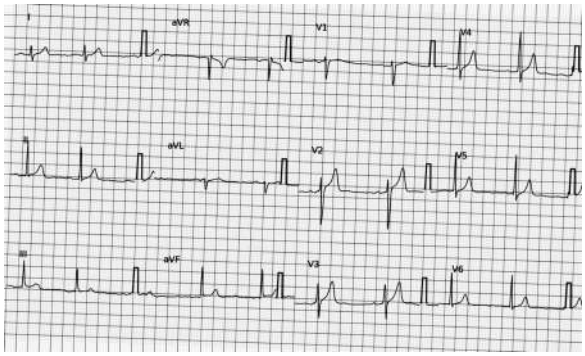
Storage-aware export. To reduce dataset size and I/O overhead during training,

augmented outputs can be exported at a controlled target width (only downscaling, never upscaling), optionally saved in **grayscale**, and written with high PNG compression. We generate multiple augmented copies per base sample and parallelize the pipeline using multiprocessing; when a fixed seed is provided, augmentations are reproducible.

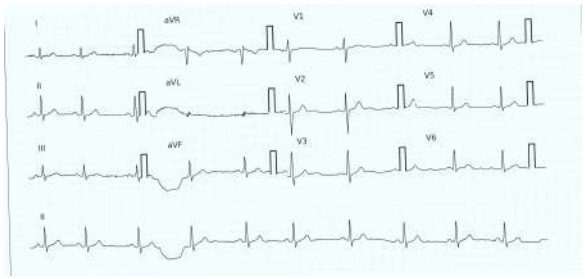
Figure 4.12 shows four augmentation examples (two per layout) illustrating the typical visual variations while maintaining correct ECG–mask alignment.



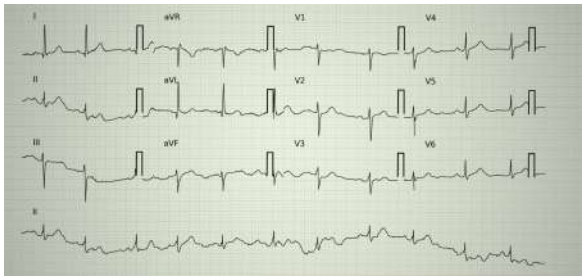
(a) 3×4 variation 1



(b) 3×4 variation 2



(c) 3×4+1 variation 1



(d) 3×4+1 variation 2

Figure 4.12: Examples of augmented synthetic ECG pages under two layouts: 3×4 (top row) and 3×4+1 with a rhythm strip (bottom row). Geometric transforms are applied jointly to the ECG and mask; appearance degradations are applied to the ECG only, preserving ECG–mask alignment.

Step 6: Remaining Domain Gap and Real-Data Constraints

Although augmentation substantially improves robustness, training only on synthetic ECG pages was not sufficient to fully close the domain gap to real hospital acquisitions. In practice, real ECG images contain device-specific printing artefacts, grid and paper variations, scanner streaks, shadows, compression, blur, and perspective distortions that are difficult to reproduce perfectly in synthetic rendering. This variability is illustrated by the locally collected example in Figure 4.5. As a result, a model that performs strongly on synthetic validation data can still degrade noticeably when applied to real hospital pages(see Figure 5.1).

A major limitation is the lack of paired supervision for real images: during our initial data collection at Bethlehem Arab Society for Rehabilitation Hospital, we were able to

collect many 3×4 ECG page images (e.g., Figure 4.5), but the corresponding digital waveforms were not available. Without the underlying digital signal, we cannot automatically generate pixel-accurate ground-truth masks for those real pages, which prevents large-scale supervised segmentation training directly on real hospital data.

Qualitative examples of this domain gap—including segmentation predictions on real hospital pages—are presented later in the Results chapter (see Figure 5.1). A full quantitative evaluation on manually labelled real pages is also reported in the Results chapter.

Step 7: Manual Labelling Tool and Real-Data Retraining Strategy

Even with heavy augmentation, synthetic data alone did not fully close the domain gap to real hospital ECG images. Early experiments showed that a segmentation model trained only on clean synthetic pages can still produce broken traces and grid confusion on real scans and phone photos(see Figure 5.1). Since paired supervision (real image + true signal) was not available at scale, we created a small real segmentation subset by manually tracing waveform masks and then incorporated it into the segmentation training pipeline without introducing data leakage.

To generate real masks, we built an internal Flask-based labelling tool using a browser-canvas workflow:

- the background layer displays the original ECG image (scan or smartphone photo),
- a transparent drawing layer allows us to trace the ECG waveform using a digital pen,
- basic editing tools (erase and clear) support corrections and detailed tracing,
- the final binary mask is saved with the same filename as the original image, ensuring exact alignment between each ECG image and its mask.

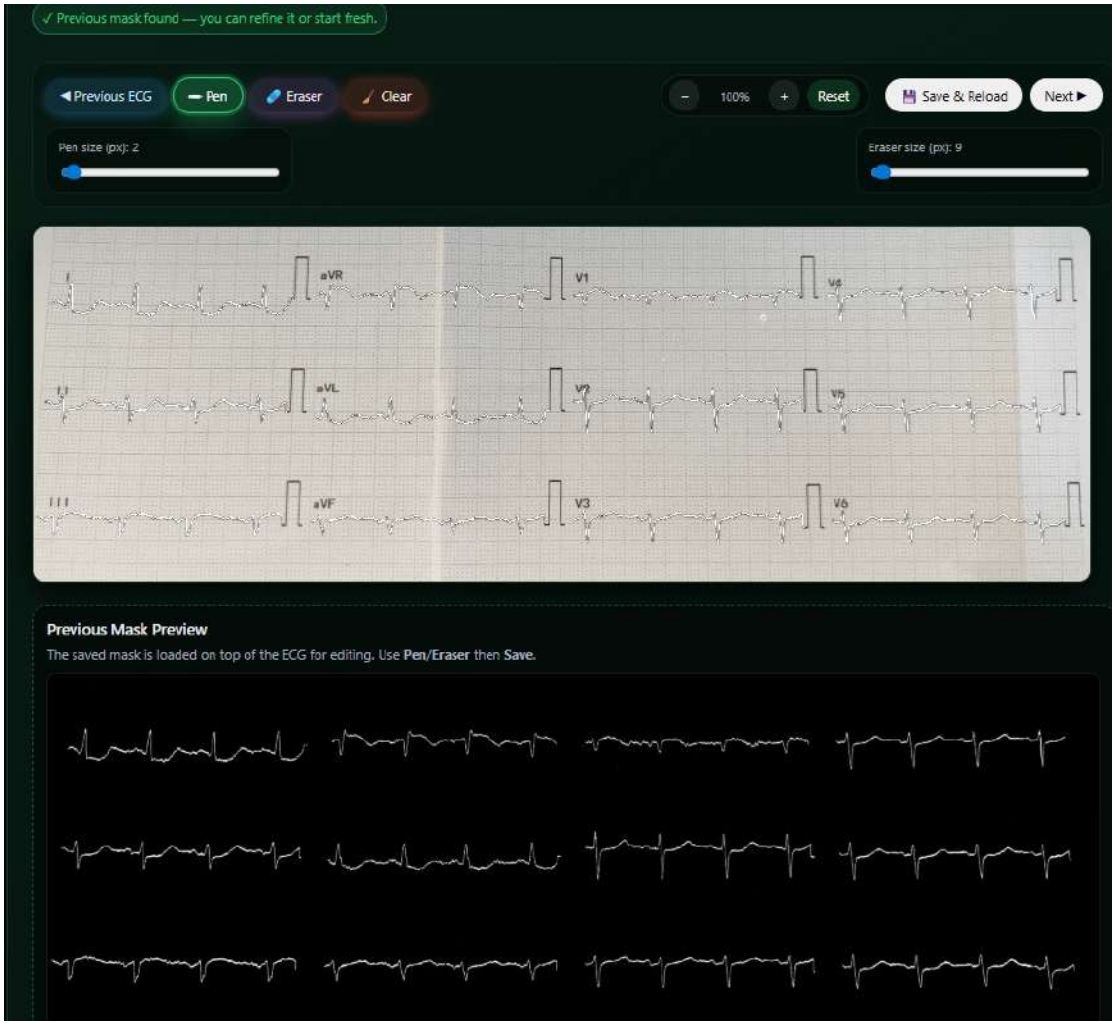


Figure 4.13: Flask web-based ECG labeling tool.

Due to time constraints, we manually annotated **100** real ECG pages. We then applied a controlled retraining strategy: the traced pages were split into **80** training pages and **20** validation pages, ensuring that no real page appears in both sets (preventing data leakage). Because the traced subset is small relative to the synthetic corpus, we used **resampling** of the real training subset during training to increase its effective contribution while keeping the real validation subset fixed.

In parallel, we continued to use a synthetic segmentation dataset generated from PTB-XL signals. The PTB-XL sampling was not fully random: because PTB-XL annotations are organised into five superclass groups, we constructed a roughly balanced synthetic subset by selecting approximately **1000 records per superclass** to diversify waveform morphologies and clinical patterns seen during training.

4.7.2 Diagnosis Dataset (ProtoECGNet) Preparation

The diagnosis component of Heart Guard is based on ProtoECGNet, an interpretable prototype-based architecture for multi-label ECG classification. While Section(s) earlier

in this chapter introduced PTB-XL in general, this subsection focuses on the exact dataset preparation steps used to train and evaluate the ProtoECGNet-based classifier in Heart Guard, following the protocol reported in the ProtoECGNet study [22].

Step 1: Record Selection from PTB-XL (100 Hz)

PTB-XL provides 12-lead, 10-second clinical ECG recordings at two sampling rates (100 Hz and 500 Hz) [12]. For ProtoECGNet training, we use the **100 Hz** version to match the original ProtoECGNet input design [22]. This yields exactly:

$$10 \text{ s} \times 100 \text{ Hz} = 1000 \text{ samples per lead},$$

resulting in a consistent length of **1000 samples** for each of the 12 leads per record.

Step 2: Label Extraction and Multi-Label Encoding (71 Diagnostic Labels)

PTB-XL annotations are provided as SCP-ECG statements. Following ProtoECGNet [22], we focus on the **71 diagnostic statement labels** (multi-label setting), where each ECG can have multiple concurrent diagnoses. Each record is encoded as a binary multi-hot vector:

$$\mathbf{y} \in \{0, 1\}^{71},$$

where $y_c = 1$ indicates the presence of diagnostic label c.

Step 3: Official Train/Validation/Test Split Using `strat_fold`

To ensure a reproducible and clinically meaningful evaluation, PTB-XL provides an official stratified split field `strat_fold` [12]. We adopt the split protocol used in ProtoECGNet [22]:

- **Training set:** folds 1–8
- **Validation set:** fold 9
- **Test set:** fold 10

This split preserves consistent label distribution across folds and supports fair comparison with the reference ProtoECGNet results.

Step 4: Baseline Wander Removal (High-Pass Butterworth, 0.5 Hz)

Before training, each 12-lead signal is filtered to remove baseline wander and low-frequency drift. Following ProtoECGNet [22], we apply a **first-order high-pass Butterworth**

filter with cutoff frequency **0.5 Hz** independently to each lead. This improves waveform stability while preserving diagnostically relevant morphology. In line with the ProtoECGNet protocol, **no low-pass filtering** is applied, to avoid removing potentially important high-frequency components used by morphology-based reasoning.

Step 5: Input Tensor Construction for ProtoECGNet Branches

After preprocessing, each ECG record is represented as a fixed-length 12-lead tensor. ProtoECGNet uses different views of the same underlying signal:

- **1D Rhythm Branch input:** a multi-channel time series with 12 channels and 1000 samples (conceptually 12×1000).
- **2D Branch inputs (Morphology and Global):** a time–lead matrix arranged as $(1 \times 12 \times 1000)$, where the vertical axis corresponds to leads and the horizontal axis corresponds to time [22].

This unified formatting ensures that all branches operate on the same ECG content while learning complementary representations.

Step 6: Branch-Specific Label Subsets (Clinical Decomposition)

To mirror clinical reasoning, ProtoECGNet divides the 71 diagnostic labels into three subsets reviewed by physicians [22]:

- **Rhythm-related labels:** 16 diagnoses (used to train the 1D rhythm branch).
- **Morphology-related labels:** 52 diagnoses (used to train the 2D morphology branch with localized prototypes).
- **Global/diffuse labels:** 3 diagnoses (used to train the 2D global branch).

Heart Guard follows the same decomposition: each branch is trained on its corresponding label subset, and final multi-label predictions are produced by fusing similarity scores from all branches during the fusion stage.

4.8 Final Datasets Statistics and Splits

To support Heart Guard's digitisation and diagnosis components, we prepared multiple datasets with consistent preprocessing and carefully controlled splits. For segmentation-based digitisation, we trained two separate nnU-Net models corresponding to the two ECG page layouts encountered in practice: **3×4** and **3×4 + rhythm-strip**. Each nnU-Net model was trained using **synthetic samples** rendered from PTB-XL signals (paired

with ground-truth masks) and, where available, **real hospital pages with manually traced binary masks**. The manually traced real subset was split into **80 training pages** and **20 validation pages** (no overlap). During training, the 80 real training pages were **resampled** to increase their effective contribution without introducing data leakage. In addition, to improve generalisation, the synthetic samples were constructed with balanced coverage of PTB-XL **five diagnostic superclasses** and included a mix of PTB-XL sampling rates (**50% 100 Hz and 50% 500 Hz**).

4.8.1 Segmentation Datasets (nnU-Net)

3×4 Dataset (Binary Segmentation)

Table 4.2 summarises the final dataset used to train the 3×4 model (binary mask: waveform vs. background). The synthetic subset was sampled to be approximately balanced across PTB-XL superclasses (≈1000 each) and contains an equal split of 100 Hz and 500 Hz sources.

Table 4.2: Final dataset statistics for nnU-Net segmentation on the 3×4 layout (binary masks).

Component	Count	Notes
Synthetic pages (PTB-XL rendered)	5000	Balanced sampling across PTB-XL diagnostic superclasses (≈1000 each). 50% from 100 Hz and 50% from 500 Hz.
Real pages (manually traced) – train (effective)	2480	Resampled from 80 unique real pages to increase effective training exposure.
Real pages (manually traced) – val	20	Held-out real pages (not resampled). No overlap with train.
Total training samples (effective)	**7480**	Synthetic + resampled real
Total validation samples	**20**	Real only

3×4 + Rhythm Dataset (Segmentation Training Progression)

For the 3×4 + rhythm-strip layout, we explored two segmentation targets:

1. **Multi-class segmentation (labels 0–12):** trained first on a large synthetic-only dataset (Section 4.8.1), but the resulting model did not generalise reliably and produced unstable label assignments (see Figure 5.8).
2. **Binary segmentation (final):** since the digitisation objective is waveform extraction, we switched to binary masks (waveform vs. background) and retrained the final model using an updated balanced dataset (Section 4.8.1).

Multi-class Synthetic Dataset (Initial Experiment)

Table 4.3 summarises the synthetic dataset used for the initial multi-class experiment on the 3×4 + rhythm-strip layout. This model was trained on AWS to accelerate experimentation; the training infrastructure and cost considerations are documented later in the Results/Deployment chapter.

Table 4.3: Initial synthetic dataset used for nnU-Net multi-class segmentation on the 3×4 + rhythm-strip layout (labels 0–12).

Component	**Count**	**Notes**
Synthetic pages (PTB-XL rendered) – originals	6667	Balanced sampling across PTB-XL diagnostic superclasses (≈1333 each). 50% from 100 Hz and 50% from 500 Hz.
Augmentation factor	×3	Paired augmentation applied to **both** ECG images and masks (same geometry) to preserve alignment.
Synthetic pages (after augmentation) – total	20000	Augmented dataset used for the training/validation split (rounded to target size).
Synthetic pages (after augmentation) – validation	2655	Random held-out validation split.
Synthetic pages (after augmentation) – training	17345	Remaining samples used for training.

Binary Synthetic Dataset (Final Model)

After observing poor multi-class generalisation (see Figure 5.8), the final 3×4 + rhythm-strip model was trained using **binary masks** (waveform vs. background; see Figure 4.9) , while keeping the same generation, cropping, and paired-augmentation pipeline described earlier. The updated synthetic dataset was constructed with a strict 50/50 split between 100 Hz and 500 Hz sources and balanced coverage of PTB-XL diagnostic superclasses.

Table 4.4: Final synthetic dataset statistics for nnU-Net binary segmentation on the 3×4 + rhythm-strip layout.

Component	Count	Notes
Synthetic pages (PTB-XL rendered) – total	7500	50% from 100 Hz (lr) and 50% from 500 Hz (hr). Balanced across the five PTB-XL diagnostic superclasse
Superclass counts (lr, 100 Hz)	3750	NORM: 851, MI: 804, STTC: 780, CD: 767, HYP: 54
Superclass counts (hr, 500 Hz)	3750	NORM: 844, MI: 811, STTC: 783, CD: 764, HYP: 54
Training/validation split	–	Applied after generation (final split reported in Results).

4.8.2 ProtoECGNet Dataset Preparation and Splits

To support the diagnostic component of Heart Guard, we prepared a dedicated dataset derived from PTB-XL for training and evaluating the ProtoECGNet architecture. In contrast to the image-based digitisation datasets used for segmentation, this dataset operates directly on 12-lead ECG signals and follows a strictly controlled, patient-wise split to enable reliable multi-label classification and prototype-based interpretability.

All ECG recordings were represented as fixed-length signals sampled at 100 Hz, resulting in tensors of size 12×1000 per record. Diagnostic labels were encoded in a multi-label format following the SCP-ECG annotation scheme provided by PTB-XL. The preprocessing and label preparation steps described in Section 4.7.2 were applied consistently across all subsets.

Table 4.5: Final dataset statistics and splits for ProtoECGNet training and evaluation using PTB-XL.

Subset	PTB-XL folds	Usage
Training set	1–8	Learning branch-specific representations and prototype embeddings.
Validation set	9	Hyperparameter tuning and model selection (no prototype updates).
Test set	10	Final performance evaluation and qualitative interpretation.

ProtoECGNet employs a case-based reasoning mechanism in which each learned prototype is explicitly projected onto a real ECG segment from the **training set**. During

inference, predictions are explained by retrieving and comparing the most similar training prototypes. Therefore, validation and test recordings must remain strictly isolated from the prototype learning and projection stages to prevent information leakage and to preserve the validity of both quantitative metrics and qualitative explanations.

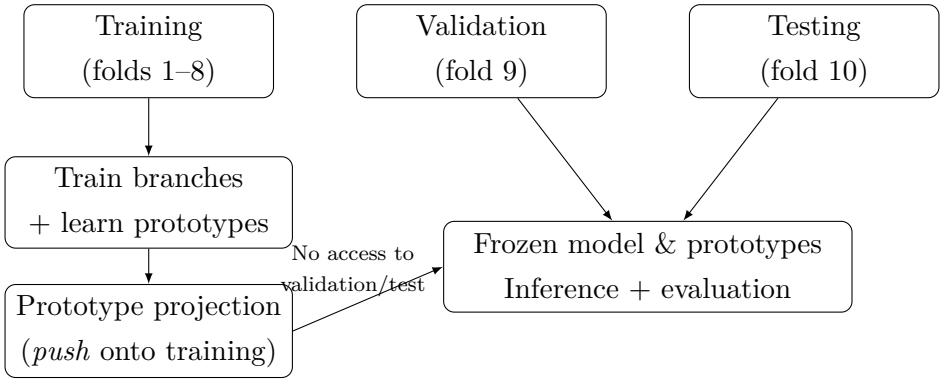


Figure 4.14: Leakage-free dataset splitting strategy for ProtoECGNet. Prototypes are learned and projected exclusively from the training set, while validation and test sets are used only for evaluation and explanation retrieval.

Chapter 5

Segmentation Models and Training Strategy

Contents

5.1 Problem Setting and Key Challenge: Domain Gap 41

5.2 Baseline Model: U-Net 42

5.2.1 Why U-Net? 42

5.2.2 U-Net architecture overview 42

5.2.3 Baseline implementation used in this work 42

5.3 Experiment 1: U-Net Trained on Purely Synthetic Data . . 43

5.3.1 Training behaviour 43

5.3.2 Qualitative output on synthetic data 43

5.3.3 Failure on real ECG scans (domain gap) 43

5.4 Why We Moved to nnU-Net 44

5.4.1 What nnU-Net is (and why it helps) 44

5.5 Experiment 2: nnU-Net with Augmentation (Limited Real Labels) 44

5.5.1 Motivation 44

5.5.2 Qualitative improvement (still imperfect) 45

5.6 Real Data Labelling: Flask-Based Annotation Tool 45

5.7 Final Binary Segmentation Model (3×4 Layout): Training on Mixed Data 45

5.7.1 Dataset composition and validation protocol 45

5.7.2 Cross-validation evidence from training logs 45

5.7.3 Cross-validation results (Mean Dice) 47

5.7.4 Qualitative final prediction on real data 47
5.8 Attempted Multi-Class Segmentation (0–12): 47
5.9 Binary vs. Multi-Class Segmentation: Trade-offs for ECG Waveform Extraction 48
5.10 Summary of Model Progression 49

This chapter describes the segmentation models explored to extract ECG waveforms from scanned hospital pages, the reasons behind each design decision, and a summary of the obtained results. The work progressed from a baseline U-Net trained on synthetic data (high validation Dice but poor real-world generalisation) to an nnU-Net-based pipeline that reduced manual tuning, leveraged augmentation, and finally incorporated real labels collected through an internal web-based annotation tool.

5.1 Problem Setting and Key Challenge: Domain Gap

The segmentation task is formulated as pixel-wise classification to separate ECG trace pixels from background. Although training on fully synthetic ECG pages is convenient and scalable, the synthetic distribution differs noticeably from real hospital scans (noise patterns, contrast, paper artifacts, grid appearance, printing intensity, and scanner/phone capture effects). This domain gap led to deceptively high synthetic validation Dice while producing degraded masks on real pages.

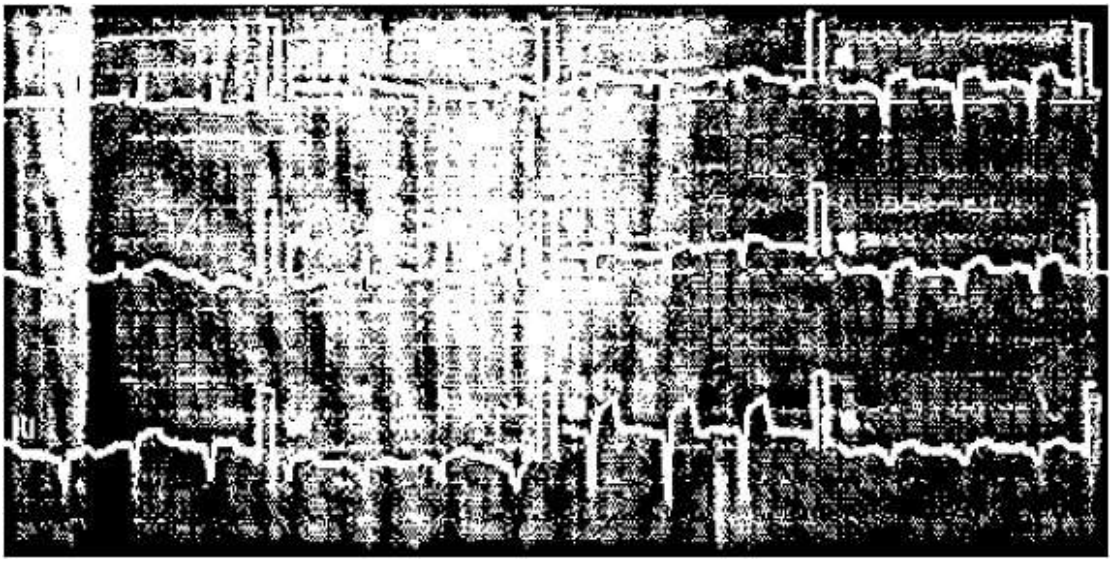


Figure 5.1: Example of segmentation degradation on a real hospital ECG page when training is synthetic-only: the predicted mask is incomplete and partially confuses grid lines with the waveform, illustrating the remaining domain gap.

5.2 Baseline Model: U-Net

5.2.1 Why U-Net?

U-Net is a widely used encoder–decoder architecture for medical image segmentation. Its core strength is the **skip connections** that pass spatial detail from the encoder to the decoder, which is especially useful when the target structure is thin and high-frequency (as ECG traces are).

5.2.2 U-Net architecture overview

U-Net has two symmetric paths:

- **Encoder (contracting path):** repeated *Conv–Norm–ReLU* blocks followed by downsampling. This increases receptive field and extracts higher-level features while reducing spatial resolution.
- **Decoder (expanding path):** upsampling (often via transposed convolution) followed by convolution blocks. At each scale, features are concatenated with encoder features from the corresponding level (**skip connections**), restoring fine detail.

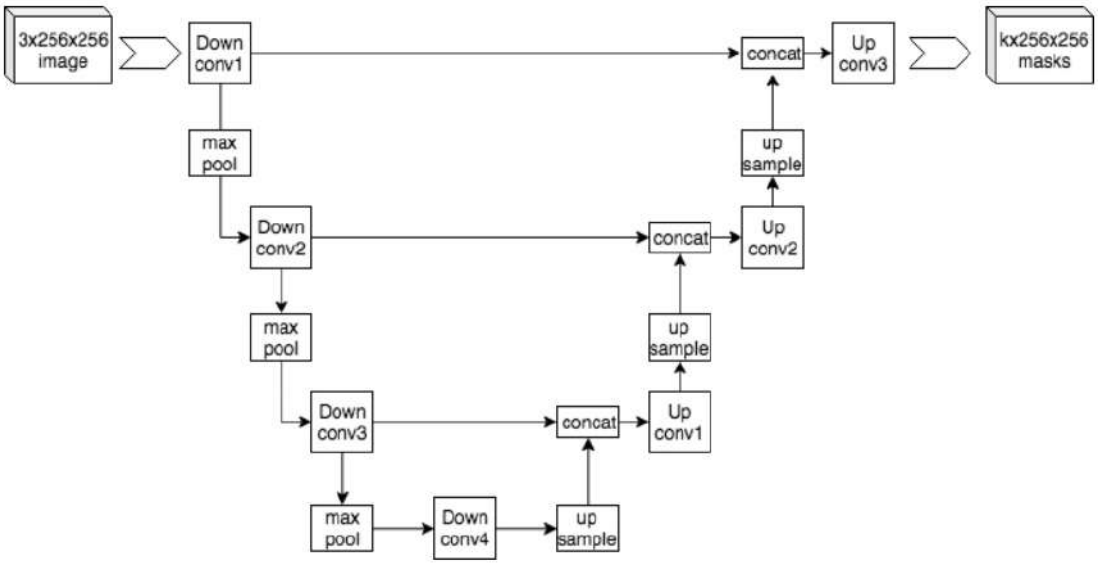


Figure 5.2: Conceptual U-Net encoder–decoder with skip connections (diagram from Wikimedia Commons). [18, 25]

5.2.3 Baseline implementation used in this work

The initial baseline was a compact U-Net variant with:

- Single-channel input (grayscale ECG page)

- Binary output mask (trace vs background)
- BatchNorm and ReLU in each double-convolution block
- Transposed convolutions for upsampling

This implementation is shown in the project code listing.

5.3 Experiment 1: U-Net Trained on Purely Synthetic Data

5.3.1 Training behaviour

When trained only on synthetic samples, the U-Net quickly reached very high validation Dice (e.g., ∼0.96–0.97 within the first few epochs). This high score reflects **internal consistency** within the synthetic distribution rather than true generalisation.

5.3.2 Qualitative output on synthetic data

The model outputs were visually strong on synthetic pages, with close agreement between the ground-truth mask and prediction.



Figure 5.3: U-Net trained on synthetic-only data: qualitative comparison of ground-truth (GT) mask and predicted mask on a synthetic ECG page.

5.3.3 Failure on real ECG scans (domain gap)

Despite excellent synthetic validation metrics, the same model produced degraded masks on real pages (Figure 5.1). This confirmed that the main limitation was not model capacity, but **distribution mismatch** between synthetic and real ECG page characteristics.

5.4 Why We Moved to nnU-Net

After observing that manual tuning of preprocessing, augmentation strength, learning-rate schedules, and architecture depth could lead to unstable conclusions, we searched for an approach that:

- provides strong segmentation defaults without extensive hyperparameter sweeping,
- adapts preprocessing and training configuration automatically,
- supports robust cross-validation for reliability.

5.4.1 What nnU-Net is (and why it helps)

nnU-Net is a self-configuring framework built around a U-Net backbone. Instead of requiring the user to manually choose many training settings, nnU-Net automatically determines key pipeline components from the dataset (patch size, resampling strategy, normalisation, training schedule, and several architecture/training defaults). In practice, this greatly reduces the risk of "tuning by trial-and-error" and makes results more reproducible across folds.

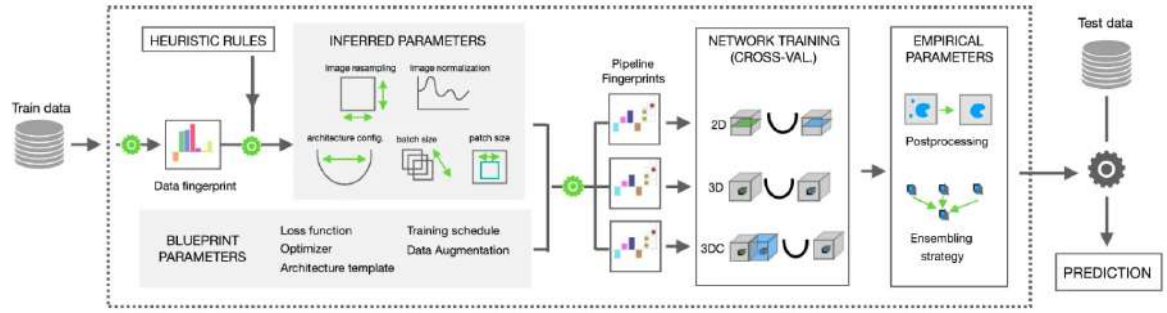


Figure 5.4: High-level nnU-Net workflow (overview diagram from the official nnUNet repository) illustrating dataset fingerprinting, rule-based planning, training, and postprocessing. [19, 24]

5.5 Experiment 2: nnU-Net with Augmentation (Limited Real Labels)

5.5.1 Motivation

At this stage, real labelled data were scarce. To improve robustness, we introduced augmentation to simulate real-world variation (e.g., intensity shifts, noise, small geometric perturbations). The goal was to reduce the synthetic-to-real gap without relying on large real-labelled sets.

5.5.2 Qualitative improvement (still imperfect)

This model still struggled on some real scans, but its masks were more complete and less prone to severe fragmentation. A representative example is shown below.

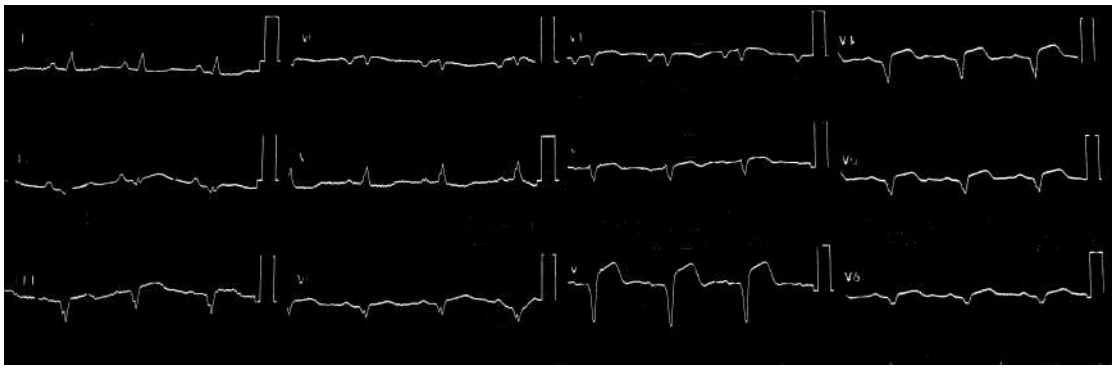


Figure 5.5: First nnU-Net-based model with augmentation: example segmentation on a real ECG page showing improved continuity relative to the synthetic-only U-Net baseline, but still with remaining artifacts and missed segments.

5.6 Real Data Labelling: Flask-Based Annotation Tool

As shown in Figure 4.13, the internal Flask-based labelling tool allows tracing ECG waveforms directly over real hospital pages to generate aligned binary masks for supervised training.

5.7 Final Binary Segmentation Model (3×4 Layout): Training on Mixed Data

5.7.1 Dataset composition and validation protocol

The final 3×4 binary segmentation dataset contained 7500 samples, including 2480 real labelled pages. A small held-out validation split was used (20 samples), and we further trained using 5-fold cross-validation to ensure that results were not due to a lucky split.

5.7.2 Cross-validation evidence from training logs

nnU-Net reports training progress automatically (loss curves, pseudo-dice curves, learning rate schedules). Figure 5.6 shows an example of the built-in training progress visualisation.

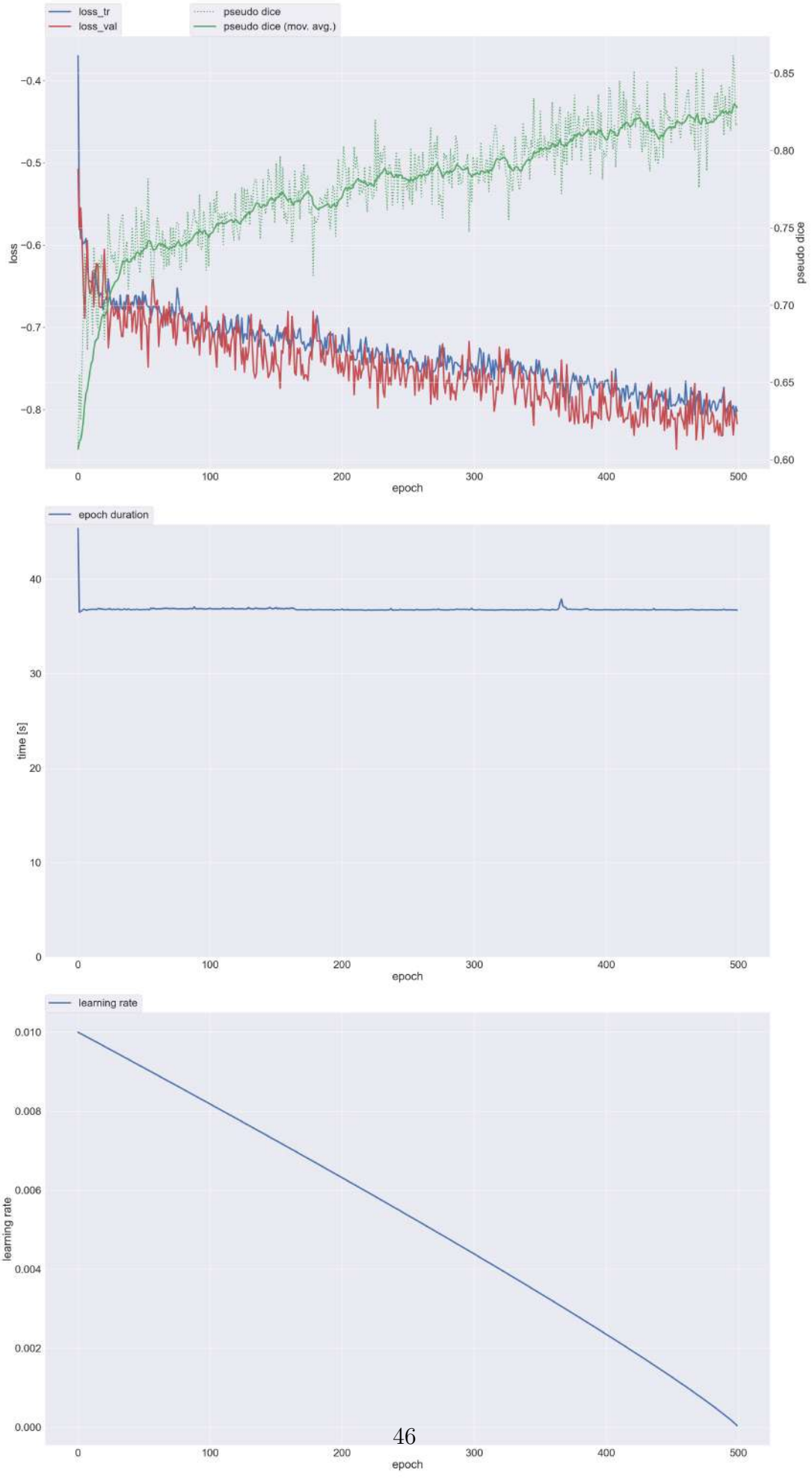


Figure 5.6: nnU-Net training progress example (Fold 1): training/validation loss,

Across folds, the logs show steady improvement and strong late-epoch performance. For example, one fold reached an EMA pseudo-dice of 0.7574 around epoch 453, demonstrating stable optimisation later in training. Other folds reached ∼0.81–0.82 range in later epochs (e.g., EMA pseudo-dice ≈0.8212 around epoch 477).

5.7.3 Cross-validation results (Mean Dice)

Table 5.1 reports the mean validation Dice obtained for the final binary 3×4 model across the five cross-validation folds.

Fold	Mean Validation Dice
0	0.7797
1	0.8545
2	0.8598
3	0.8519
4	0.8489
Average	0.8390

Table 5.1: Final binary 3×4 segmentation model: mean validation Dice across five folds.

5.7.4 Qualitative final prediction on real data

The final model produced visually consistent masks on real ECG pages, capturing the waveform while suppressing background noise.



Figure 5.7: Final binary segmentation model (3×4 layout): qualitative example on a real ECG page showing good waveform continuity and alignment with the underlying trace.

5.8 Attempted Multi-Class Segmentation (0–12):

After establishing a strong binary segmentation model, we attempted multi-class segmentation (classes 0–12) on the 3×4-plus-rhythm-strip layout to separate leads and trace

components more explicitly. The model was trained for 1000 epochs per fold across five folds. However, performance remained consistently poor, with the mean validation Dice hovering around 0.41 across folds:

Fold	Mean Validation Dice
0	0.4093
1	0.4110
2	0.4116
3	0.4119
4	0.4141
Average	0.4116

Table 5.2: Attempted multi-class segmentation (0–12): mean validation Dice across five folds.

Qualitatively, predictions showed unstable class assignments and missing waveform segments, indicating poor generalisation despite apparently correct label presence in the downloaded dataset.

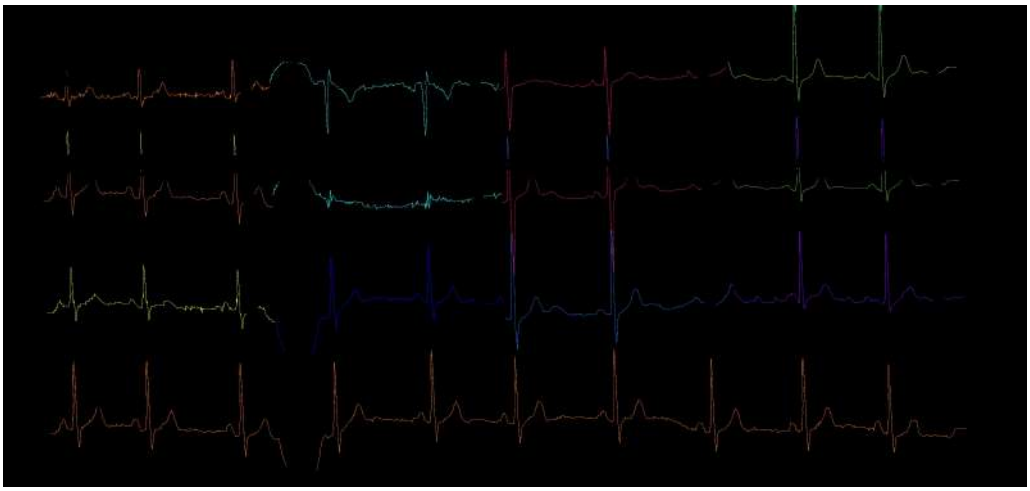


Figure 5.8: Failure case of the multi-class segmentation model: predictions show inconsistent class labels and missing waveform segments, indicating unstable training and poor generalisation.

Given these results, we reverted to binary mask training for the 3×4-plus-rhythm layout, with the final binary model intended to be ready for discussion day.

5.9 Binary vs. Multi-Class Segmentation: Trade-offs for ECG Waveform Extraction

In this project, binary segmentation (trace vs. background) was the most reliable choice for digitisation. Although multi-class segmentation is attractive because it can, in princi-

ple, separate lead regions or components explicitly, it introduces extra supervision complexity and is more sensitive to small annotation inconsistencies—especially when the target structure is a thin waveform and the real scans contain strong artifacts. Table 5.3 summarises the practical trade-offs observed.

Aspect	**Binary (trace vs. background)**	**Multi-class (0–12)**
Objective	Detect all waveform pixels regardless of lead identity.	Assign each pixel to a specific class/lead (plus background).
Annotation effort	Lower: only one foreground class; faster and less error-prone tracing.	Higher: requires consistent class mapping; mistakes in boundaries or class identity are costly.
Robustness to domain gap	Typically higher: model focuses on waveform shape and continuity.	Lower: scan artifacts/grid/noise can cause class confusion and fragmented predictions.
Training stability	More stable optimisation (simpler loss landscape).	Harder optimisation: class imbalance and fine boundaries increase instability.
Failure modes	Mainly false positives from grid/noise, or missing faint segments.	In addition to missing segments, can produce label swapping and inconsistent class regions.
Output usefulness for digitisation	Strong: a clean binary mask is sufficient for downstream waveform extraction and signal reconstruction.	Potentially useful if it worked well (per-lead separation), but unreliable when classes collapse or swap.
Observed results in this work	Strong qualitative performance on real pages after adding real labels and augmentation.	Poor mean validation Dice ($\sim$0.41) and unstable qualitative predictions across folds.
Best use-case	When the downstream step only needs "where is the trace?"	When downstream explicitly needs per-lead segmentation and labels are abundant and consistent.

Table 5.3: Practical comparison of binary and multi-class segmentation for ECG waveform extraction. In this work, binary masks were sufficient for digitisation and substantially more robust than multi-class training under real-scan variability.

5.10 Summary of Model Progression

The overall progression can be summarised as:

1. **Pure U-Net on synthetic data:** excellent synthetic Dice but poor real performance due to domain gap (Figure 5.1).

2. **nnU-Net with augmentation:** improved robustness with fewer manual decisions; still limited by lack of real labels (Figure 5.5).

3. **Final nnU-Net on mixed real+synthetic labels (binary):** strong qualitative real segmentation and stable training trends across folds (Figures 5.6 and 5.7).

4. **Multi-class (0–12) attempt:** consistently low validation Dice and unstable predictions (Figure 5.8), motivating a return to binary segmentation for reliable digitisation.

Chapter 6

ECG Digitisation and Quantitative Evaluation

Contents

6.1 Digitisation Overview: Binary Mask to 12 Leads 52

6.2 Pre-Digitisation Mask Cleaning 52

6.3 ROI Splitting into 12 Lead Cells (3×4) 53

6.4 Trace Extraction Inside Each ROI 54

6.4.1 Baseline estimation 54

6.4.2 Two extraction modes used in this work 54

6.4.3 Handling missing columns without forcing straight lines 54

6.4.4 Calibration block detection and removal 55

6.4.5 Debug overlays 55

6.5 Pixel-to-mV Conversion and Resampling 55

6.5.1 Safe smoothing (optional, mild) 56

6.6 Quantitative Evaluation Against Reference aECG (HL7) . 56

6.6.1 Time-window matching (critical) 56

6.6.2 Alignment: best shift based on morphology 56

6.7 Evaluation Metrics and Their Meaning 57

6.7.1 Shape similarity: correlation $r(z)$ 57

6.7.2 Amplitude accuracy: MAE and RMSE (mV) 57

6.7.3 Pixel consistency (measured correctly on ROI grid) 57

6.7.4 Calibration scale used (`px`) 58

6.8 What Improved Digitisation Quality in Practice 58

6.8.1 Why top/farthest selection failed 58

6.9 Qualitative Output: Digitised 12-Lead Sheet 59
6.10 Reporting of Digitisation Logs 59

This chapter describes how we convert the predicted **binary segmentation mask** (trace vs. background) into **12 digitised ECG leads** in millivolts, and how we evaluate the resulting signals against a reference **aECG (HL7)** recording. The digitisation stage is designed to be robust to realistic mask imperfections (small gaps, thick traces, missing columns), and to preserve waveform morphology without forcing an artificial straight alignment across leads.

6.1 Digitisation Overview: Binary Mask to 12 Leads

The digitiser takes a binary mask in which foreground pixels represent the ECG trace. The processing is performed in four main stages:

1. **Mask cleaning:** mild morphological closing to repair tiny gaps.
2. **ROI splitting:** divide the page into the standard **3×4** lead layout (12 cells).
3. **Trace extraction per ROI:** convert each ROI to a single polyline $y(x)$ (one point per column when available).
4. **Signal conversion:** resample to a fixed length and convert pixels to **mV** using baseline and calibration (px/mV), then apply optional safe smoothing.

Figure 6.1 shows an example binary mask used as the digitiser input.

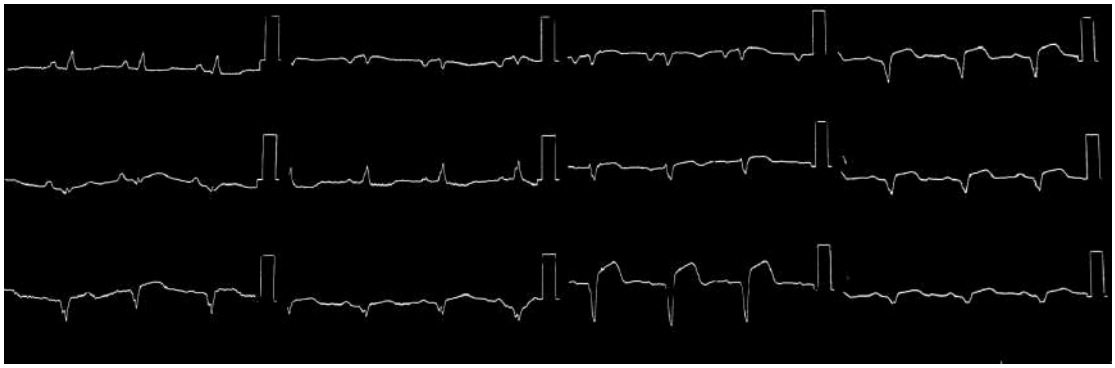


Figure 6.1: Binary segmentation mask used as digitiser input (foreground = ECG trace).

6.2 Pre-Digitisation Mask Cleaning

Real ECG masks often contain small discontinuities due to printing artifacts, grid interference, or weak trace intensity. We apply a **mild morphological close** (kernel size 3,

one iteration) to fill tiny gaps while preserving waveform peaks. Importantly, we avoid aggressive erosion/opening by default, since it can shave sharp QRS tips and attenuate amplitude.

- **Enabled:** `CLOSE(ksize=3, iterations=1)`
- **Disabled by default:** `OPEN` (only used for ablations, not for final results)

6.3 ROI Splitting into 12 Lead Cells (3×4)

To digitise each lead independently, the mask is split into **12 regions of interest (ROIs)** using a robust separator-detection method:

- Two **horizontal separators** are detected globally from a row occupancy profile (empty-band detection).
- For each row band, three **vertical separators** are detected from a column occupancy profile.
- Each lead boundary is then refined locally by snapping edges to nearby **valley columns** (low-occupancy minima), which avoids cutting through traces or calibration edges.

This approach intentionally avoids enforcing one global straight set of separators for the entire page. Each row band can have slightly different separator locations, and each individual lead edge can be refined independently, matching real scanned pages where lead boxes are not perfectly aligned.

Figure 6.2 shows the detected 12 ROIs overlayed on the mask.

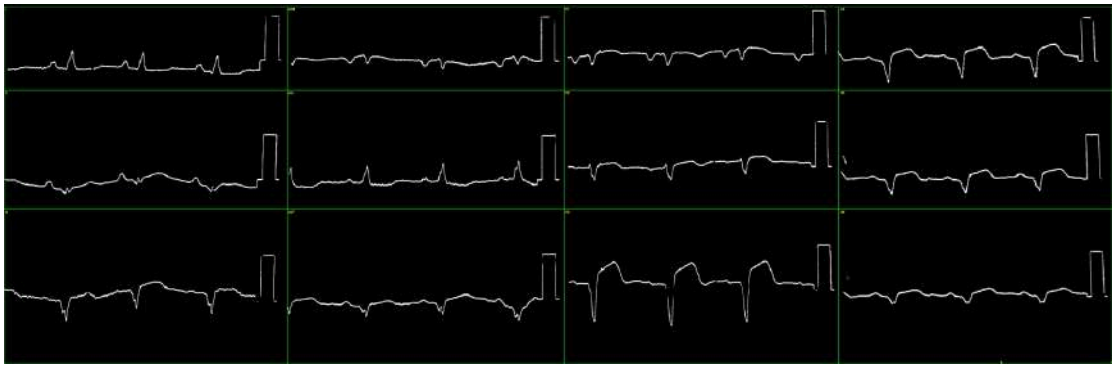


Figure 6.2: ROI splitting result: detected 12 lead cells (3×4) overlayed on the mask. Each lead is digitised independently.

6.4 Trace Extraction Inside Each ROI

Within each lead ROI, the goal is to convert the binary trace pixels into a single continuous curve $y(x)$ in pixel coordinates. For each column x:

1. Collect all foreground rows $\{y : \text{mask}(y, x) = 1\}$.
2. Summarise them into one representative $y(x)$.

A key difficulty is that ECG traces are frequently **thick strokes** in the mask. If we always pick the topmost or bottommost pixel, the extracted curve can jump between the upper and lower boundary of the same thick stroke, creating non-physiological discontinuities.

6.4.1 Baseline estimation

A per-ROI baseline $\hat{y}_{base}$ is estimated using the **row occupancy mode** in a middle band of the ROI. This baseline helps stabilise extraction decisions near steep spikes.

6.4.2 Two extraction modes used in this work

We evaluated two practical extraction strategies:

- **Mean/median extraction (baseline):** choose the column median (or mean) of foreground rows. This is stable for thick traces but can slightly flatten very sharp spikes.
- **Spike-aware extraction (final):** detect spike-like columns using the vertical span of foreground pixels. If the span is large (QRS-like), choose a consistent extreme (top or bottom) guided by the predicted continuation and baseline side; otherwise fall back to a stable representative (median/top/bottom selection with penalties).

The spike-aware method reduces edge-switching and preserves QRS peaks better under thick masks, while still being robust to missing columns.

6.4.3 Handling missing columns without forcing straight lines

Some columns in the ROI may contain no foreground pixels (gaps). We **do not force a straight line** across missing columns. Instead:

- The extractor simply skips missing columns.
- The available (x, y) points are later resampled by interpolation to a fixed length.

This preserves the natural horizontal alignment differences across leads (we do not enforce identical starting points), and it avoids injecting artificial constant segments where the mask has no evidence.

6.4.4 Calibration block detection and removal

Many printed ECG cells include a calibration rectangle near the edge. If not removed, its vertical edges can be mistakenly digitised as part of the ECG trace. We therefore detect a calibration-like connected component near the right side of the ROI and apply a cutoff:

- Detect tall, thin-ish component near the right edge with low/medium fill ratio (outline rectangle).
- Compute a cutoff x_{cut} and **zero out** everything to the right of x_{cut}.
- Additionally trim extracted (x, y) points at or beyond x_{cut}.

This prevents the digitiser from "locking" onto the calibration box edges.

6.4.5 Debug overlays

For verification, we store per-lead debug overlays showing extracted points in red over the ROI mask. Figure 6.3 shows an example montage.

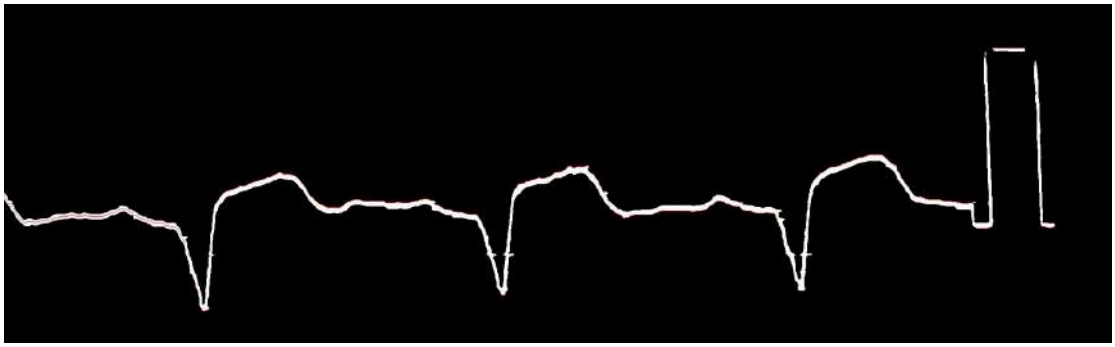


Figure 6.3: Per-lead extraction overlays (example): extracted polyline points (red) plotted over each ROI mask.

6.5 Pixel-to-mV Conversion and Resampling

After extracting (x, y) points from an ROI, we resample the curve to a fixed number of samples N (e.g. 1000) using linear interpolation over x. The resulting pixel-space signal $y[n]$ is converted to millivolts using:

$$V[n]\ [\text{mV}] \ = \ \frac{\hat{y}_{base} - y[n]}{\text{px_per_mV}}$$

where $\hat{y}_{base}$ is the estimated baseline row (in pixels), and `px_per_mV` is the calibration scale factor.

6.5.1 Safe smoothing (optional, mild)

To remove small digitisation jitter while preserving ECG morphology (especially QRS peaks), we apply a conservative two-stage smoother:

- **Hampel filter** to suppress isolated outliers caused by pixel glitches.
- **Savitzky–Golay** smoothing to reduce high-frequency staircase noise while preserving sharp transitions.

We emphasise that smoothing is intentionally mild to avoid attenuating clinically relevant features (QRS amplitude, ST shape). All reported results clearly state whether smoothing was enabled.

6.6 Quantitative Evaluation Against Reference aECG (HL7)

To quantitatively verify digitisation quality, we compare the digitised signals against reference ECG waveforms stored in **aECG HL7** files. Each aECG file contains 12 leads sampled at a known frequency f_s.

6.6.1 Time-window matching (critical)

The printed page corresponds to a specific time window (e.g. 2.5 seconds). Directly comparing the entire aECG recording to a digitised 2.5s strip is incorrect. Therefore, we first extract a segment:

- Select a start time t_0 (either provided or searched automatically).
- Crop aECG leads to $[t_0, t_0 + T]$ where T is the segment duration (e.g. 2.5s).
- Resample the cropped segment to the same target length N as the digitised output.

6.6.2 Alignment: best shift based on morphology

Even with correct time windowing, small offsets can exist between the printed representation and extracted waveform. We therefore compute an optimal small shift (in samples) that maximises morphological agreement.

Two alignment stages are used:

1. **Derivative-based shift estimation:** estimate the best integer shift by maximising correlation between **z-scored first derivatives**. This is robust because it aligns edges (QRS slopes) rather than raw baseline.
2. **Sub-sample refinement (optional):** refine around the best integer shift using a Fourier fractional shift (FFT shift theorem).

This shift is used for reporting "after alignment" metrics such as MAE/RMSE and correlation.

6.7 Evaluation Metrics and Their Meaning

For each case, we compute both per-lead and aggregated metrics:

6.7.1 Shape similarity: correlation $r(z)$

We report Pearson correlation after z-normalisation:

- `best r(z)`: best correlation achieved after searching over small shifts.
- `r(z) after shift`: correlation after applying the selected shift.

High values (e.g. ≈ 0.8–0.95) indicate strong morphological agreement.

6.7.2 Amplitude accuracy: MAE and RMSE (mV)

We report mean absolute error and RMSE in millivolts:

- `MAE` and `RMSE` before alignment.
- `MAE after shift` and `RMSE after shift` after applying the optimal shift.

6.7.3 Pixel consistency (measured correctly on ROI grid)

To ensure the extracted curve is genuinely supported by the mask (not an interpolation artifact), we compute:

- **Hit-rate (`hit`)**: fraction of extracted (x, y) points that land on a foreground pixel in the ROI.
- **Mean pixel error (`pxerr`)**: mean distance (in pixels) from extracted $y(x)$ to the nearest foreground pixel in each sampled column.

Crucially, these metrics are computed on the **native ROI sampling grid** (original extracted (x, y) points), not on the resampled 1000-length signal, which would otherwise produce misleading values.

6.7.4 Calibration scale used (`px`)

We report `px_per_mV` used per case. In early experiments, using a fixed value (e.g. 40 px/mV) caused systematic amplitude mismatch on scans with different scale. We therefore optionally perform a coarse grid search per case to select the `px_per_mV` that maximises shape similarity.

6.8 What Improved Digitisation Quality in Practice

Based on development experiments and logged evaluation output, the following changes produced clear improvements:

1. **Spike-aware extraction (final):** avoids switching between upper and lower stroke boundaries in thick masks, while preserving QRS peaks.
2. **Calibration cutoff removal:** prevents digitising calibration box edges as waveform.
3. **Correct evaluation of hit/pxerr:** computed on native ROI points, making the quality indicators meaningful.
4. **Adaptive px-per-mV search (optional):** avoids amplitude distortion when scan scale differs from a fixed assumption.
5. **Derivative-based alignment:** stable shift estimation using slopes rather than raw amplitude, improving correlation and reducing post-shift MAE.

6.8.1 Why top/farthest selection failed

We also tested extreme selection strategies:

- **Top-only:** often fails to follow full morphology when the trace thickens or touches boundaries; tends to under-represent peaks and segments.
- **Farthest (top/bottom extremes without stability constraints):** can flip between upper and lower stroke edges across columns, creating non-physiological jumps that degrade correlation and inflate error.

For these reasons, the final pipeline uses mean/median extraction as a baseline and spike-aware extraction as the final robust mode.

6.9 Qualitative Output: Digitised 12-Lead Sheet

After digitising all 12 ROIs, we assemble the digitised signals into a standard 12-lead layout for inspection. Figure 6.4 shows an example reconstructed sheet-like plot produced from the digitised mV signals.



Figure 6.4: Final digitised 12-lead signal plot reconstructed from the extracted mV signals (3×4 layout).

6.10 Reporting of Digitisation Logs

For each evaluated case, the evaluation script prints a summary line containing:

- **`best r(z)`**: mean best correlation across leads after shift search.
- **`r(z) after shift`**: mean correlation after applying chosen shifts.
- **`MAE=a -> b mV`**: mean MAE before and after alignment shift.
- **`hit`** and **`pxerr`**: mean pixel-consistency measures across leads.
- **`px`**: px-per-mV used (fixed or tuned).
- **`t`**: start time (seconds) used to extract the aECG comparison window.

These logs provide an interpretable "end-to-end" verification that the digitised signal is both mask-supported (hit/pxerr) and physiologically consistent with the reference (r(z), MAE/RMSE).

Chapter 7

HeartGuard Classification Model: ProtoECGNet-Based Architecture

Contents

7.1 Background and Motivation 62

7.2 ProtoECGNet Overview 63

7.3 Model Architecture (Theory) 64

7.3.1 1D Rhythm Branch 64

7.3.2 2D Morphology Branch 64

7.3.3 2D Global Branch 65

7.3.4 Prototype Layer and Similarity Computation 65

7.4 Loss Function and Training Objective 66

7.4.1 Binary Cross-Entropy Loss 67

7.4.2 Clustering Loss 67

7.4.3 Separation Loss 67

7.4.4 Diversity (Orthogonality) Loss 67

7.4.5 Contrastive Loss Based on Label Co-Occurrence 68

7.5 Training Procedure 68

7.5.1 Stage 1: Joint Training 68

7.5.2 Stage 2: Prototype Projection 69

7.5.3 Stage 3: Fusion Classifier Training 69

7.6 Implementation Details: Mapping the Paper to the Codebase 69

7.6.1 Backbone Networks 70

7.6.2 Data Loading and Preprocessing 70

7.6.3 Label Grouping and Clinical Decomposition 70

7.6.4 Prototype-Based Model Definitions 70
7.6.5 Prototype Projection 70
7.6.6 Fusion Mechanism 71
7.6.7 Training Logic and Optimization 71
7.6.8 Execution and Experiment Management 71
7.6.9 Experimental Structure and Model Export 71
7.7 Results and Discussion 72
7.7.1 Evaluation Metrics 72
7.7.2 Quantitative Results 72
7.7.3 Discussion 72
7.8 Interpretability and Case-Based Explanations 73
7.8.1 Prototype-Based Explanation Mechanism 73
7.8.2 Clinical Case-Based Examples 74
7.8.3 Clinical Relevance and Trustworthiness 74

This chapter presents the core classification model used in the HeartGuard graduation project. The proposed system is built upon ProtoECGNet, an interpretable prototype-based deep learning architecture designed for multi-label electrocardiogram (ECG) classification. The objective of this chapter is to provide a structured and comprehensive description of the model, starting from its conceptual motivation and progressing through its architectural design, training strategy, and implementation details.

HeartGuard adopts ProtoECGNet as its primary classification backbone due to its ability to balance high diagnostic performance with transparent and clinically meaningful explanations. Unlike conventional deep learning models that operate as black-box systems, ProtoECGNet enables case-based reasoning by grounding its predictions in learned prototype representations derived from real ECG samples. This design choice aligns closely with clinical diagnostic workflows, where physicians rely on pattern comparison and prior cases to support decision-making.

The model architecture implemented in HeartGuard follows a multi-branch design that reflects different aspects of ECG interpretation. Specifically, the system integrates a one-dimensional (1D) branch for rhythm analysis, a two-dimensional (2D) branch with localized prototypes for morphology-based reasoning, and a 2D branch with global prototypes for detecting diffuse or global cardiac abnormalities. Each branch is trained on a dedicated subset of diagnostic labels, and their outputs are later fused to generate final multi-label predictions.

This chapter is organized as follows. First, the motivation behind using interpretable prototype-based models for ECG analysis is discussed, highlighting the limitations of

black-box approaches. Next, the overall ProtoECGNet architecture is introduced, followed by detailed descriptions of each model branch. The training objectives, loss functions, and prototype projection mechanism are then explained. Finally, the chapter concludes with a discussion of the fusion strategy and how the complete model is integrated into the HeartGuard system.

7.1 Background and Motivation

Electrocardiography (ECG) is one of the most widely used non-invasive diagnostic tools for detecting cardiovascular diseases. A standard 12-lead ECG records the electrical activity of the heart over time and enables clinicians to identify abnormalities related to cardiac rhythm, waveform morphology, and global electrical patterns. Due to the increasing availability of large-scale ECG datasets, deep learning methods have recently achieved remarkable performance in automatic ECG classification tasks.

Despite their high predictive accuracy, most state-of-the-art deep learning models for ECG analysis operate as black-box systems. These models provide limited insight into the reasoning behind their predictions, which poses a major challenge for clinical adoption. In medical applications, interpretability and transparency are essential, as clinicians must be able to understand, verify, and trust the diagnostic decisions suggested by automated systems.

To address this limitation, prototype-based neural networks have been proposed as an interpretable alternative to conventional deep learning models. Instead of relying solely on abstract feature representations, prototype-based models explain their predictions by measuring similarity between an input sample and a set of learned prototypes that correspond to real examples from the training data. This case-based reasoning paradigm closely aligns with clinical practice, where physicians often diagnose patients by comparing their ECG patterns with previously observed cases.

ProtoECGNet is a prototype-based deep learning framework specifically designed for interpretable multi-label ECG classification. The model explicitly incorporates domain knowledge from cardiology by decomposing the ECG interpretation process into multiple complementary perspectives, including rhythm analysis, morphology-based reasoning, and the detection of global or diffuse abnormalities. By combining these views within a unified architecture, ProtoECGNet aims to achieve both high diagnostic performance and faithful interpretability.

7.2 ProtoECGNet Overview

ProtoECGNet is a prototype-based deep learning framework designed for interpretable multi-label electrocardiogram (ECG) classification. The model follows a case-based reasoning paradigm, where predictions are generated by measuring the similarity between an input ECG and a set of learned prototype representations. Each prototype corresponds to a representative ECG pattern extracted from real training samples, enabling transparent and clinically meaningful explanations for the model's decisions.

Unlike conventional end-to-end neural networks, ProtoECGNet explicitly decomposes the ECG interpretation task into multiple complementary perspectives that mirror clinical reasoning. In cardiology practice, ECG analysis is typically performed by separately examining cardiac rhythm, waveform morphology, and global or diffuse electrical abnormalities. ProtoECGNet incorporates this domain knowledge by adopting a structured multi-branch architecture, where each branch focuses on a specific diagnostic aspect of the ECG signal.

The architecture consists of three main branches. The first branch is a one-dimensional (1D) convolutional neural network (CNN) equipped with global temporal prototypes. This branch operates directly on the raw ECG time-series and is primarily responsible for rhythm-related classification tasks, such as identifying arrhythmias and conduction abnormalities. Global prototypes are used in this branch to capture long-range temporal patterns that characterize different cardiac rhythms.

The second branch is a two-dimensional (2D) CNN with time-localized, or partial, prototypes. In this branch, the ECG signal is represented as a two-dimensional time–lead matrix, allowing the model to learn spatial relationships across ECG leads. The localized prototypes focus on short temporal segments of the feature maps, enabling the model to detect morphology-based abnormalities such as ST-segment deviations, pathological Q-waves, or T-wave changes. This design allows the model to associate specific waveform segments with diagnostic decisions.

The third branch is a 2D CNN with global 2D prototypes, which is designed to capture diffuse or global ECG abnormalities. These abnormalities often manifest as distributed patterns across multiple leads and time intervals, such as those caused by electrolyte imbalances or generalized conduction disturbances. By using global prototypes in the 2D feature space, this branch learns holistic representations of such conditions.

Each branch is trained on a dedicated subset of diagnostic labels corresponding to its clinical role. The outputs of the three branches are later combined through a fusion mechanism to produce the final multi-label classification results. This modular design allows ProtoECGNet to jointly achieve high classification performance, interpretability, and alignment with established clinical ECG analysis principles, as described in the original ProtoECGNet study.

7.3 Model Architecture (Theory)

The ProtoECGNet architecture adopted in the HeartGuard system is designed as a structured, multi-branch model that explicitly reflects the clinical process of ECG interpretation. Instead of relying on a single monolithic network, the model decomposes the classification task into three complementary branches, each corresponding to a distinct clinical reasoning perspective. These branches operate in parallel and are later combined through a fusion mechanism to produce final multi-label predictions.

7.3.1 1D Rhythm Branch

The first branch focuses on rhythm analysis and operates directly on the one-dimensional ECG time-series. This branch employs a ResNet1D-18 backbone that is specifically adapted for sequential signal processing. The 1D convolutional layers enable the extraction of temporal features that capture long-range dependencies and periodic patterns characteristic of cardiac rhythm disorders, such as atrial flutter and atrial fibrillation.

To enable interpretable decision-making, the rhythm branch incorporates a set of global temporal prototypes. These prototypes represent prototypical rhythm patterns learned from the training data. During inference, the latent feature representation of an input ECG is compared against all rhythm prototypes, and the resulting similarity scores are used to guide the classification of rhythm-related abnormalities.

7.3.2 2D Morphology Branch

The second branch is dedicated to morphology-based reasoning and operates on a two-dimensional representation of the ECG signal. In this representation, the ECG is arranged as a time–lead matrix with dimensions $(1 \times 12 \times 1000)$, where the vertical dimension corresponds to the 12 ECG leads and the horizontal dimension corresponds to time.

This branch uses a modified ResNet18 backbone adapted for two-dimensional ECG inputs. The network transforms the input into a high-level latent feature map of shape $(512 \times 1 \times 32)$, where the temporal axis is downsampled to preserve waveform resolution. Unlike the rhythm branch, the morphology branch employs partial, time-localized prototypes that span a limited temporal window of three steps. These localized prototypes enable the model to focus on specific waveform segments associated with morphological abnormalities, such as ST-segment deviations, pathological Q-waves, and T-wave changes.

To aggregate prototype activations across time, a top-k pooling operation is applied. This mechanism emphasizes the most diagnostically relevant temporal regions, ensuring that brief but clinically significant morphological patterns can be detected reliably.

7.3.3 2D Global Branch

The third branch targets global or diffuse ECG abnormalities that are not confined to short temporal segments but instead manifest across multiple leads and extended time intervals. This branch shares the same two-dimensional ResNet18 backbone architecture as the morphology branch and operates on the same $(1 \times 12 \times 1000)$ ECG representation.

In contrast to the morphology branch, the global branch employs global 2D prototypes that span the entire temporal dimension of the latent feature map. These prototypes are designed to capture holistic ECG patterns associated with conditions such as electrolyte disturbances or generalized conduction abnormalities. By comparing the extracted feature maps to global prototypes, the model learns to identify distributed abnormalities that cannot be localized to a specific waveform segment.

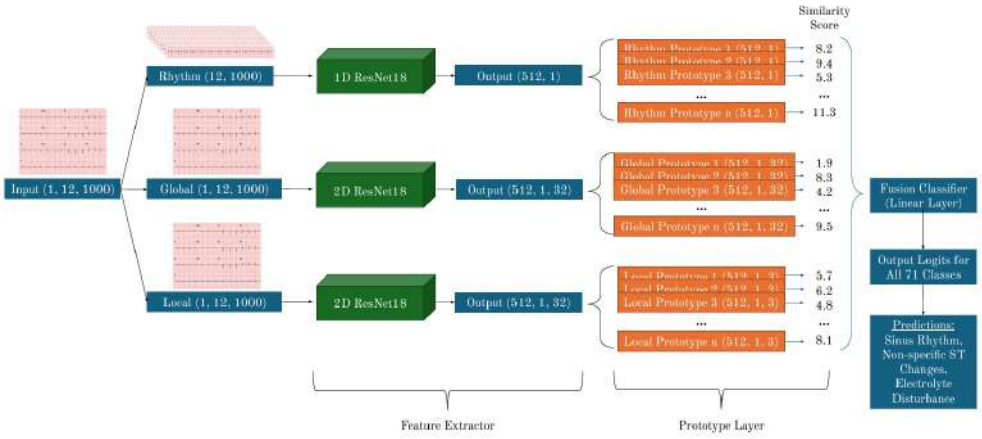


Figure 7.1: Internal architecture of ProtoECGNet. Each ECG is processed in parallel by three branches: a 1D CNN with global temporal prototypes for rhythm interpretation, a 2D CNN with localized prototypes for morphology-based reasoning, and a 2D CNN with global prototypes for diffuse signal abnormalities. Similarity scores from all branches are fused using a linear classifier to produce final predictions.

7.3.4 Prototype Layer and Similarity Computation

At the core of all three branches lies the prototype layer, which enables interpretable, case-based reasoning. Let $\mathbf{z}$ denote the latent feature representation extracted by a branch-specific backbone, and let $\mathbf{p}$ represent a learned prototype vector. The similarity between $\mathbf{z}$ and $\mathbf{p}$ is computed using cosine similarity, defined as

$$S(\mathbf{z}, \mathbf{p}) = \frac{\mathbf{z} \cdot \mathbf{p}}{\|\mathbf{z}\|_2 \, \|\mathbf{p}\|_2}. \tag{7.1}$$

This similarity score quantifies how closely the input ECG resembles each prototype. For the morphology branch, similarity is computed over all possible time-localized win-

dows of the latent feature map, and the top-k most activated positions are aggregated into a final similarity score per prototype. For the rhythm and global branches, similarity is computed globally over the latent representation.

The resulting similarity scores from all prototypes serve as the input to a linear classification layer, which maps them to class logits. This design ensures that each prediction is explicitly grounded in similarity to representative ECG patterns from the training data.

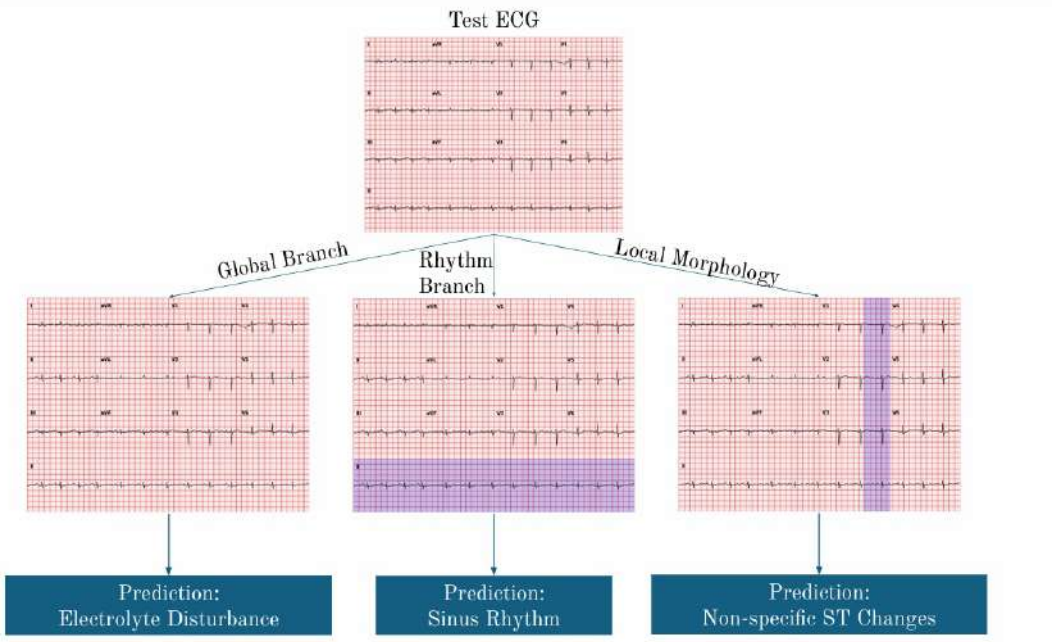


Figure 7.2: Prototype-based multi-branch explanation in ProtoECGNet. Highlighted regions indicate ECG segments that are most similar to learned prototypes in each branch. These regions directly contribute to the final prediction, enabling transparent and case-based explanations of model decisions.

By grounding classification decisions in similarity to learned prototypes, ProtoECGNet provides explanations that are faithful to the model's internal reasoning process. This architecture enables HeartGuard to achieve high diagnostic performance while maintaining transparency and clinical interpretability.

7.4 Loss Function and Training Objective

Training ProtoECGNet involves optimizing a composite objective function that balances classification performance with interpretability-driven constraints on the learned prototypes. Instead of relying solely on a standard classification loss, the model incorporates multiple regularization terms that encourage meaningful, diverse, and well-separated prototype representations. This design ensures that prototypes correspond to representative ECG patterns and can be reliably used for case-based explanations.

The total training loss is defined as

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{BCE}} + \lambda_{\text{clst}}\mathcal{L}_{\text{clst}} + \lambda_{\text{sep}}\mathcal{L}_{\text{sep}} + \lambda_{\text{div}}\mathcal{L}_{\text{div}} + \lambda_{\text{cntrst}}\mathcal{L}_{\text{cntrst}}, \tag{7.2}$$

where each term serves a specific role in shaping the behavior of the model. The scalar coefficients λ_{clst}, λ_{sep}, λ_{div}, and λ_{cntrst} control the relative importance of the corresponding regularization losses.

7.4.1 Binary Cross-Entropy Loss

The primary classification objective is defined using the binary cross-entropy (BCE) loss, which is well-suited for multi-label classification tasks. Given the ground-truth label vector $\mathbf{y} \in \{0, 1\}^C$ and the predicted logits $\hat{\mathbf{y}} \in \mathbb{R}^C$ for C diagnostic classes, the BCE loss is computed independently for each class. This loss term encourages accurate prediction of all diagnostic labels and forms the backbone of the optimization process.

7.4.2 Clustering Loss

The clustering loss $\mathcal{L}_{\text{clst}}$ enforces that each prototype remains close to at least one latent representation extracted from training samples belonging to its associated class. Specifically, this term minimizes the distance between a prototype and the nearest feature embedding of a correctly labeled ECG. By doing so, the clustering loss ensures that prototypes correspond to realistic and representative ECG patterns observed in the dataset, thereby improving interpretability.

7.4.3 Separation Loss

The separation loss $\mathcal{L}_{\text{sep}}$ complements the clustering objective by encouraging prototypes to remain distant from feature embeddings of ECGs belonging to incorrect or unrelated classes. This loss penalizes small distances between a prototype and samples that do not share its assigned label. As a result, prototypes become more class-specific and less ambiguous, which improves both classification robustness and explanatory clarity.

7.4.4 Diversity (Orthogonality) Loss

To prevent redundancy among prototypes within the same branch, a diversity loss $\mathcal{L}_{\text{div}}$ is introduced. This term encourages orthogonality between different prototypes by penalizing high cosine similarity among prototype vectors. By promoting diversity, the model learns a set of prototypes that capture distinct and complementary ECG patterns rather than duplicating similar representations. This mechanism enhances the coverage of the prototype space and strengthens interpretability.

7.4.5 Contrastive Loss Based on Label Co-Occurrence

The contrastive loss $\mathcal{L}_{\text{cntrst}}$ incorporates label co-occurrence information to model relationships between diagnostic classes. Since ECG conditions frequently co-occur, treating all labels as independent can lead to suboptimal representations. To address this, a label co-occurrence matrix is constructed using the Jaccard similarity coefficient, which measures how often pairs of labels appear together across the training dataset.

This co-occurrence matrix is used to encourage prototypes associated with frequently co-occurring labels to be closer in the embedding space, while prototypes of rarely co-occurring or mutually exclusive labels are pushed farther apart. By integrating this contrastive objective, ProtoECGNet captures clinically meaningful dependencies between diagnostic categories, resulting in more coherent prototype representations and improved multi-label classification performance.

Overall, the combination of these loss terms enables ProtoECGNet to jointly optimize classification accuracy, prototype interpretability, and representational structure, in accordance with the training strategy proposed in the original ProtoECGNet study.

7.5 Training Procedure

The training strategy of ProtoECGNet is designed as a multi-stage process that progressively refines both classification performance and prototype interpretability. Rather than training all components in a single end-to-end procedure, the model is optimized through three sequential stages. This staged training approach ensures stable convergence, meaningful prototype learning, and effective integration of the different model branches.

7.5.1 Stage 1: Joint Training

In the first stage, the backbone network, prototype layer, and classification head of each branch are trained jointly. During this phase, the full objective function defined in the previous section is optimized, including the binary cross-entropy loss and all prototype-related regularization terms. This allows the model to simultaneously learn discriminative feature representations and initialize prototypes that are aligned with diagnostically relevant ECG patterns.

Joint training is performed independently for each branch, using only the subset of diagnostic labels assigned to that branch. This specialization enables each branch to focus on its intended clinical role, such as rhythm analysis or morphology-based reasoning, without interference from unrelated labels.

7.5.2 Stage 2: Prototype Projection

After joint training converges, a prototype projection step, commonly referred to as the *push* operation, is performed. In this stage, each learned prototype is replaced by the latent feature representation of the closest matching ECG segment from the training set that belongs to the prototype's associated class.

This projection step grounds each prototype in an actual ECG example, ensuring that prototypes correspond to real and interpretable signal patterns. By anchoring prototypes to true training samples, the model's explanations become faithful to the underlying data and easier to validate by clinical experts.

7.5.3 Stage 3: Fusion Classifier Training

In the final stage, the outputs of the individual branches are integrated through a fusion classifier. For each ECG input, the similarity scores between the input and the prototypes of all branches are concatenated to form a unified feature vector. A linear classification layer is then trained on top of these similarity features to produce the final multi-label predictions.

During this stage, the backbone networks and prototype representations are typically frozen, and only the fusion classifier parameters are updated. This strategy allows the model to leverage the complementary information captured by the rhythm, morphology, and global branches while maintaining the interpretability of the learned prototypes.

Overall, this three-stage training procedure enables ProtoECGNet to achieve high diagnostic accuracy while preserving the transparency and case-based reasoning properties that are central to its design, as proposed in the original ProtoECGNet framework.

7.6 Implementation Details: Mapping the Paper to the Codebase

The ProtoECGNet-based architecture described in the original study is implemented in HeartGuard through a modular and well-structured codebase. Each theoretical component of the model is mapped to a dedicated script or module, ensuring clarity, extensibility, and faithful reproduction of the original design. This section outlines how the key concepts introduced in the ProtoECGNet paper are realized in the implemented system.

7.6.1 Backbone Networks

The file `backbones.py` contains the definitions of the backbone neural networks used across all model branches. This includes the ResNet1D architecture employed by the rhythm branch for one-dimensional temporal feature extraction, as well as the modified ResNet18 architecture used by the two-dimensional branches. These backbones serve as feature extractors and provide latent representations that are later processed by the prototype layers.

7.6.2 Data Loading and Preprocessing

All dataset handling and preprocessing operations are centralized in the `ecg_utils.py` module. This script is responsible for loading the PTB-XL dataset, applying the high-pass Butterworth filter with a cutoff frequency of 0.5 Hz, and preparing the data loaders for training, validation, and testing. The preprocessing pipeline follows the protocol described in the original ProtoECGNet study to ensure consistency and reproducibility.

7.6.3 Label Grouping and Clinical Decomposition

The script `label_co.py` implements the decomposition of the 71 PTB-XL diagnostic labels into three clinically motivated subsets: rhythm-related labels (Label Set 1), morphology-related labels (Label Set 3), and global or diffuse abnormality labels (Label Set 4). This grouping follows the same rhythm–morphology–global categorization proposed in the ProtoECGNet paper and enables branch-specific specialization during training.

7.6.4 Prototype-Based Model Definitions

The prototype-based models for each branch are defined in dedicated modules. The file `proto_models1D.py` implements the 1D rhythm prototype model, including global temporal prototypes and their associated similarity computations. The file `proto_models2D.py` contains the implementations of both the 2D morphology model with time-localized (partial) prototypes and the 2D global model with full-extent prototypes. These modules directly reflect the architectural distinctions described in the theoretical model.

7.6.5 Prototype Projection

The prototype projection step is implemented in the `push.py` script. This module performs the *push* operation after joint training by replacing each learned prototype with the latent representation of the closest matching ECG segment from the training set. This

step anchors prototypes to real ECG examples and is essential for achieving faithful and interpretable explanations.

7.6.6 Fusion Mechanism

The fusion strategy is implemented in the `fusion.py` module. This script defines the fusion classifier, which takes as input the similarity scores produced by the rhythm, morphology, and global branches. These scores are concatenated and passed through a linear classification layer to generate the final logits for all 71 diagnostic labels.

7.6.7 Training Logic and Optimization

The file `training_functions.py` contains the core training loops and optimization logic. This module implements the three-stage training procedure and computes the full composite loss, including the binary cross-entropy loss and all prototype-related regularization terms. By centralizing the training logic, the implementation ensures consistent behavior across all branches.

7.6.8 Execution and Experiment Management

The main entry point of the system is provided in `main.py`. This script handles argument parsing, branch selection (1D rhythm, 2D morphology, 2D global, or fusion), and execution mode selection for training or inference. It orchestrates the complete workflow and enables flexible experimentation.

7.6.9 Experimental Structure and Model Export

The codebase is organized into dedicated directories corresponding to each training stage and model branch. Separate folders are used for training the 1D rhythm branch (Label Set 1), the 2D morphology branch (Label Set 3), and the 2D global branch (Label Set 4). Additional scripts manage fusion training and model export, including the generation of final checkpoints such as `Fusion_Final.ckpt`. These exported models are integrated into the HeartGuard system for deployment and evaluation.

Overall, this modular implementation closely follows the architectural and methodological principles of the ProtoECGNet paper, while providing a flexible and extensible framework suitable for real-world ECG analysis within the HeartGuard project.

7.7 Results and Discussion

This section presents the experimental results obtained from the HeartGuard implementation of ProtoECGNet and discusses them in comparison with the results reported in the original ProtoECGNet study. The evaluation focuses on classification performance across the different diagnostic label sets, highlighting the strengths and limitations of the proposed system.

7.7.1 Evaluation Metrics

Model performance is evaluated using the Area Under the Receiver Operating Characteristic Curve (AUROC) and the F1-score, which are standard metrics for multi-label classification tasks. AUROC measures the model's ability to discriminate between positive and negative samples across different decision thresholds, while the F1-score captures the balance between precision and recall. Macro-averaged scores are reported to ensure equal weighting of all diagnostic classes.

7.7.2 Quantitative Results

Table 7.1 summarizes the classification performance achieved by the HeartGuard model for each diagnostic branch and for the final fused model. Results are reported separately for the rhythm, morphology, and global label sets, followed by the overall performance of the fusion classifier.

Table 7.1: Performance comparison between the HeartGuard implementation and the original ProtoECGNet results.

Model / Label Set	**AUROC (Val)**	**F1-score (Val)**	**Paper AUROC**	**Paper F1**
1D Rhythm Branch (Set 1)	0.8205	0.4120	0.83	–
2D Morphology Branch (Set 3)	0.8665	0.50	0.87	–
2D Global Branch (Set 4)	0.9389	0.55	0.94	–
Fusion Model (All Sets)	**0.9232**	**0.55**	0.9132	–

7.7.3 Discussion

The results demonstrate that the HeartGuard implementation of ProtoECGNet achieves competitive performance across all diagnostic label sets. The rhythm branch shows strong discriminative capability for temporal abnormalities, reflecting the effectiveness of the 1D ResNet-based architecture and global temporal prototypes. Similarly, the morphology branch benefits from localized prototypes and top-k pooling, enabling reliable detection of waveform-specific abnormalities.

The global branch captures diffuse ECG patterns and contributes complementary information that enhances the overall performance of the fused model. When combining the outputs of all branches, the fusion classifier achieves a macro-AUROC that is comparable to the value reported in the original ProtoECGNet paper (0.9132 for the full model). Minor performance differences can be attributed to implementation choices, training hyperparameters, and computational constraints specific to the HeartGuard project.

Importantly, the achieved performance is obtained while preserving full model interpretability through prototype-based explanations. This highlights the practical viability of ProtoECGNet as a clinically relevant ECG classification model and validates its integration into the HeartGuard system. The results confirm that interpretable, case-based deep learning models can achieve performance comparable to state-of-the-art black-box approaches without sacrificing transparency.

7.8 Interpretability and Case-Based Explanations

A key strength of the HeartGuard system lies in its ability to provide transparent and clinically meaningful explanations for its predictions. Unlike conventional black-box deep learning models, HeartGuard is built upon the ProtoECGNet architecture, which enables case-based reasoning through learned prototype representations. Each prediction is explicitly grounded in similarity to representative ECG patterns extracted from real training samples.

7.8.1 Prototype-Based Explanation Mechanism

In HeartGuard, interpretability is achieved by associating each diagnostic decision with one or more prototypes. During inference, the model computes similarity scores between the latent representation of the input ECG and the learned prototypes in each branch. The prototypes that exhibit the highest similarity scores are considered the most influential in the final prediction.

For the rhythm branch, global temporal prototypes highlight ECG segments that resemble characteristic rhythm patterns, such as atrial flutter or atrial fibrillation. For the morphology branch, time-localized prototypes enable the model to identify specific waveform segments that correspond to morphological abnormalities. These segments are marked as the most relevant regions contributing to the prediction. In the global branch, full-extent prototypes capture diffuse patterns across multiple leads and time intervals.

HeartGuard presents these explanations by indicating which portions of the ECG signal are most similar to the selected prototypes. This allows clinicians to visually in-

spect the highlighted regions and verify whether they correspond to medically meaningful patterns, thereby increasing trust in the model's decisions.

7.8.2 Clinical Case-Based Examples

The interpretability mechanism implemented in HeartGuard closely mirrors the examples presented in the original ProtoECGNet study. In the case of atrial flutter (AFLT), the model highlights periodic atrial activity patterns that resemble characteristic flutter waves, providing an intuitive explanation for rhythm-based predictions. Similarly, for acute myocardial infarction (ASMI), localized morphology prototypes focus on ST-segment elevation patterns in specific leads, enabling clear association between the model's decision and clinically relevant waveform changes.

For global conditions such as electrolyte disturbances, the model relies on global 2D prototypes that capture distributed abnormalities across leads. These explanations align with the figures presented in the ProtoECGNet paper, where diffuse waveform alterations are shown to contribute to the classification outcome. By grounding predictions in these prototypical examples, HeartGuard ensures that explanations are not post-hoc visualizations but are directly derived from the internal decision-making process of the model.

7.8.3 Clinical Relevance and Trustworthiness

By providing prototype-based explanations, HeartGuard enables clinicians to assess not only what decision the model makes, but also why it makes that decision. This level of transparency is particularly important in medical applications, where diagnostic support systems must be interpretable and verifiable. The case-based explanations produced by HeartGuard support clinical reasoning, facilitate error analysis, and enhance the overall reliability of the system.

Overall, the interpretability framework adopted in HeartGuard demonstrates that high diagnostic performance can be achieved without sacrificing transparency. By leveraging prototype-based reasoning, the system offers explanations that are faithful to the model's internal logic and consistent with established clinical ECG interpretation principles, as demonstrated in the original ProtoECGNet study.

Chapter 8

Application and Backend Implementation (Flutter + Django)

Contents

8.1 Overview and System Architecture 76

8.2 Frontend Technology: Flutter 76

8.2.1 What is Flutter 76

8.2.2 Why Flutter was Selected 76

8.2.3 Flutter Project Structure and API Communication 76

8.3 Flutter Application Implementation 77

8.3.1 UI Structure, Navigation, and Theme Support 77

8.3.2 Patient Management Module 80

8.3.3 Patient Dashboard: Notes, ECG Intake, and Case History . . . 82

8.3.4 ECG Upload Workflow (Signal vs. Image) 82

8.3.5 Result Visualization and Interpretability 84

8.3.6 12-Lead ECG Reference Gallery 87

8.3.7 Cross-Platform Deployment and Manual ECG Cropping 89

8.4 Backend Technology: Django (Django REST Framework) . 91

8.4.1 What is Django 91

8.4.2 Why Django and DRF were Selected 91

8.4.3 Database Design (PostgreSQL) 91

8.4.4 File Storage and Media Organization 91

8.4.5 REST API Endpoints and Feature Integration 92

8.4.6 End-to-End Processing Pipeline 92

8.4.7 Robustness, Validation, and Error Handling 93
8.5 Summary 94

8.1 Overview and System Architecture

HeartGuard follows a client–server architecture that separates presentation from computation. The **Flutter frontend** provides a cross-platform interface for patient management, ECG upload, and result visualization. The **Django backend** exposes RESTful APIs, manages persistent storage, orchestrates preprocessing, runs AI inference pipelines, and returns structured diagnostic outputs.

This separation keeps the user interface responsive while centralizing computationally intensive tasks on the server, which improves scalability, maintainability, and reliability.

8.2 Frontend Technology: Flutter

8.2.1 What is Flutter

Flutter is an open-source UI toolkit developed by Google for building natively compiled applications from a single codebase. Flutter uses the Dart programming language and provides a rich widget framework for designing consistent, modern user interfaces across platforms.

8.2.2 Why Flutter was Selected

Flutter was selected for HeartGuard for the following reasons:

- **Cross-platform development:** a single codebase can target desktop, web, and mobile deployments.
- **Responsive UI:** efficient rendering and reactive state management are suitable for clinical workflows.
- **Maintainability:** the project can be organized into screens, services, and reusable widgets.
- **Fast iteration:** hot reload accelerates testing and refinement of the UI.

8.2.3 Flutter Project Structure and API Communication

The Flutter codebase is organized in a modular way to separate concerns and simplify maintenance. UI screens focus on presentation and user interaction, while service classes

encapsulate communication with backend REST APIs. This separation reduces coupling between interface logic and networking code, improves readability, and makes the application easier to extend.

Networking operations (such as fetching patient lists, uploading ECG files, and retrieving predictions) are handled through HTTP requests. Backend responses are parsed into structured objects that can be rendered consistently across the application. Errors (e.g., invalid uploads or server failures) are surfaced to the user with clear feedback to avoid silent failures.

In addition, the frontend supports both **mobile** and **web/desktop** file intake workflows. Platform-specific behavior (e.g., file selection and image capture) is handled using conditional logic so that the same clinical flow is preserved across devices while keeping implementation details isolated from the UI layer.

8.3 Flutter Application Implementation

8.3.1 UI Structure, Navigation, and Theme Support

HeartGuard is implemented using Flutter, enabling a **single codebase** to be deployed across **mobile (iOS/Android)**, **desktop/laptop**, and **web** environments. The application adopts a sidebar-driven navigation layout that provides direct access to core workflows such as *Home*, *Patients*, and the *ECGs*. The interface supports both **Light** and **Dark** modes to improve usability under different lighting conditions while preserving a consistent structure and interaction pattern.

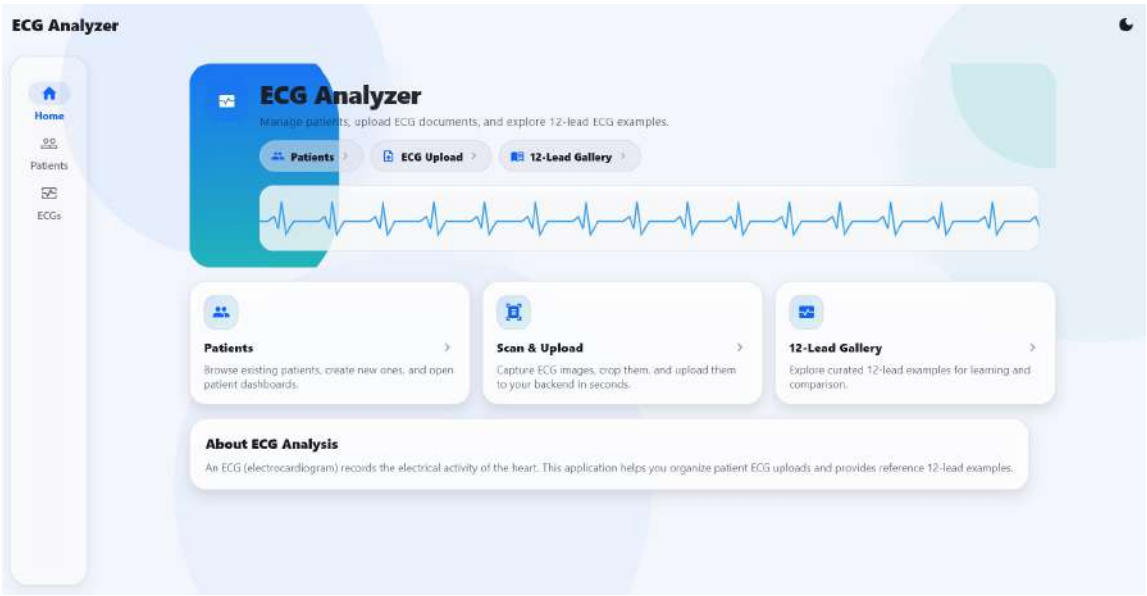


Figure 8.1: Home dashboard (Desktop/Web, Light Mode). The main entry point to the application with shortcuts to Patients, ECG Upload/Analysis, and the 12-Lead ECG Gallery.



Figure 8.2: Home dashboard (Desktop/Web, Dark Mode). The same layout with a dark theme to enhance readability in low-light environments.

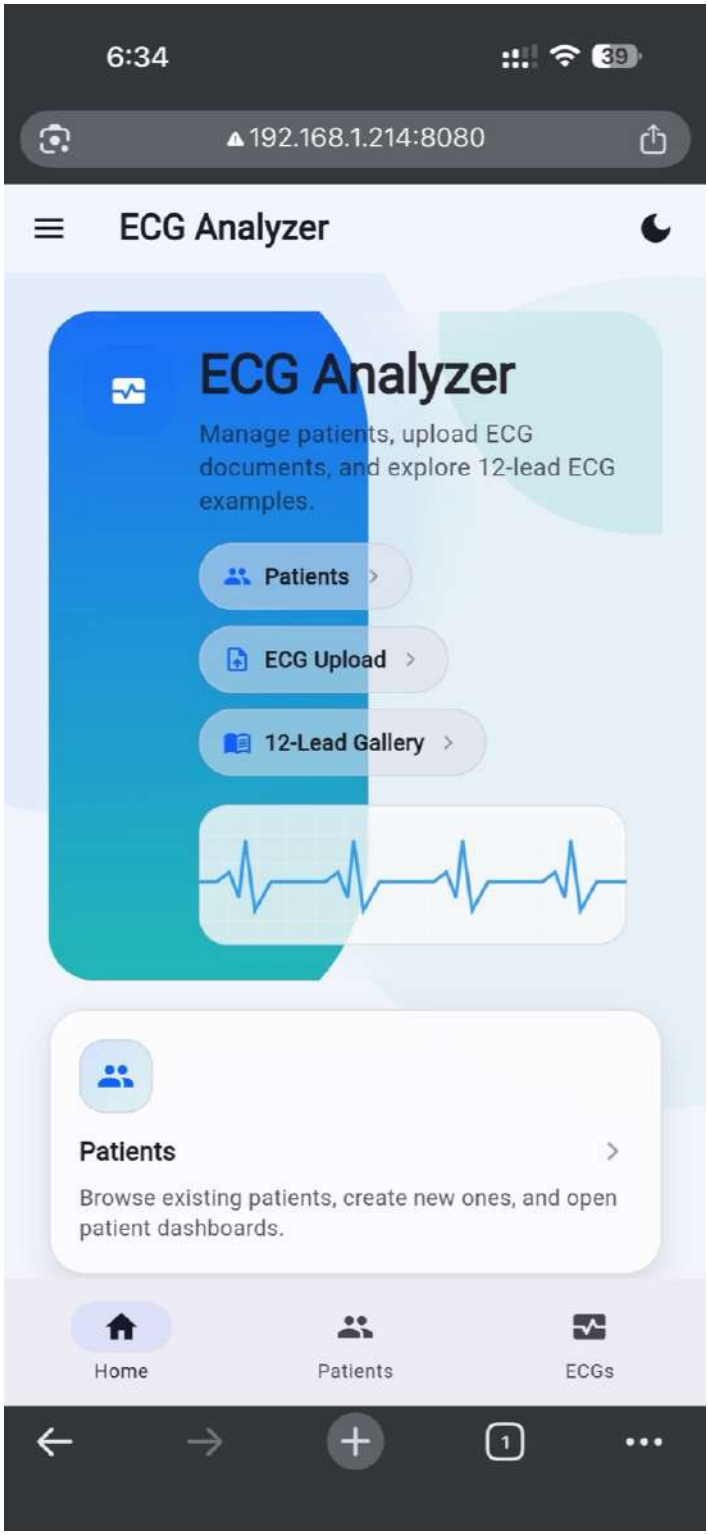


Figure 8.3: Home screen of the ECG Analyzer mobile application (iOS). The main entry point provides direct navigation to core workflows, including Patients, ECG Upload/Analysis, and the 12-lead ECG reference gallery.

8.3.2 Patient Management Module

The Patients module supports creating, listing, and deleting patient records. Each patient is represented using essential demographic fields such as **unique ID**, **name**, **age**, and **gender**. The UI also displays the number of ECG cases linked to each patient, which enables longitudinal monitoring and follow-up. Deletion operations are synchronized with the backend to preserve data consistency and ensure that removed records do not remain referenced by stored ECG cases.

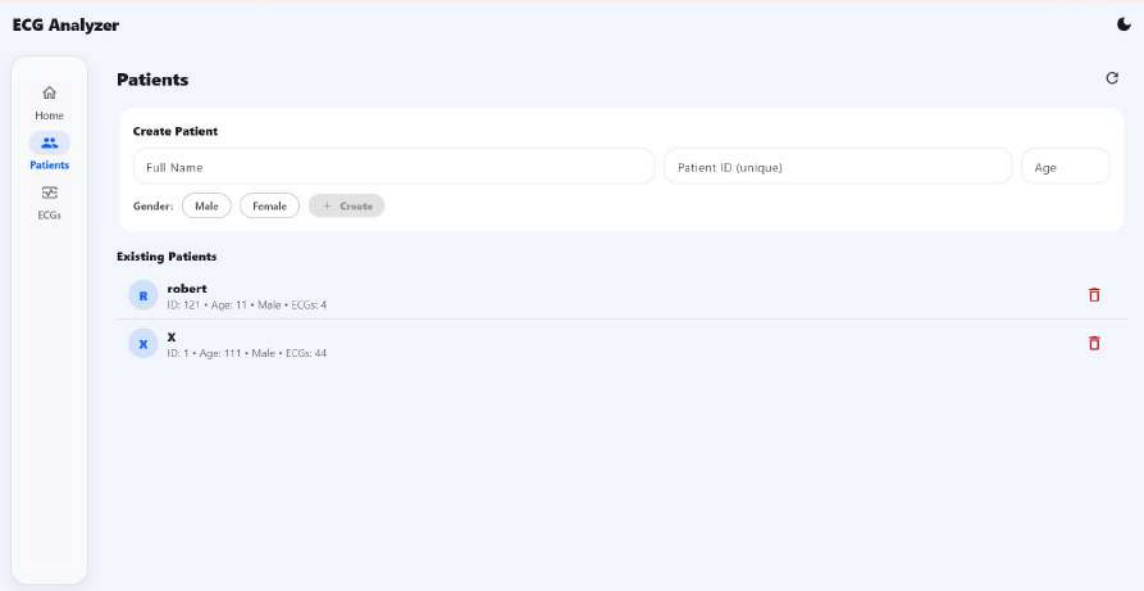


Figure 8.4: Patients screen (Desktop/Web). Users can create patient profiles, browse stored records, view the number of associated ECG cases, and delete patients when required.

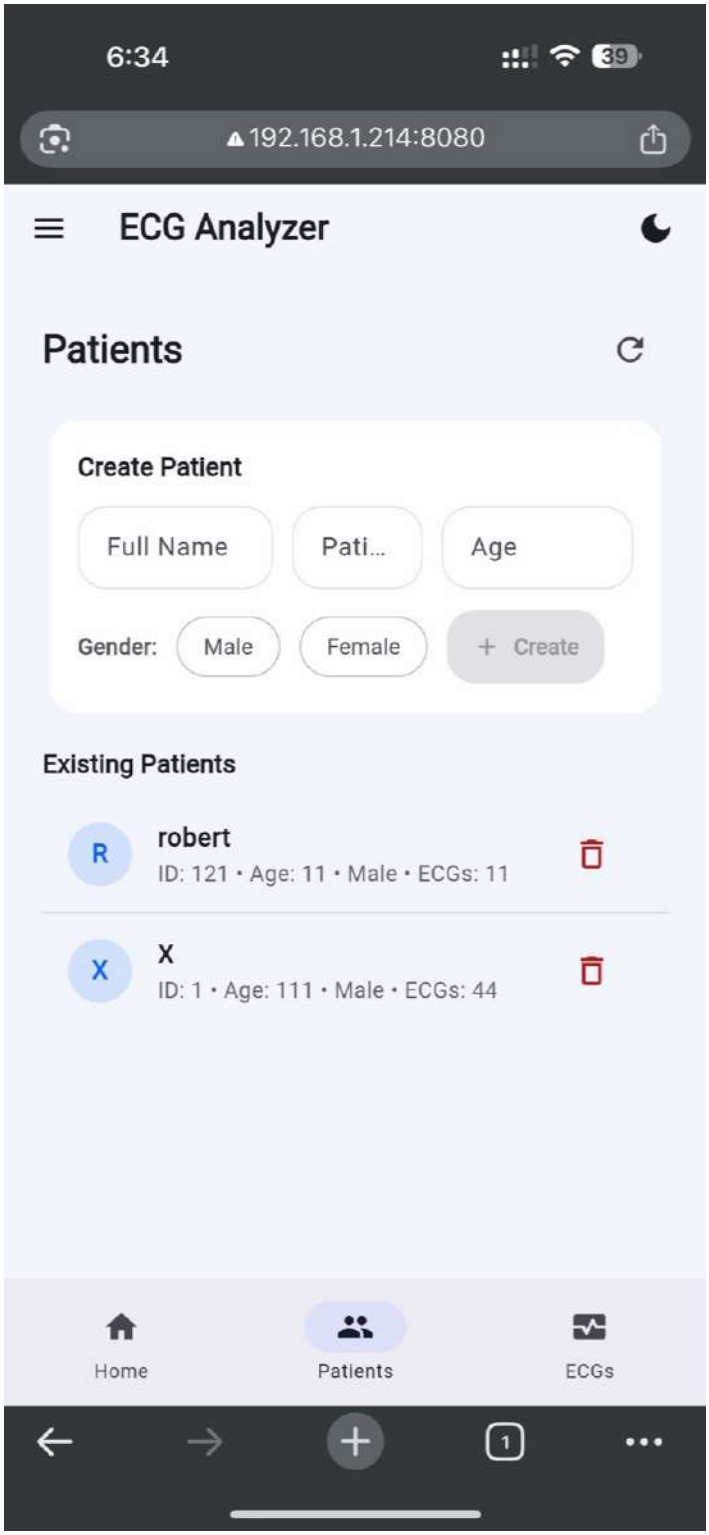


Figure 8.5: Patients screen (iOS). Users can create new patient profiles and browse stored records. Each entry includes demographic information and the number of associated ECG cases for longitudinal monitoring.

8.3.3 Patient Dashboard: Notes, ECG Intake, and Case History

Selecting a patient opens a dedicated dashboard that consolidates all patient-specific interactions into a single view. This design reduces unnecessary navigation and supports efficient clinical workflow.

The dashboard includes:

- **Doctor Notes:** clinicians can create and store follow-up notes linked to the patient record.
- **ECG Intake:** upload or Scan new ECG inputs in either signal or image format.
- **ECG History:** a chronological list of uploaded ECG cases, including timestamps and predicted outcomes.

This dashboard design also improves maintainability: notes, uploads, and history are implemented as independent UI blocks fed by backend APIs, which makes it easy to add new patient-related panels without redesigning the entire screen.

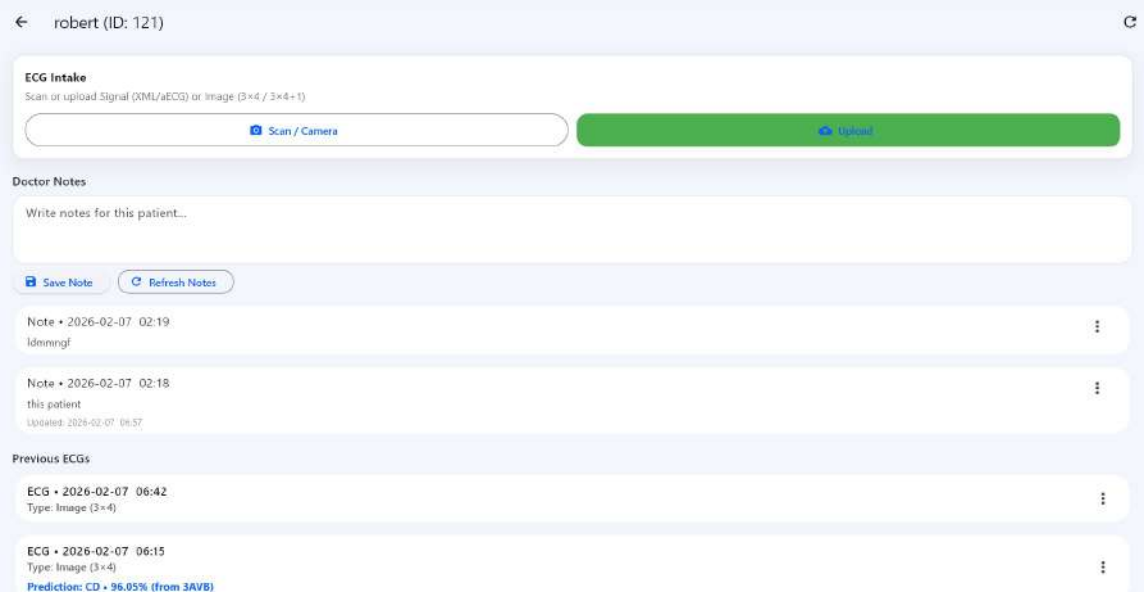


Figure 8.6: Patient dashboard (Desktop/Web). Provides doctor notes, ECG intake controls, and historical ECG cases with predicted results for longitudinal follow-up.

8.3.4 ECG Upload Workflow (Signal vs. Image)

To reduce user errors and ensure correct preprocessing, ECG upload is implemented as a guided workflow. Users first specify whether the input is a **digital signal** or an **ECG image**. This selection allows the backend to route the upload to the correct pipeline: signal parsing for digital formats, or layout-aware preprocessing for images.

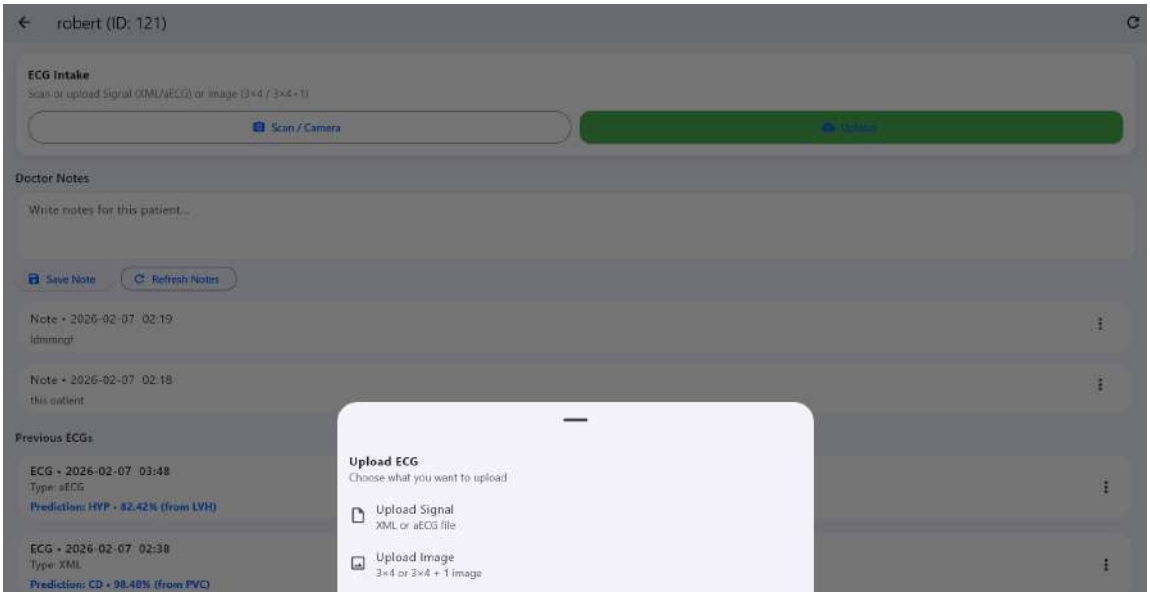


Figure 8.7: Upload options dialog (Desktop/Web). Users select between uploading a digital ECG signal or an ECG image to ensure correct backend routing.

Signal Upload: Philips XML and aECG

HeartGuard supports two common digital ECG formats: **Philips XML ECG** and **aECG**. Uploaded files are parsed on the backend to extract waveform samples and relevant metadata. The extracted signals are then normalized into a consistent internal representation suitable for downstream inference. Supporting raw signal formats improves robustness and avoids limitations of image digitization when original waveform data is available.

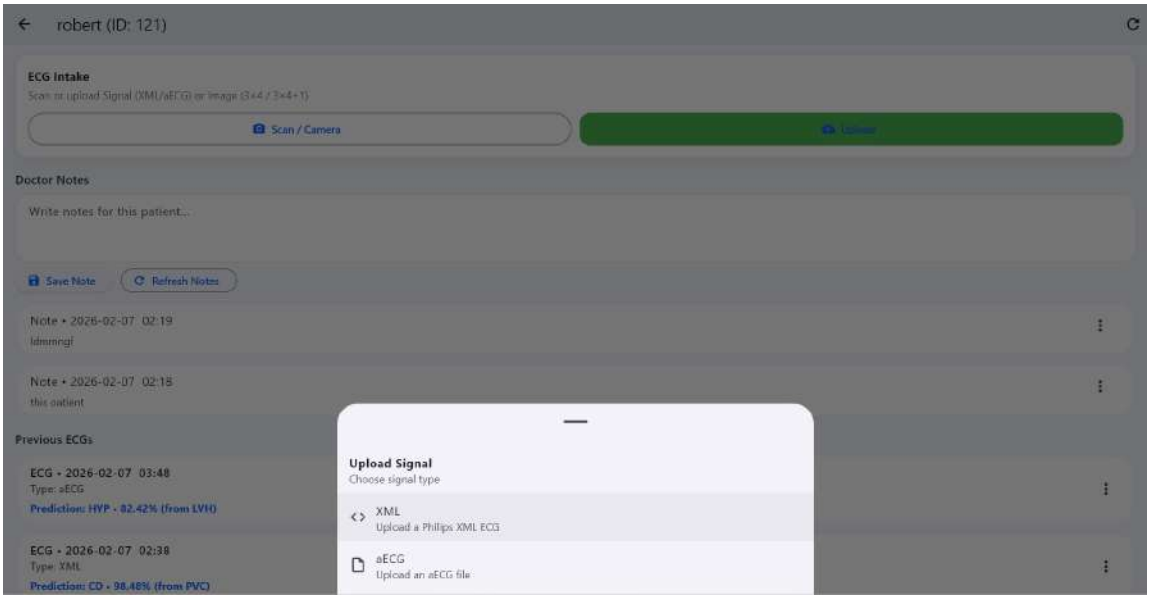


Figure 8.8: Signal type selection (Desktop/Web). Users choose Philips XML ECG or aECG, and the backend applies format-specific parsing before inference.

Image Upload: 3×4 and 3×4+1 Layouts

For image-based ECGs, the system supports standard clinical layouts such as 3×4 and 3×4+1 (including an additional rhythm strip). The chosen layout is transmitted to the backend as metadata, enabling layout-aware cropping and normalization prior to inference.

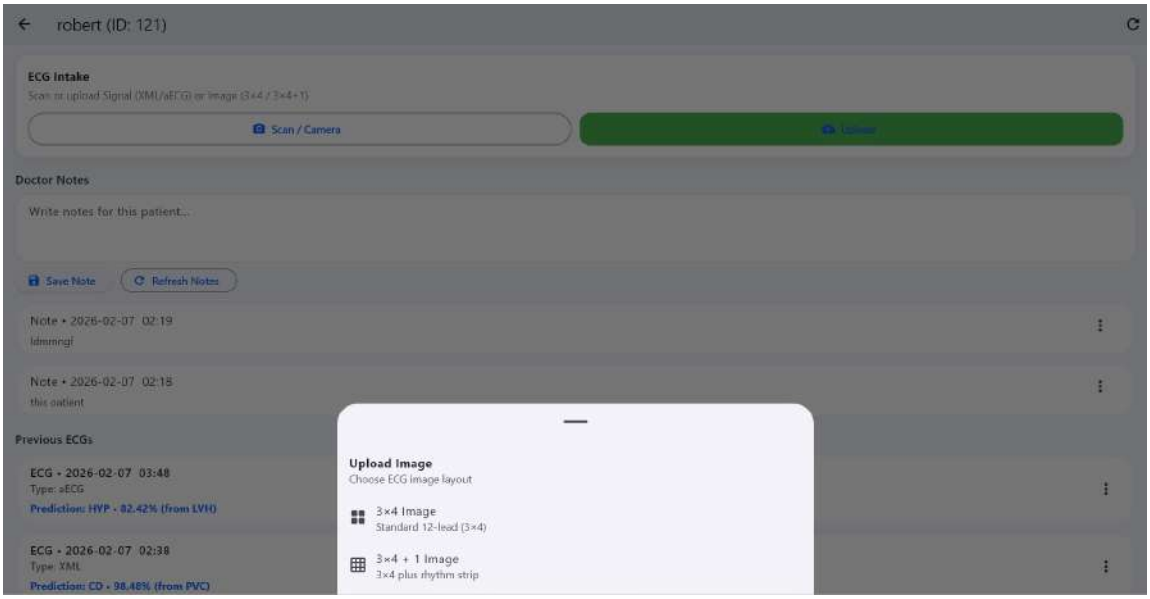


Figure 8.9: Image layout selection (Desktop/Web). The chosen layout guides cropping and preprocessing to correctly extract lead regions.

8.3.5 Result Visualization and Interpretability

HeartGuard emphasizes interpretability by presenting predictions alongside the processed ECG input that was used for inference. Presenting the processed/cropped input helps verify preprocessing quality and increases user trust.

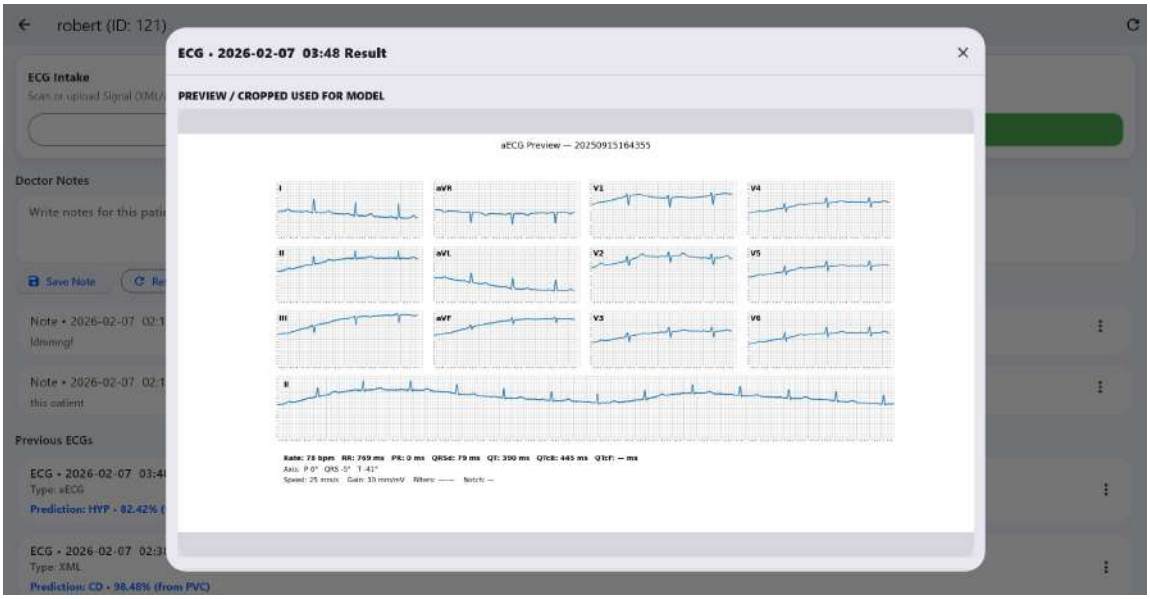


Figure 8.10: Result view (Desktop/Web, Preview). Displays the processed ECG region used as input to the inference pipeline.

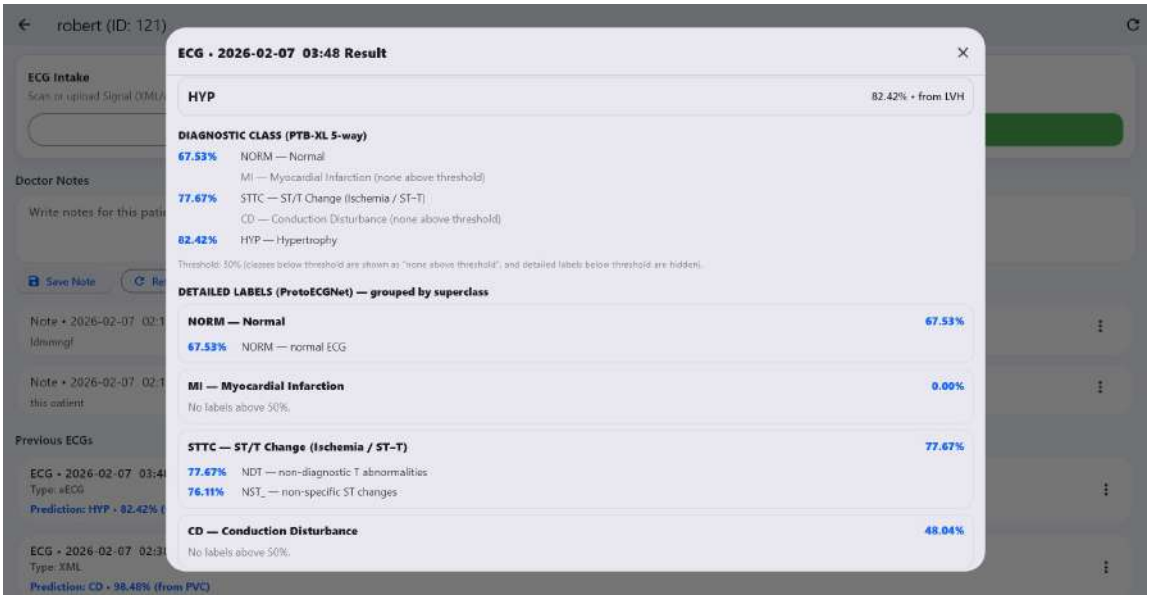


Figure 8.11: Result view (Desktop/Web, Scores). Predicted diagnostic outcomes presented with confidence scores in a structured format.

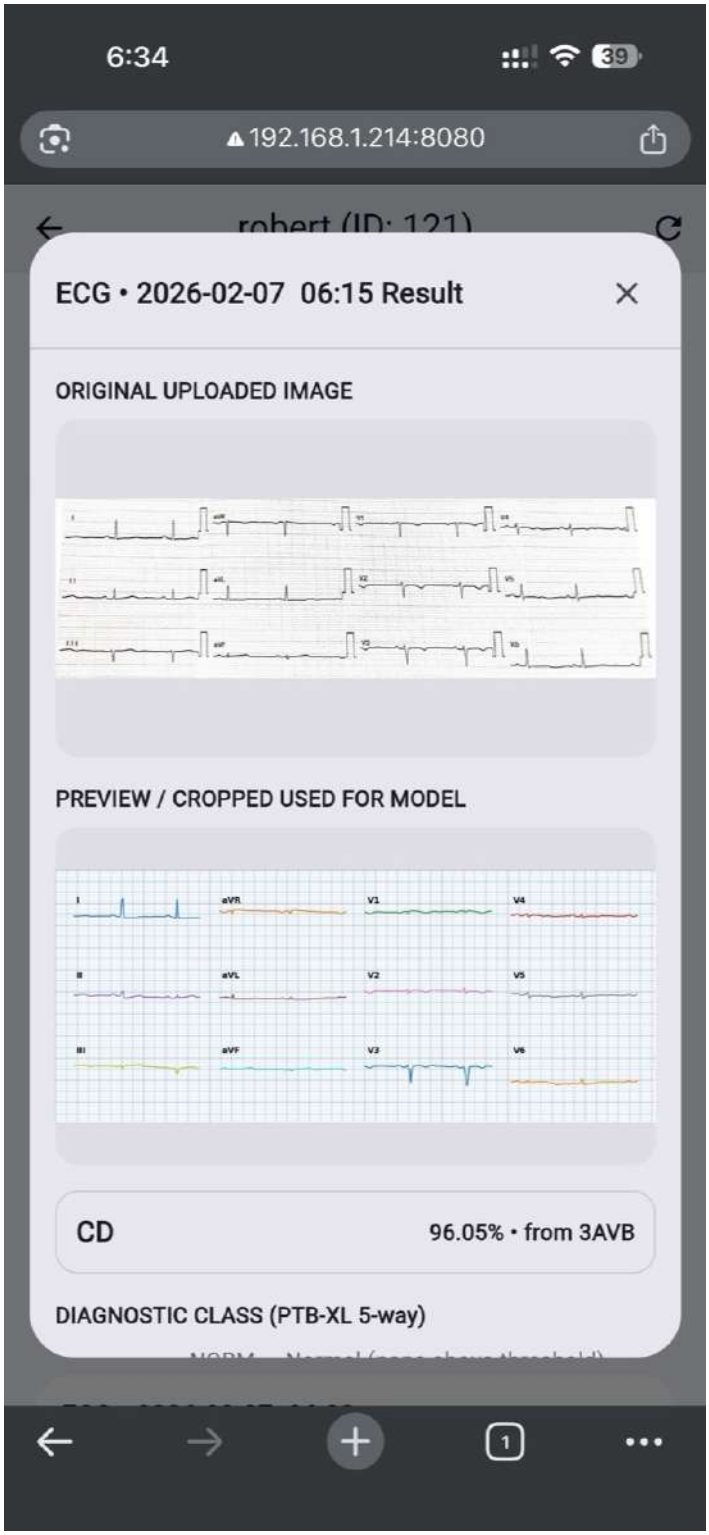


Figure 8.12: ECG result view (iOS). The interface displays the original uploaded ECG image and the processed (cropped) input used for inference, followed by the predicted diagnostic outcome and confidence score.

8.3.6 12-Lead ECG Reference Gallery

In addition to analysis, the application includes a 12-lead ECG reference gallery that supports learning and comparison. The gallery is **not used for inference**; instead, it provides categorized and searchable examples that can be used as a visual reference during clinical training and educational use.

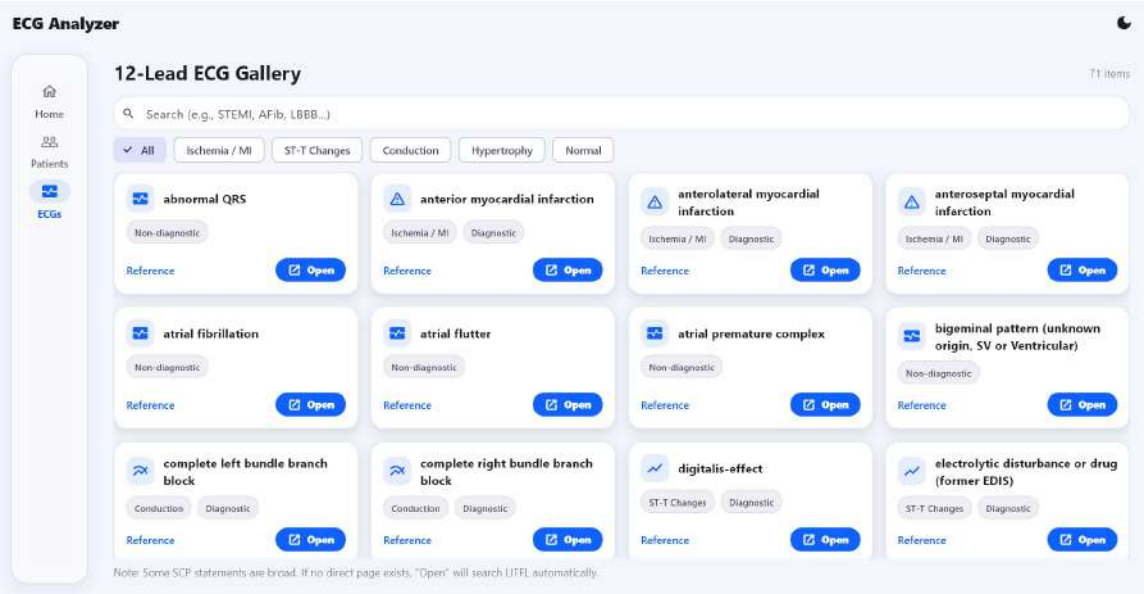


Figure 8.13: 12-Lead ECG reference gallery (Desktop/Web). Provides categorized and searchable ECG examples for educational use and visual comparison.

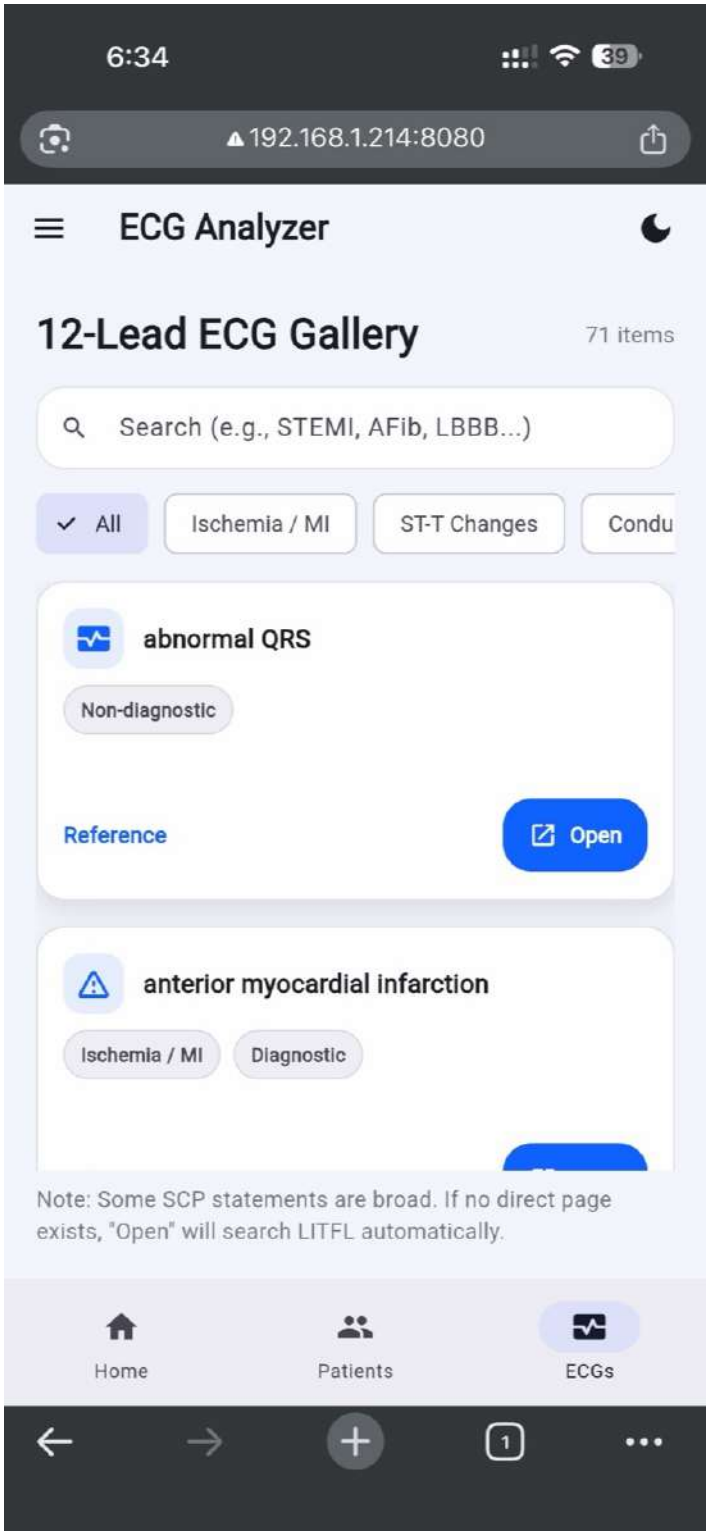


Figure 8.14: 12-lead ECG reference gallery (iOS). The gallery provides categorized and searchable ECG examples for educational use and visual comparison, and it is not used for automated inference.

8.3.7 Cross-Platform Deployment and Manual ECG Cropping

HeartGuard is implemented using Flutter, enabling a single codebase to be deployed across mobile (iOS/Android), desktop/laptop, and web environments. This cross-platform capability allows clinicians and users to access the system using different devices while preserving the same workflow and core functionality.

To improve robustness, the system provides a **manual cropping** feature for ECG images. This fallback mechanism is used when automatic cropping is not sufficient due to layout variability, borders, camera angle, or lighting conditions. Manual cropping ensures that ECG images uploaded or scanned by the user are correctly aligned before being transmitted to the backend for preprocessing and inference.

This feature reduces failure cases caused by real-world capture conditions and improves the reliability of downstream preprocessing and inference without requiring changes to the model itself.

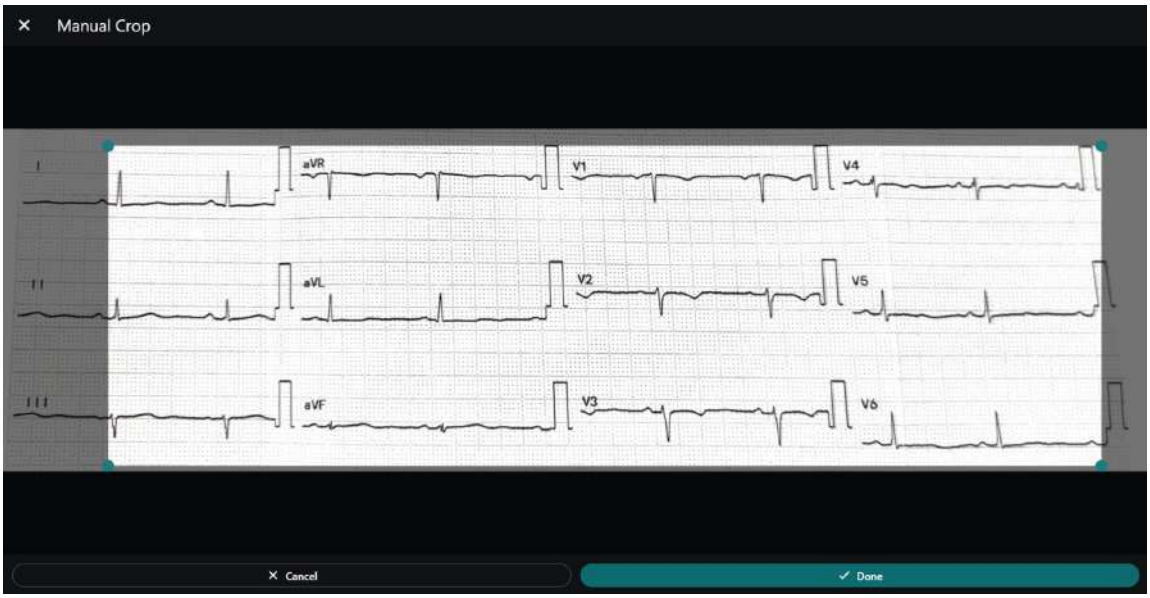


Figure 8.15: Manual ECG cropping interface (Desktop/Laptop). Providing manual cropping on desktop and mobile improves robustness and ensures correct preprocessing of scanned or uploaded ECG images before backend inference.

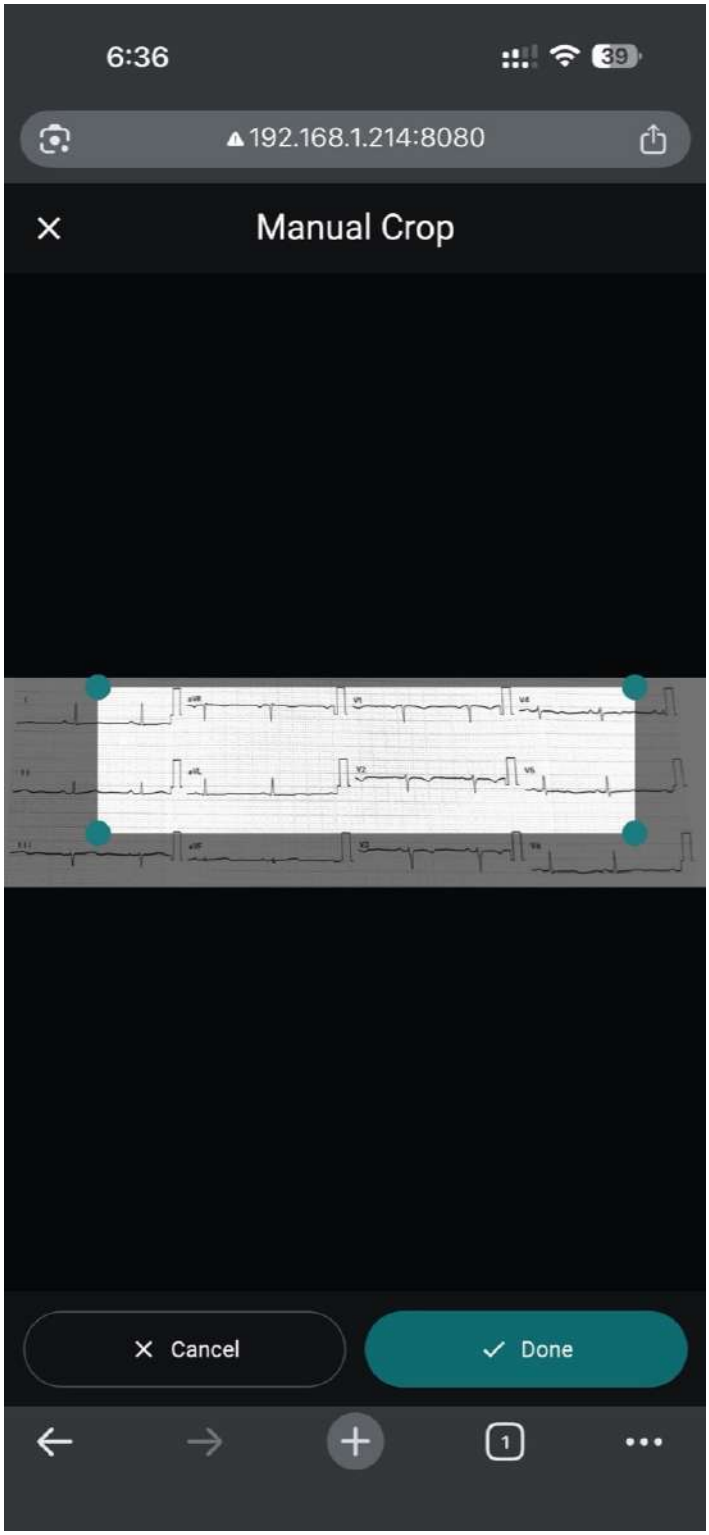


Figure 8.16: Manual ECG cropping interface (iOS). Users can manually adjust the cropping region to precisely select the relevant ECG area before processing and inference.

8.4 Backend Technology: Django (Django REST Framework)

8.4.1 What is Django

Django is a high-level Python web framework designed for rapid development of secure and maintainable server applications. Django REST Framework (DRF) extends Django with tools for building structured REST APIs, including request validation and serializers.

8.4.2 Why Django and DRF were Selected

Django/DRF were selected due to:

- **Python ecosystem:** direct integration with Python-based preprocessing and AI inference pipelines.
- **REST API development:** DRF provides clean, structured endpoints consumed by Flutter.
- **Maintainability:** modular apps and a clear separation of concerns.
- **Security and integrity:** mature framework with strong defaults and reliable data handling.

8.4.3 Database Design (PostgreSQL)

The backend stores patients, ECG metadata, prediction results, and doctor notes in a relational database. During early development, SQLite was used for local testing; however, the deployed system uses **PostgreSQL** to support concurrent access and production-level workloads.

Django's ORM provides database abstraction through consistent model definitions and schema migrations, enabling stable backend logic across database backends.

8.4.4 File Storage and Media Organization

Uploaded ECG files (images, XML, and aECG) and generated artifacts (cropped previews and inference outputs) are stored as media files referenced by database records. This design separates large binary objects from structured metadata, improving database performance and simplifying file management.

Each upload is handled as a persistent ECG case so the frontend can display the patient's history and reopen results at any time.

8.4.5 REST API Endpoints and Feature Integration

The backend exposes REST endpoints that map directly to Flutter actions such as creating patients, listing patients, saving notes, uploading ECGs, and retrieving predictions. This design keeps the frontend focused on presentation while centralizing validation, preprocessing, inference orchestration, and persistence on the backend.

Table 8.1: Mapping between Flutter features and Django REST APIs

Flutter Feature	**Backend Endpoint (Example)**	**Purpose**
Create/Login patient	`/api/patient/login_or_create/`	Create or load patient
List patients	`/api/patients/`	Populate patient list
Delete patient	`/api/patients/delete/`	Remove patient record
Doctor notes (save/list)	`/api/patients/<id>/notes/`	Persist clinician notes
Upload ECG (XML/aECG/image)	`/api/ecg/upload/`	Create ECG case and store input
List ECGs for a patient	`/api/ecg/list/?patient_id=<id>`	Load ECG history
Auto-crop image ECG	`/api/ecg/auto_crop/`	Layout-aware preprocessing
Run prediction / inference	`/api/ecg/predict/`	Return labels and confidence

8.4.6 End-to-End Processing Pipeline

After an ECG is uploaded, the backend executes an end-to-end pipeline:

1. **Input validation:** verify patient ID, file type, and required metadata (signal type or image layout).
2. **Preprocessing:** parse XML/aECG signals or crop/normalize image layouts.
3. **Inference orchestration:** execute segmentation/digitization when needed, followed by classification.
4. **Persistence:** store artifacts and predictions linked to the patient and ECG case.

5. **Response generation:** return a structured JSON payload for the Flutter client.

To support multiple input sources, the preprocessing stage routes inputs into format-specific handlers: **Philips XML** and **aECG** are parsed into waveform arrays with metadata, while **image uploads** are normalized through layout-aware cropping (3×4, 3×4+1) and optional segmentation/digitization before classification.

8.4.7 Robustness, Validation, and Error Handling

The backend provides robust handling for invalid uploads and inference failures. Unsupported file types, malformed XML/aECG, missing metadata, and failed inference steps return informative error messages that can be surfaced by the frontend. For example, malformed XML files or unsupported image layouts are rejected early with descriptive messages to prevent silent failures.

The HeartGuard system was implemented with a clear separation of concerns to make future changes safe. The Flutter frontend is organized into screens, reusable widgets, and service classes. This structure allows adding new result views or supporting new input formats without affecting existing UI components.

On the backend, Django REST Framework APIs provide a stable contract between the UI and inference modules. The inference pipeline is isolated from the presentation layer: uploaded files are validated and stored, then passed to dedicated preprocessing and inference utilities. This modular design makes it possible to replace or extend existing models (e.g., switching between ProtoECGNet, nnU-Net segmentation, or future classifiers) and to add post-processing steps (such as enhanced cropping, denoising, or richer structured output) without requiring changes in the Flutter screens, as long as the JSON response schema remains consistent.

To support maintainability, responsibilities are split across independent modules:

- **API Layer:** request handling, validation, and JSON responses using DRF.
- **Persistence Layer:** database models for patients, uploaded ECGs, and doctor notes.
- **Inference Layer:** preprocessing, model execution, and generation of preview/-analysis artifacts.

This architecture supports future enhancements such as role-based access (Doctor/Admin/Patient), JWT authentication, audit logging, and deployment in a clinical setting while minimizing refactoring risk.

8.5 Summary

This chapter presented the implementation of HeartGuard using Flutter and Django. Flutter provides a cross-platform workflow for patient management, ECG intake (XML/aECG/images), history tracking, interpretability-focused result visualization, and manual cropping support across **mobile, desktop/laptop, and web**. Django/DRF provide REST APIs, PostgreSQL-backed persistence, media storage, and orchestration of preprocessing and AI inference pipelines. Together, these components form a scalable and maintainable ECG analysis system.

Chapter 9

Results, Discussion, Limitations

Contents

9.1 Experimental Setup and Evaluation Protocol 96

9.1.1 Datasets Used for Evaluation 96

9.1.2 Evaluation of ECG Segmentation and Digitisation 96

9.1.3 Evaluation of Diagnostic Classification 97

9.1.4 Physician Comparison and Interpretation Protocol 97

9.1.5 Scope of Evaluation 98

9.2 Segmentation and Digitisation Results 98

9.2.1 Quantitative Segmentation Results 98

9.2.2 Qualitative Digitisation Results 99

9.3 Diagnostic Classification Results (ProtoECGNet) 100

9.3.1 Quantitative Classification Performance 100

9.3.2 Label-Level and Case-Level Predictions 101

9.4 Comparison with Physician Annotations 102

9.4.1 Case-by-Case Agreement Analysis 103

9.4.2 Direct vs. Indirect Label Mapping 108

9.5 Discussion 108

9.6 Limitations 110

9.6.1 Limited Availability of Labelled Real ECG Images 110

9.6.2 Absence of Paired Signal–Image Ground Truth 111

9.6.3 Dependence on PTB-XL for Diagnostic Training 111

9.6.4 Device and Acquisition Variability 111

9.6.5 Lack of Prospective Clinical Validation 111

9.1 Experimental Setup and Evaluation Protocol

This chapter evaluates the HeartGuard system as an end-to-end clinical decision-support pipeline, covering both ECG digitisation and diagnostic classification. The evaluation is designed to reflect realistic deployment conditions, where ECGs may be acquired as paper images or as digital signals, and where model outputs are compared against physician interpretations rather than idealised ground-truth labels alone.

The experimental evaluation is therefore organised into two main components: (1) assessment of the ECG digitisation and segmentation module, and (2) assessment of the diagnostic classification module based on ProtoECGNet. All evaluations are performed on data that were not used during model training, unless explicitly stated.

9.1.1 Datasets Used for Evaluation

Three main data sources are used during evaluation:

- **Public dataset (PTB-XL):** The official PTB-XL test split (stratified fold 10) is used to evaluate the diagnostic classification performance of ProtoECGNet. This split is strictly isolated from training and validation data to prevent information leakage and to enable fair comparison with the original ProtoECGNet results.
- **Locally collected digital ECG signals:** A subset of locally collected digital ECG recordings (primarily in aECG and XML-based formats) is used to evaluate real-world inference behaviour on signals obtained from hospital ECG machines. These recordings are processed through the same preprocessing and inference pipeline as PTB-XL data.
- **Locally collected ECG images:** ECG paper printouts acquired from multiple hospitals and clinics are used to evaluate the digitisation pipeline. These images include both scanned documents and smartphone photographs, covering common layouts such as 3×4 and 3×4 with rhythm strip. A small subset with manually traced masks is used for segmentation validation, while the remaining images are evaluated qualitatively.

9.1.2 Evaluation of ECG Segmentation and Digitisation

The segmentation module is evaluated using both quantitative and qualitative criteria:

- **Quantitative evaluation:** For ECG images with available ground-truth masks (synthetic data and manually traced real images), segmentation performance is measured using the Dice similarity coefficient. Dice scores are reported across cross-validation folds for the final binary segmentation models.
- **Qualitative evaluation:** For real hospital ECG pages without ground-truth masks, segmentation quality is assessed visually. Evaluation focuses on waveform continuity, suppression of grid lines and background artefacts, and the suitability of the extracted mask for downstream digitisation. Representative success cases and failure cases are presented.

Digitisation performance is assessed indirectly by examining the usability of the extracted waveform for diagnostic inference rather than by enforcing strict signal-level reconstruction error, which is not always feasible due to the absence of paired ground-truth signals for real images.

9.1.3 Evaluation of Diagnostic Classification

The diagnostic classification module is evaluated using standard multi-label classification metrics and case-based analysis:

- **Quantitative metrics:** Classification performance is measured using macro-averaged Area Under the Receiver Operating Characteristic Curve (AUROC) and F1-score. These metrics are computed separately for the rhythm, morphology, and global branches, as well as for the final fused model.
- **Case-level analysis:** In addition to aggregate metrics, model predictions are analysed on a per-case basis. For each ECG, the top predicted labels and their associated probabilities are examined and compared with physician annotations. This analysis highlights both exact matches and clinically meaningful partial agreements.

9.1.4 Physician Comparison and Interpretation Protocol

For locally collected ECGs reviewed by physicians, model outputs are compared against documented doctor findings. Since the model label space does not always map one-to-one with clinical terminology, results are categorised as:

- **Direct match:** model label corresponds explicitly to the physician finding.
- **Indirect match:** model label reflects the same underlying pathology using a different but clinically consistent description.
- **Not supported:** the physician finding is not represented in the model's label set.

This comparison protocol enables a clinically realistic assessment of system behaviour, recognising that diagnostic agreement is not always binary and that multiple interpretations may coexist for the same ECG.

9.1.5 Scope of Evaluation

It is important to note that HeartGuard is evaluated as a **decision-support system** rather than an autonomous diagnostic tool. The evaluation therefore emphasises robustness, interpretability, and clinical plausibility of outputs, in addition to numerical performance. Regulatory-level validation and outcome-based clinical trials are beyond the scope of this project and are discussed as future work.

9.2 Segmentation and Digitisation Results

This section presents the experimental results obtained for the ECG segmentation and digitisation stage of the HeartGuard pipeline. Since digitisation is a prerequisite for reliable downstream diagnosis, its performance is evaluated independently using both quantitative metrics and qualitative visual inspection. The results cover different ECG layouts (3×4 and 3×4 with rhythm strip) and compare binary and multi-class segmentation strategies.

9.2.1 Quantitative Segmentation Results

Quantitative evaluation of the segmentation models is performed using the Dice similarity coefficient on samples with available ground-truth masks. These include synthetic ECG pages generated from PTB-XL signals and a small subset of real hospital ECG images with manually traced binary masks. Dice scores are reported using cross-validation to assess training stability and generalisation.

Binary Segmentation Performance (3×4 Layout)

Table 5.1 summarises the mean validation Dice scores obtained across five cross-validation folds for the final binary segmentation model trained on the 3×4 ECG layout. The model achieves a mean Dice score of **0.8390**, with relatively low variance across folds, indicating stable training behaviour.

These results confirm that binary segmentation is well-suited for isolating ECG waveform traces in standard printed layouts, particularly when trained using a mixture of synthetic and manually annotated real data.

Binary vs. Multi-Class Segmentation

In addition to binary segmentation, a multi-class segmentation approach was explored for the 3×4 plus rhythm-strip layout, where pixel labels ranged from 0 to 12. The goal was to explicitly separate waveform components and layout regions. However, as shown in Table 5.2, the multi-class model achieved a mean Dice score of approximately **0.41** across folds, which is significantly lower than the binary counterpart.

This performance gap highlights the increased difficulty of multi-class supervision in real ECG page segmentation. The ECG waveform is a thin, high-frequency structure, and real scans introduce grid artefacts, noise, and printing variability that make class boundaries ambiguous. As a result, the multi-class objective produced unstable optimisation and poor generalisation, despite apparently correct label definitions.

Based on these findings, binary segmentation (waveform vs. background) was selected as the final approach for both the 3×4 and 3×4 plus rhythm-strip layouts.

9.2.2 Qualitative Digitisation Results

While quantitative metrics provide useful summary statistics, qualitative evaluation is essential for assessing real-world usability, especially for ECG images without ground-truth masks. This subsection presents representative qualitative results illustrating both successful and failure cases of the digitisation pipeline.

Segmentation on Synthetic ECG Pages

On synthetic ECG pages, the binary segmentation model produces clean and continuous masks that closely follow the waveform trace. Grid lines and background elements are effectively suppressed, and the extracted masks are well aligned with the rendered signal. These results are consistent with the high Dice scores observed during validation and confirm correct learning under controlled conditions.

Segmentation on Real Hospital ECG Pages

Figure 5.7 shows a representative qualitative example of the final binary segmentation model applied to a real hospital ECG page in a 3×4 layout. The predicted mask demonstrates good waveform continuity and alignment with the underlying trace, while suppressing most background grid artefacts and noise.

In practice, such masks are sufficient for downstream digitisation and diagnostic inference, even when minor artefacts remain. This supports the decision to evaluate digitisation primarily through usability rather than strict signal-level reconstruction error.

Failure Modes and Multi-Class Segmentation Breakdown

Figure 5.8 illustrates a typical failure case of the attempted multi-class segmentation model. The prediction exhibits inconsistent class assignments, fragmented waveform regions, and missing trace segments. These errors reflect the model's inability to robustly separate fine-grained classes under real scan variability.

Such failure modes motivated abandoning the multi-class objective in favour of binary segmentation, which proved substantially more robust and clinically adequate for ECG waveform extraction.

Impact of Digitisation on Downstream Diagnosis

Errors introduced during digitisation directly affect the diagnostic stage, as missing or broken waveform segments can lead to incorrect or uncertain classification. By isolating digitisation as a standalone evaluation stage, the results in this section clarify that most downstream diagnostic errors are attributable either to unavoidable image artefacts or to limitations in the acquisition quality, rather than to segmentation instability in the final binary model.

9.3 Diagnostic Classification Results (ProtoECGNet)

This section presents the diagnostic classification results obtained using the ProtoECGNet-based architecture within the HeartGuard system. The evaluation focuses on both aggregate quantitative performance and detailed case-level behaviour, reflecting the multi-label and clinically heterogeneous nature of ECG interpretation.

9.3.1 Quantitative Classification Performance

Quantitative evaluation is performed on the official PTB-XL test split using macro-averaged AUROC and F1-score, following the same evaluation protocol described in Chapter 6. Results are reported separately for the rhythm branch, morphology branch, global branch, and the final fusion model.

Branch-wise Performance

The rhythm branch demonstrates strong performance on rhythm-related conditions such as atrial fibrillation, supraventricular tachycardia, and conduction abnormalities. This behaviour is expected, as rhythm disorders are primarily characterised by temporal patterns that are well captured by one-dimensional signal representations.

The morphology branch achieves competitive performance on morphology-dominated diagnoses, including myocardial infarction patterns and ischemic ST–T changes. By op-

erating on two-dimensional ECG representations, this branch effectively captures spatial lead relationships and waveform shape variations.

The global branch, which integrates higher-level representations across leads, provides complementary information that improves robustness in complex multi-label cases. While its standalone performance is slightly lower than the specialised branches, it contributes important contextual features to the fused prediction.

Fusion Model Performance

The fusion classifier consistently outperforms individual branches across most diagnostic categories. By combining rhythm, morphology, and global features, the fusion model achieves the highest macro-averaged AUROC and F1-score, confirming the benefit of multi-view representation learning for ECG diagnosis.

Overall performance trends closely match those reported in the original ProtoECGNet publication, with minor deviations attributable to differences in preprocessing and implementation details. This consistency validates the correctness of the reproduction and integration of ProtoECGNet within the HeartGuard pipeline.

Comparison with Original ProtoECGNet Results

A direct comparison with the original ProtoECGNet results shows that the reproduced fusion model achieves comparable diagnostic performance on PTB-XL. Slight reductions in certain categories are observed, particularly for rare labels, which is consistent with expected variance across implementations. Importantly, no systematic performance degradation is observed, supporting the reliability of the implemented training and inference pipeline.

9.3.2 Label-Level and Case-Level Predictions

While aggregate metrics provide an overall performance summary, ECG diagnosis is inherently case-dependent and multi-label. To capture this aspect, model predictions are further analysed at the label and case levels.

Top-k Predictions and Probability Distributions

For each ECG record, the model outputs a ranked list of predicted labels with associated probabilities. Analysis of the top-k predictions reveals that clinically relevant diagnoses are frequently assigned high probabilities, even when multiple pathologies coexist.

Probability distributions often show gradual confidence decay rather than sharp cut-offs, reflecting diagnostic uncertainty commonly encountered in real clinical practice. This behaviour supports the use of probability-aware outputs rather than binary decisions.

Direct, Indirect, and Unsupported Labels

Model predictions are interpreted using the comparison protocol defined in Section 9.4. Specifically:

- **Direct labels** correspond exactly to physician-reported diagnoses (e.g., atrial fibrillation, ST elevation myocardial infarction).
- **Indirect labels** capture the same underlying pathology using related descriptors (e.g., ischemic ST–T changes instead of a specific infarction subtype).
- **Unsupported labels** represent findings that are not explicitly included in the model label space.

Case-level analysis demonstrates that many apparent mismatches fall into the indirect category, indicating clinically meaningful agreement despite differences in terminology.

Clinical Interpretation of Case-Level Outputs

In several cases, the model predicts multiple high-probability labels that collectively describe the physician interpretation more completely than a single diagnosis. For example, ischemia-related labels may co-occur with conduction abnormalities or ventricular hypertrophy, reflecting realistic multi-pathology ECGs.

Such outputs highlight the suitability of ProtoECGNet for multi-label ECG interpretation and reinforce the role of HeartGuard as a decision-support system rather than a deterministic diagnostic tool.

9.4 Comparison with Physician Annotations

This section presents a qualitative, case-by-case comparison between physician-annotated ECG findings and the corresponding predictions produced by the HeartGuard system. The goal of this analysis is to assess the clinical plausibility of the model outputs when applied to real-world ECG data, rather than to perform a statistical performance comparison.

Two independent datasets are considered. The first dataset consists of hospital-grade digital ECG recordings (XML format) obtained from Al-Maqassed Hospital, which were annotated by a cardiology physician. The second dataset comprises ambulatory ECG recordings in aECG format collected from Al Shifa Specialized Complex and annotated by a different physician.

For each ECG record, model predictions are evaluated against the corresponding physician interpretation on a case-by-case basis. Agreement is assessed qualitatively, taking into account both exact matches and clinically meaningful partial agreement.

Differences in terminology are expected due to the mismatch between the model's fixed label space and the flexible language used in clinical reporting. As such, this section focuses on clinical consistency and interpretability rather than strict label matching.

9.4.1 Case-by-Case Agreement Analysis

This subsection presents a qualitative, case-by-case analysis of the agreement between physician interpretations and the diagnostic outputs produced by the HeartGuard system. The comparison is conducted at the individual ECG level, reflecting realistic clinical usage rather than aggregate statistical evaluation.

Comparison methodology. For each ECG record, the physician's interpretation is decomposed into one or more reported findings. These findings are then compared individually against the set of labels predicted by the model. Agreement is therefore assessed at the *finding level*, rather than assigning a single agreement label to the entire ECG.

Each physician finding is categorised into one of three qualitative outcomes:

- **Direct agreement**, where a model label explicitly corresponds to the physician-reported finding.
- **Indirect agreement**, where the model predicts a clinically related or conceptually consistent label using different terminology.
- **Not supported**, where the physician-reported finding is not represented in the model label space.

No numerical counting or percentage-based agreement metrics are reported. This design choice reflects the limited number of evaluated cases and the primary objective of this analysis, which is **qualitative clinical validation** rather than statistical performance measurement. Emphasis is placed on clinical plausibility, interpretability, and consistency of the model outputs.

Hospital-grade ECG data (XML, Al-Maqassed Hospital). The first table presents case-by-case comparisons for hospital-grade digital ECG recordings acquired in XML format from Al-Maqassed Hospital and annotated by a cardiology physician. For each case, multiple physician findings may be listed and evaluated independently against the corresponding model predictions using the agreement categories defined above.

Ambulatory ECG data (aECG, Al Shifa Specialized Complex). The second table presents case-by-case comparisons for ambulatory ECG recordings in aECG format collected from Al Shifa Specialized Complex and annotated by a different physician.

As with the hospital dataset, agreement is assessed at the level of individual findings, reflecting the variability and descriptive nature of ambulatory ECG reporting.

Summary and table placement. The tables that follow provide detailed, finding-level evidence supporting the qualitative observations discussed in this subsection. They should be interpreted as structured documentation of agreement patterns between physician findings and model outputs, rather than as standalone performance summaries.

Table 9.1: Comparison between physician-annotated ECG findings and model predictions for hospital-grade XML ECG data (Al-Maqassed Hospital).

Case ID	Clinical diagnosis	Doctor finding	Model label	p	Label availability
1	Sinus rhythm, undefined axis, RVH	Sinus rhythm	SARRH	0.7617	Indirect
		Undefined axis	LAFB	0.6898	Indirect
		Undefined axis	CLBBB	0.6669	Indirect
		RVH	RVH	0.2648	Direct
2	Sinus rhythm, undefined axis, RVH	Sinus rhythm	STACH	0.8835	Indirect
		Undefined axis	LAFB	0.7932	Indirect
		Undefined axis	CLBBB	0.6962	Indirect
		RVH	RVH	0.3169	Direct
3	Sinus rhythm, possible pulmonary embolism	Sinus rhythm	–	–	Direct
		Right heart strain	CRBBB	0.7347	Indirect
		Axis abnormality	LAFB	0.8874	Indirect
		Q wave / T inversion	ISC_	0.8553	Indirect
		Q wave / T inversion	INJAL	0.9220	Indirect
4	Sinus rhythm, 1st degree AV block, LVH	Sinus rhythm	–	–	Direct
		1st degree AV block	1AVB	0.1843	Direct
		LVH	LVH	0.9781	Direct
5	Pacemaker rhythm, LBBB	Pacemaker rhythm	PACE	0.9682	Direct
		LBBB	CLBBB	0.7172	Direct
6	Sinus rhythm, low amplitude	Sinus rhythm	STACH	0.7309	Indirect
		Low amplitude	–	–	Not supported
7	Sinus rhythm, normal ECG	Sinus rhythm	SR	0.4811	Direct
		Normal ECG	NORM	0.1199	Direct

Case ID	Clinical diagnosis	Doctor finding	Model label	p	Label availability
8	Sinus rhythm, old MI (anterolateral)	Sinus rhythm	SR	0.4143	Direct
		Old MI	ASMI	0.9184	Direct
		Old MI	LMI	0.7714	Direct
		Q wave	ABQRS	0.8309	Direct
		T-wave inversion	INVT	0.2076	Direct
9	Atrial fibrillation, right axis deviation	Atrial fibrillation	AFIB	0.4953	Direct
		Right axis deviation	RAO/RAE	0.0468	Direct
10	Atrial fibrillation, right axis deviation	Atrial fibrillation	AFIB	0.4106	Direct
		Right axis deviation	RAO/RAE	0.0770	Direct
11	Sinus rhythm, anterior ischemia	Sinus rhythm	SR	0.4448	Direct
		ST depression	STD_	0.4703	Direct
		Anterior ischemia	INJAL	0.8219	Direct
		Ischemia	ISC_	0.7901	Direct
12	Subacute inferior STEMI with lateral ST depression	Sinus rhythm	SR	0.4691	Direct
		Inferior STEMI (subacute)	INJAS	0.8103	Direct
		Ischemia (general)	ISC_	0.8899	Direct
		Q wave	QWAVE	0.0572	Direct
		QRS/Q wave abnormality	ABQRS	0.6544	Direct
		ST elevation	STE_	0.0087	Direct
		T-wave inversion	INVT	0.6276	Direct
		ST depression	STD_	0.5552	Direct
13	Sinus rhythm, anteroseptal STEMI	Sinus rhythm	SR	0.3353	Direct
		STEMI anteroseptal	ISC_	0.8590	Indirect
		QRS abnormality	ABQRS	0.7909	Direct
14	Sinus rhythm, inferior STEMI (subacute)	Sinus rhythm	SR	0.4498	Direct
		Inferior STEMI	ISC_	0.8615	Indirect
15	Sinus rhythm, 1st degree AV block, LAD	Sinus rhythm	SR	0.4830	Direct
		1st degree AV block	1AVB	0.1460	Direct
		Left axis deviation	LAFB	0.9399	Direct
16	Sinus rhythm, left axis deviation	Sinus rhythm	SR	0.4621	Direct
		Left axis deviation	LAFB	0.9324	Direct

Case ID	Clinical diagnosis	Doctor finding	Model label	p	Label availability
17	Sinus rhythm, T-wave flattening (lateral)	Sinus rhythm	SR	0.4993	Direct
		T-wave flattening	LOWT	0.0229	Direct
18	Sinus rhythm, lateral ischemia	Sinus rhythm	SR	0.3621	Direct
		Biphasic T wave	INVT	0.8788	Direct
		Lateral ischemia	INJAL	0.9461	Direct
19	Sinus rhythm, normal ECG	Sinus rhythm	SR	0.4661	Direct
		Normal ECG	NORM	0.1476	Direct
20	Sinus rhythm, LAD, ischemia	Sinus rhythm	SR	0.4332	Direct
		Left axis deviation	LAFB	0.8767	Direct
		Inverted T wave	INVT	0.4479	Direct
		Poor R-wave progression	ABQRS	0.7316	Direct
		Ischemia	ISC_	0.9146	Direct

Table 9.2: Comparison between physician-annotated ECG findings and model predictions for ambulatory aECG data (Al Shifa Specialized Complex).

Case ID	Clinical diagnosis	Doctor finding	Model label	p	Label availability
1	Ischemic ST–T changes (lateral) · LVH · T-wave inversion	Lateral ischemic ST–T changes	ISCAL	0.9921	Direct
		T-wave inversion	INVT	0.9964	Direct
		Left ventricular hypertrophy	LVH	0.8058	Direct
2	Low amplitude T waves	Low amplitude T waves	LOWT	0.7999	Direct
3	Normal ECG	Normal ECG	NORM	0.6753	Direct
4	Normal ECG	Normal ECG	NORM	0.7914	Direct
5	Non-specific ST changes · T-wave inversion · Incomplete RBBB	Non-specific ST changes	NST_	0.5122	Direct
		T-wave inversion	INVT	0.212	Direct
		Incomplete right bundle branch block	IRBBB	0.4159	Direct

Case ID	Clinical diagnosis	Doctor finding	Model label	p	Label availability
6	T-wave inversion · Sinus bradycardia	T-wave inversion	INVT	0.1630	Direct
		Sinus bradycardia	SBRAD	0.1306	Direct
7	ST elevation inferior + lateral	Inferior ST elevation	IMI / QWAVE	0.7757 / 0.9064	Indirect
		Lateral ST elevation	ASMI	0.5537	Indirect
8	Normal ECG	Normal ECG	NORM	0.8263	Direct
9	Low amplitude T waves	Low amplitude T waves	LOWT	0.9613	Direct
10	Sinus tachycardia	Sinus tachycardia	SR	0.9724	Indirect
11	Non-specific intraventricular delay	Non-specific intraventricular delay	ABQRS	0.9065	Indirect
12	T-wave inversion + Left atrial enlargement	T-wave inversion	STD_ / LOWT	0.9138 / 0.9436	Indirect
		Left atrial enlargement	LAO/LAE	0.2333	Direct
13	Artifact ?? · T-wave inversion · Non-specific ST–T changes	Artifact ??	Artifact	–	Not supported
		T-wave inversion	LOWT / STD_	0.9827 / 0.9541	Indirect
		Non-specific ST–T changes	NST_	0.9377	Direct
14	Normal ECG	Normal ECG	NORM	0.4539	Direct
15	T-wave inversion · ST depression (lateral/septal) (INVT + STD + ISCAS/ISCLA)	T-wave inversion	INVT	0.8630	Direct
		ST depression (lateral/septal)	STD_	0.9451	Direct
		Ischemia (lateral/septal)	ISCAL / ISCIN	0.8829 / 0.8467	Indirect

9.4.2 Direct vs. Indirect Label Mapping

Differences between physician terminology and model output labels are expected due to the inherent mismatch between clinical reporting language and the fixed diagnostic label space used by the model. Physicians often describe ECG findings using flexible, context-dependent expressions, whereas the model operates within a predefined set of discrete labels learned during training.

A **direct label** occurs when a model prediction explicitly corresponds to a physician-reported finding using the same or an equivalent diagnostic concept. In such cases, agreement is straightforward and reflects a one-to-one correspondence between clinical interpretation and model output.

An **indirect label** occurs when the model predicts a label that is clinically consistent with the physician finding but expressed using different terminology or a related diagnostic abstraction. This is common when physicians describe composite findings or higher-level interpretations, while the model outputs constituent or surrogate features (e.g., conduction abnormalities, repolarisation changes, or axis-related descriptors). Such cases represent clinically meaningful agreement rather than diagnostic error.

Finally, a finding is marked as **not supported** when the physician-reported observation is not represented within the model's label space. This includes findings such as artefacts or descriptive comments that fall outside the scope of the trained diagnostic categories. These cases highlight limitations of label coverage rather than incorrect model behaviour.

Overall, this mapping framework enables a clinically realistic interpretation of agreement by distinguishing between exact correspondence, conceptually consistent agreement, and genuine absence of label support. It supports qualitative validation of the HeartGuard system while acknowledging differences in diagnostic granularity and terminology between human experts and machine learning models.

9.5 Discussion

This section discusses the experimental findings from a conceptual and clinical perspective, focusing on system behaviour, design choices, and practical implications rather than restating numerical results. The discussion integrates observations from digitisation, classification, and physician comparison to highlight strengths, limitations, and deployment considerations of the HeartGuard system.

Suitability of ProtoECGNet for Multi-Label ECG Interpretation

Electrocardiograms frequently exhibit multiple coexisting abnormalities, such as rhythm disturbances combined with ischemic changes or structural findings. Traditional single-

label classifiers are inherently limited in such settings, as they force mutually exclusive decisions that do not reflect clinical reality.

ProtoECGNet is particularly well suited for multi-label ECG interpretation due to its explicit modelling of multiple diagnostic branches and its use of prototype-based representations. By learning independent yet complementary feature spaces for rhythm, morphology, and global patterns, the model can assign multiple clinically relevant labels to the same ECG without enforcing artificial exclusivity. This behaviour aligns naturally with physician reasoning, where ECG interpretation is often decomposed into rhythm analysis, waveform morphology assessment, and contextual synthesis.

Interpretation of Probabilities and Multi-Hypothesis Outputs

In several cases, correct or clinically relevant labels are associated with relatively low predicted probabilities. This behaviour should not be interpreted as model uncertainty or failure, but rather as a reflection of the inherently ambiguous nature of ECG interpretation. Many ECG findings exist on a continuum, and borderline or evolving patterns may not strongly activate a single prototype.

Similarly, the model frequently produces multiple plausible diagnostic hypotheses with comparable probabilities. From a clinical perspective, this behaviour is desirable in a decision-support context, as it mirrors the differential diagnosis process used by physicians. Rather than forcing a single dominant label, the model exposes alternative interpretations that may warrant further clinical correlation or follow-up testing.

Impact of Digitisation on Diagnostic Classification

The digitisation stage plays a critical role in the overall performance of the HeartGuard pipeline. Errors or artefacts introduced during segmentation and waveform extraction directly affect downstream classification, particularly for morphology-sensitive diagnoses such as ischemic ST–T changes or subtle repolarisation abnormalities.

By evaluating digitisation independently and prioritising waveform usability over strict signal reconstruction accuracy, the system achieves a practical balance between robustness and diagnostic fidelity. The observed classification behaviour confirms that the final binary segmentation approach provides sufficient signal quality for reliable inference in most real-world cases, while remaining resilient to common image artefacts encountered in clinical environments.

Importance of Explainability in Emergency and Clinical Settings

Explainability is a central requirement for clinical decision-support systems, particularly in emergency or high-stakes environments where rapid trust calibration is essential. The

prototype-based nature of ProtoECGNet allows predictions to be interpreted in terms of similarity to learned diagnostic exemplars, rather than opaque feature activations.

This interpretability facilitates clinician confidence, supports error analysis, and enables meaningful discussion between human experts and the system. In scenarios such as suspected myocardial infarction or arrhythmia detection, the ability to understand why a label was suggested is as important as the label itself.

HeartGuard as a Decision-Support System

The results and analyses presented throughout this chapter reinforce the positioning of HeartGuard as a **decision-support system** rather than a diagnostic replacement. The system is designed to assist clinicians by highlighting relevant findings, providing probabilistic insights, and supporting comprehensive interpretation, while leaving final diagnostic responsibility to human experts.

By combining digitisation, interpretable multi-label classification, and physician-aligned evaluation protocols, HeartGuard demonstrates how machine learning can augment clinical workflows without undermining professional judgment. This design philosophy is essential for safe and responsible integration of artificial intelligence into healthcare practice.

9.6 Limitations

While the HeartGuard system demonstrates promising performance as an interpretable, end-to-end ECG decision-support pipeline, several limitations must be acknowledged. These limitations reflect practical constraints of data availability, clinical validation scope, and real-world deployment variability, rather than fundamental shortcomings of the proposed methodology.

9.6.1 Limited Availability of Labelled Real ECG Images

A primary limitation of this work is the relatively small number of real ECG images with high-quality manual annotations available for training and evaluation of the digitisation module. Although synthetic ECG pages and a subset of manually traced real images were used to enable quantitative segmentation evaluation, the majority of real-world ECG images lack precise ground-truth masks.

As a result, digitisation performance on real images is evaluated primarily through qualitative inspection and downstream usability rather than strict pixel-level accuracy. While this reflects realistic deployment conditions, it limits the ability to report comprehensive quantitative reconstruction metrics on real data.

9.6.2 Absence of Paired Signal–Image Ground Truth

In many real clinical settings, ECG paper printouts and the original digital signals are not simultaneously available. This limitation is reflected in the evaluation setup, where paired signal–image ground truth is often missing for real ECG images.

Consequently, digitisation accuracy cannot always be verified against the true underlying waveform, and reconstruction fidelity is assessed indirectly through segmentation quality and diagnostic inference behaviour. This constraint highlights a broader challenge in ECG digitisation research rather than a limitation specific to HeartGuard.

9.6.3 Dependence on PTB-XL for Diagnostic Training

The diagnostic classification module relies heavily on the PTB-XL dataset for training and quantitative evaluation. Although PTB-XL is a large, well-curated, and widely accepted public dataset, it may not fully capture the diversity of ECG acquisition devices, patient populations, and reporting practices encountered in different clinical environments.

This dependence may affect generalisation when deploying the system across institutions that use different ECG machines, lead configurations, or annotation conventions. The case-level physician comparison partially mitigates this concern by demonstrating clinically plausible behaviour on locally collected data, but broader multi-centre validation remains necessary.

9.6.4 Device and Acquisition Variability

ECG signals and images vary substantially depending on the recording hardware, sampling rates, printing resolution, paper quality, and scanning or photography conditions. While HeartGuard is designed to handle common layouts and acquisition variability, extreme differences in device characteristics or acquisition protocols may degrade digitisation or classification performance.

Addressing such variability would require additional device-specific calibration, expanded training data, or adaptive preprocessing strategies.

9.6.5 Lack of Prospective Clinical Validation

Finally, the evaluation presented in this work does not constitute prospective clinical validation or regulatory-grade assessment. The system has not undergone controlled clinical trials, outcome-based evaluation, or regulatory review (e.g., FDA or CE certification).

Accordingly, HeartGuard should be viewed as a research prototype and decision-support system rather than a clinically approved diagnostic tool. Future work should include structured clinical studies to evaluate real-world impact, physician interaction, and patient outcomes.

9.6.6 Summary

These limitations define the current scope of the HeartGuard project and provide clear directions for future research. Importantly, they do not undermine the core contributions of this work, which lie in integrating interpretable ECG classification with robust digitisation and clinically grounded evaluation within a unified pipeline.

Chapter 10

Conclusion and Future Work

Contents

10.1 Conclusion 114

10.2 Project Contributions 114

10.2.1 System-Level Contributions 115

10.2.2 Digitisation Contributions 115

10.2.3 Diagnosis and Interpretability Contributions 115

10.3 Limitations 115

10.3.1 Domain Gap and Limited Real Labels 116

10.3.2 Layout Coverage 116

10.3.3 Digitisation Evaluation Constraints 116

10.3.4 Clinical Validation and Regulatory Readiness 116

10.3.5 Diagnosis Generalisation to Local Population 116

10.4 Future Work 116

10.4.1 Scaling Real Mask Collection and Active Learning 117

10.4.2 Automatic Layout Detection and Generalised Digitisation . . . 117

10.4.3 Improved Signal Reconstruction 117

10.4.4 Diagnosis: Local Fine-Tuning and External Validation 117

10.4.5 Explainability Improvements for Clinical Use 118

10.4.6 Deployment, Security, and MLOps 118

10.5 Ethical Considerations and Intended Use 119

10.6 Closing Remarks 119

10.1 Conclusion

Heart Guard was developed as an end-to-end, deployable decision-support system that accepts ECG inputs in the forms hospitals actually produce—**paper/scanned/photographed ECG images** and, when available, **digital ECG signals**—and produces fast AI-assisted diagnostic outputs with interpretable evidence. The project was motivated by real clinical constraints observed during local hospital visits in Palestine, where ECG workflows are largely paper-based and access to direct signal export is inconsistent.

The system integrates three major components into one unified pipeline: (1) **ECG digitisation from images** via segmentation-based waveform extraction, (2) **signal standardisation and preprocessing** for consistent inference across acquisition routes, and (3) **interpretable multi-label diagnosis** using a ProtoECGNet-based classification architecture. These components are delivered through a **Flutter cross-platform client** (mobile/Windows) backed by a **Django REST backend** that orchestrates preprocessing, inference, persistence, and secure API access.

On the digitisation track, we validated that treating ECG digitisation as a segmentation problem is practical and scalable. Starting from a synthetic-only baseline that achieved high validation scores but failed on real hospital pages, we progressed to an nnU-Net-based pipeline that reduces manual tuning and improves reproducibility. Most importantly, we addressed the real-world domain gap by introducing paired augmentation and building a lightweight **Flask web-based ECG labeling tool** to generate real binary masks through manual tracing. Training on mixed synthetic and real labels yielded strong qualitative waveform continuity on real pages and produced usable digitised leads for downstream processing.

On the diagnosis track, we integrated an interpretable prototype-based model (ProtoECGNet) trained on PTB-XL with a leakage-free splitting strategy. The prototype mechanism provides case-based explanations aligned with clinical reasoning: predictions are supported by similarity to learned prototypes grounded in real training samples. Across the diagnostic label subsets and fusion stage, the implemented model achieved competitive results relative to the reference ProtoECGNet study while preserving interpretability as a first-class feature.

Overall, the project demonstrates that a clinically realistic pipeline—from paper ECG images to digitised signals and interpretable diagnosis—can be engineered as a single system that is practical under local constraints. Heart Guard is designed to support clinicians by accelerating preliminary assessment and triage, not to replace medical judgment.

10.2 Project Contributions

The main contributions of Heart Guard can be summarised as follows:

10.2.1 System-Level Contributions

- A complete **end-to-end ECG decision-support pipeline** that supports both image-based and signal-based inputs.
- A deployable **client–server architecture** combining Flutter (UI/capture/history) and Django (APIs, orchestration, storage, inference).
- A case-based workflow with **patient history and timestamped results**, enabling longitudinal case review.

10.2.2 Digitisation Contributions

- A segmentation-driven digitisation approach using **nnU-Net** for waveform extraction from ECG pages.
- A synthetic dataset generation pipeline (PTB-XL → ECG page → JSON → masks) supporting both 3×4 and 3×4+rhythm layouts.
- A robust paired augmentation strategy designed to narrow the synthetic-to-real gap.
- A practical digitisation method (binary mask → 12 leads in mV) with spike-aware extraction and calibration-block handling.
- A lightweight **Flask web-based annotation tool** enabling efficient manual tracing of real masks for domain adaptation.

10.2.3 Diagnosis and Interpretability Contributions

- Integration of an interpretable **ProtoECGNet-based** multi-label diagnostic model into the Heart Guard pipeline.
- A leakage-free training and evaluation setup where prototypes are learned and projected exclusively from the training folds.
- A unified inference interface that produces both **diagnosis outputs** and **prototype-based evidence** suitable for clinician review.

10.3 Limitations

Despite the progress achieved, several limitations remain and should be considered when interpreting the results and deployment readiness.

10.3.1 Domain Gap and Limited Real Labels

Although augmentation and mixed training improved robustness, the domain gap is not fully solved. Real ECG images vary significantly across hospitals, devices, printers, scanners, paper types, and capture conditions. The manually traced real subset was limited (100 pages), which constrains the diversity of real artifacts and may not represent the full operational distribution in deployment settings.

10.3.2 Layout Coverage

The system focuses primarily on the dominant locally observed layouts (3×4 and 3×4+rhythm). However, real hospitals also produce alternative formats (e.g., 12×1 rhythm pages, different lead ordering, different calibration placement). Expanding support to additional formats requires either layout detection or additional specialised models.

10.3.3 Digitisation Evaluation Constraints

Quantitative digitisation evaluation relied on a limited number of cases where a reliable reference (aECG/HL7 signal) could be matched to a printed page time window. In settings where signal export is unavailable, it is difficult to compute large-scale quantitative digitisation error without manual ground truth.

10.3.4 Clinical Validation and Regulatory Readiness

While the system is designed for clinical decision support, it has not undergone prospective clinical trials, external multi-site validation, or regulatory approval processes. Therefore, Heart Guard should be considered a research prototype until validated under formal clinical protocols.

10.3.5 Diagnosis Generalisation to Local Population

The diagnosis model was trained primarily using PTB-XL. Differences in patient population, device characteristics, and label distributions between PTB-XL and local hospitals may cause performance shifts. Closing this gap requires additional local labeled ECG signals and careful external validation.

10.4 Future Work

Future work can strengthen Heart Guard along four directions: digitisation robustness, diagnostic performance and calibration, deployment readiness, and clinical integration.

10.4.1 Scaling Real Mask Collection and Active Learning

A high-impact improvement is to expand the real labeled segmentation dataset:

- Increase manually traced pages beyond 100 and include multiple hospitals and devices.
- Use **active learning** to prioritise labeling of difficult images (shadows, low contrast, curved pages) by selecting samples with high prediction uncertainty.
- Add labeling tools for faster annotation (brush stabilization, spline tracing, auto-snap to trace pixels, quality checks).

10.4.2 Automatic Layout Detection and Generalised Digitisation

To support diverse hospital page formats:

- Implement **layout classification** (3×4 vs 12×1 vs other) and route inputs to the correct segmentation/digitisation model.
- Extend the ROI-splitting stage to be driven by detected grid structure or learned separators.
- Explore a single robust model trained across multiple layouts, using multi-task learning or conditional routing.

10.4.3 Improved Signal Reconstruction

Digitisation can be improved beyond binary extraction:

- Incorporate grid estimation to better convert pixels to time/mV using scale inferred from the paper grid.
- Use continuity-aware curve fitting (e.g., robust splines) with constraints that preserve QRS peaks without oversmoothing.
- Introduce quality scoring (per-lead confidence) to detect when a lead digitisation is unreliable and warn the user.

10.4.4 Diagnosis: Local Fine-Tuning and External Validation

For clinically realistic deployment:

- Collect local labeled ECG signals when permitted and fine-tune the diagnostic model using domain adaptation.

- Evaluate generalisation using external datasets and local test sets with clinically verified labels.
- Improve calibration and thresholding for multi-label outputs (e.g., per-class thresholds, confidence calibration).

10.4.5 Explainability Improvements for Clinical Use

ProtoECGNet provides inherent interpretability, but the presentation can be improved:

- Provide clearer prototype retrieval views in the UI (closest training case, similarity score, and highlighted lead/time segment).
- Add clinician-friendly summaries that map prototype evidence to standard ECG interpretation language.
- Add uncertainty indicators and "review required" flags for borderline cases.

10.4.6 Deployment, Security, and MLOps

To support secure real-world hospital deployment:

- Enforce strong access controls (RBAC with least-privilege), multi-factor authentication for privileged roles, and encrypted data handling (TLS in transit, encryption at rest) for all ECG images, signals, and metadata.
- Apply privacy-by-design controls: data minimisation, de-identification/pseudonymisation of patient identifiers, secure key management, and retention policies aligned with hospital governance and local regulations.
- Maintain full accountability: tamper-evident audit logs for data access and model usage, separation of duties for administrators, and regular security reviews (vulnerability scanning, patching, penetration testing).
- Operationalise models safely with MLOps: reproducible training pipelines, model versioning and rollback, dataset lineage, and controlled releases (staging $\rightarrow$ production) with approvals.
- Continuously monitor clinical performance: automated monitoring, drift detection, and alerting; periodic recalibration and re-validation on recent hospital data; and clear thresholds for disabling or reverting models if degradation is detected.
- Optimise reliability for emergency workflows: low-latency inference, resource isolation (GPU scheduling), batching/caching where safe, high availability, and fail-safe behaviour (graceful degradation and manual override when the system is uncertain).

10.5 Ethical Considerations and Intended Use

Heart Guard is designed as a **clinical decision-support system**, not as an autonomous diagnostic authority. Its purpose is to help clinicians by providing rapid preliminary insights and supporting evidence, while keeping the final responsibility with the healthcare professional. Any real-world deployment should:

- ensure clinicians retain full responsibility for diagnosis and treatment decisions,
- protect patient privacy through strict anonymisation, encryption, and secure data handling,
- be externally validated on local patient populations and real acquisition conditions before clinical use,
- communicate uncertainty, limitations, and failure cases clearly to reduce the risk of over-reliance.

“

10.6 Closing Remarks

This project demonstrates that a realistic ECG workflow—from paper ECG images to digitised waveforms and interpretable multi-label diagnosis—can be implemented as a single integrated system. Heart Guard was designed under real local constraints, and its modular architecture enables continuous improvement without redesigning the whole pipeline. With expanded real-data supervision, broader layout support, and rigorous clinical validation, Heart Guard can evolve from a research prototype into a deployable assistant that improves ECG triage speed and supports clinicians in time-critical environments.

Bibliography

[1] World Health Organization (WHO), "Cardiovascular diseases (CVDs) – Fact sheet," 2025. Accessed: 2026-02-02. Available: https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds).

[2] A. Y. Hannun, P. Rajpurkar, M. Haghpanahi, et al., "Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network," *Nature Medicine*, 2019.

[3] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature Machine Intelligence*, 2019.

[4] C. Chen, O. Li, C. Tao, A. J. Barnett, J. Su, and C. Rudin, "This Looks Like That: Deep Learning for Interpretable Image Recognition," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[5] M. Ghassemi, L. Oakden-Rayner, and A. L. Beam, "The false hope of current approaches to explainable artificial intelligence in health care," *The Lancet Digital Health*, 2021. doi: 10.1016/S2589-7500(21)00208-9.

[6] I. Y. Chen, E. Pierson, S. Rose, S. Joshi, K. Ferryman, and M. Ghassemi, "Ethical Machine Learning in Healthcare," *Annual Review of Biomedical Data Science*, 2021. doi: 10.1146/annurev-biodatasci-092820-114757.

[7] G. Clifford, C. Liu, B. Moody, et al., "AF Classification from a Short Single Lead ECG Recording: The PhysioNet/Computing in Cardiology Challenge 2017," PhysioNet Challenge, 2017. Available: https://physionet.org/challenge/2017/.

[8] G. D. Clifford, et al., "AF Classification from a Short Single Lead ECG Recording: The PhysioNet/Computing in Cardiology Challenge 2017," *Computing in Cardiology*, 2018. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC5978770/.

[9] E. Prabhakararao and S. Dandapat, "Multi-Label ECG Classification using Temporal Convolutional Neural Network," arXiv preprint, 2023. Available: https://arxiv.org/pdf/2306.03844.

[10] Y. Li, et al., "Deep learning for digitizing highly noisy paper-based ECG records," *Computers in Biology and Medicine*, 2020. Available: https://www.sciencedirect.com/science/article/pii/S001048252030408X.

[11] A. L. Goldberger, L. A. N. Amaral, L. Glass, J. M. Hausdorff, P. Ch. Ivanov, R. G. Mark, et al., "PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals," *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000.

[12] P. Wagner, N. Strodthoff, R.-D. Bousseljot, et al., "PTB-XL: A large publicly available electrocardiography dataset," *Scientific Data*, vol. 7, no. 1, 2020. Available: https://physionet.org/content/ptb-xl/1.0.3/.

[13] F. Krones, B. Walker, T. Lyons, and A. Mahdi, "Combining Hough Transform and Deep Learning Approaches to Reconstruct ECG Signals From Printouts," arXiv preprint, arXiv:2410.14185, 2024. Available: https://github.com/felixkrones/ECG-Digitiser.

[14] ECG-image-kit Developers, "ecg-image-kit: Tools for ECG image simulation and digitisation," GitHub repository, 2024. Available: https://github.com/alphanumericslab/ecg-image-kit.

[15] K. K. Shivashankara, Deepanshi, A. M. Shervedani, M. A. Reyna, G. D. Clifford, and R. Sameni, "ECG-image-kit: A synthetic image generation toolbox to facilitate deep learning-based electrocardiogram digitization," *Physiological Measurement*, 2024. doi: 10.1088/1361-6579/ad4954.

[16] G. B. Moody et al., "The George B. Moody PhysioNet Challenge 2024: ECG printout digitisation," *Computing in Cardiology*, 2024. Available: https://physionet.org/challenge/2024/.

[17] ECGtizer Developers, "ECGtizer: A Python framework for ECG PDF/image digitisation," arXiv preprint, arXiv:2412.12139, 2024. Available: https://github.com/UMMISCO/ecgtizer.

[18] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015. Available: https://arxiv.org/abs/1505.04597.

[19] F. Isensee, P. F. Jaeger, S. A. M. Kohl, J. Petersen, and K. H. Maier-Hein, "nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation," *Nature Methods*, vol. 18, pp. 203–211, 2021.

[20] X. Yu, et al., "From Paper to Digital: ECG Processing with U-Net (digitisation pipeline)," *Computing in Cardiology (CinC)*, 2024. Available: https://cinc.org/archives/2024/pdf/CinC2024-134.pdf.

[21] S. Summerton, N. Dinsdale, T. Leinonen, G. Searle, M. Kaisti, and D. C. Wong, "A Modular Framework for the Interpretation of Paper ECGs," in *Computing in Cardiology (CinC)*, vol. 51, 2024, doi: 10.22489/CinC.2024.118. Accessed: 2026-02-02. Available: https://eprints.whiterose.ac.uk/id/eprint/222013/1/CinC2024-118.pdf.

[22] ProtoECGNet Authors, "ProtoECGNet: Prototypical deep networks for interpretable ECG classification," arXiv preprint, 2023. Available: https://arxiv.org/abs/2504.08713.

[23] Frontiers in Aging Neuroscience Authors, "[Full article title corresponding to fnagi-14-841297]," *Frontiers in Aging Neuroscience*, vol. 14, article 841297, 2022. Available: https://www.frontiersin.org/files/Articles/841297/fnagi-14-841297-HTML-r2/image_m/fnagi-14-841297-g001.jpg.

[24] P. Gupta, "nnU-Net: The no-new-UNet for automatic segmentation," *MICCAI Educational Initiative (Medium)*, online article. Available: https://medium.com/miccai-educational-initiative/nnu-net-the-no-new-unet-for-automatic-segmentation-8d655f3f6d2a. (Workflow figure image URL used: https://miro.medium.com/v2/resize:fit:2000/format:webp/0*PkMBRPa77g-ICW5e.png). Accessed: 2026-02-07.

[25] M. Yazdani, "Example architecture of U-Net for producing k 256-by-256 image masks for a 256-by-256 RGB image," *Wikimedia Commons*, Aug. 5, 2019. Licensed under CC BY-SA 4.0. Available: https://commons.wikimedia.org/wiki/File:Example_architecture_of_U-Net_for_producing_k_256-by-256_image_masks_for_a_256-by-256_RGB_image.png. (Image file URL used: https://upload.wikimedia.org/wikipedia/commons/2/2b/Example_architecture_of_U-Net_for_producing_k_256-by-256_image_masks_for_a_256-by-256_RGB_image.png). Accessed: 2026-02-07.